PROBLEMAS DE SATISFAÇÃO DE RESTRIÇÕES

Prof. Wagner Igarashi, Dr.

Estrutura da Busca Padrão

- Busca em um **espaço de estados**.
- Para o algoritmo de busca, cada estado é uma **caixa preta**.
- **Estado** = Estrutura de dados que pode ser acessada somente por rotinas específicas do problema.
  - Função sucessor
  - Função heurística
  - Função de teste de objetivo

Problemas de Satisfação de Restrição (PSR)

- Este tópico descreve uma maneira de resolver uma grande variedade de problemas de forma mais eficiente.
- Tem-se uma representação fatorada para cada estado:
  - um conjunto de variáveis
- O problema será resolvido quando cada variável tiver um valor que satisfaça todas as restrições sobre a variável.

Problemas de Satisfação de Restrições (PSR)

- Estados e teste de objetivo obedecem uma representação padrão, estruturada e muito simples.

- Podem ser definidos algoritmos e função heurísticas de **propósito geral.**

- É um exemplo simples de uma linguagem de representação formal.

Problemas de Satisfação de Restrições (PSR)

- Um PSR pode ser visto como o problema que dado um conjunto de variáveis, cada uma com um conjunto de valores possíveis (um domínio), é necessário atribuir um valor para cada variável tal que:
  - Satisfaça um conjunto de restrições: problema de satisfabilidade – "restrições *hard*"
  - Minimize uma função de custo, na qual cada atribuição de valores a variáveis tem um custo: problema de otimização – "restrições *soft*"
- Muitos problemas são uma mistura de restrições *hard* e *soft*.

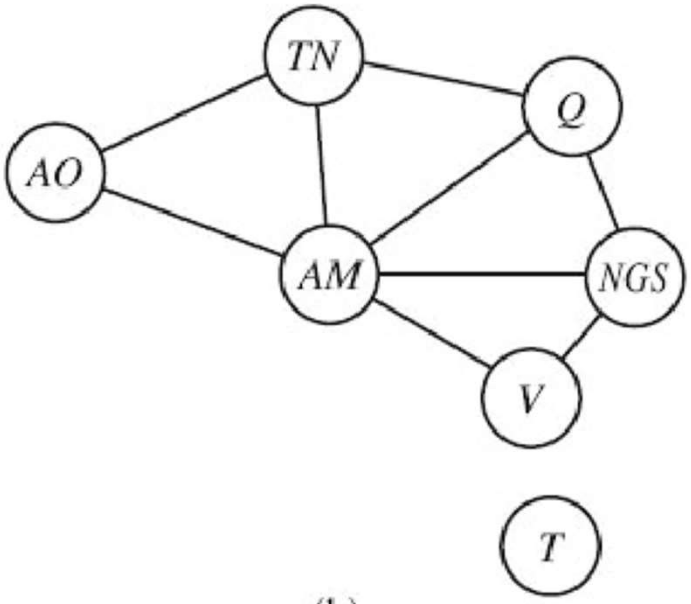
Problemas de Satisfação de Restrições (PSR)

- Um PSR consiste em:
  - Um **conjunto de variáveis** (X_i) que podem assumir um **conjunto de valores** dentro de um dado domínio (D_i).
  - Um **conjunto de restrições** que especificam propriedades da solução - valores que essas variáveis podem assumir.
    - Pode existir uma **função de avaliação** que especifica o quão bom é um conjunto de atribuições de valores para as variáveis.
  - Um **estado** do problema é definido por uma atribuição de valores a alguma ou a todas as variáveis.
  - **Atribuições**:
    - **Consistente** ou válida: que não viola nenhuma restrição
    - **Completa**: todas as variáveis são atribuídas
  - **Solução**: uma atribuição completa e válida

Exemplo: coloração de Mapas

- Queremos colorir cada região do mapa abaixo com as cores vermelho, verde e azul.
- Nenhuma região vizinha deve ter a mesma cor.

(a)



(b)

Problema da coloração de mapas como um PSR

- Variáveis - cada uma das regiões: AO, TN, Q, NGS, V, AM e T
- Domínio das variáveis: {vermelho, verde, azul} para todas
- Restrições: regiões vizinhas devem ter cores distintas
  - AO≠AM, TN≠AM, Q≠AM, NGS≠AM, V≠AM, AO≠TN, TN≠Q, Q≠NGS, NGS≠V
  - **Exemplo**: combinações possíveis para AO e TN
    - {(vermelho, verde), (vermelho, azul), (verde, vermelho), (verde, azul), (azul, vermelho), (azul, verde)}
    - Ou simplesmente AO ≠ TN
- Soluções: são atribuições que satisfazem todas as restrições

Exemplo: Atividades de escalonamento

- Variáveis: A, B, C, D, E que representam o tempo inicial de várias atividades.

- Domínios: $D_A = \{1, 2, 3, 4\}$, $D_B = \{1, 2, 3, 4\}$, $D_C = \{1, 2, 3, 4\}$, $D_D = \{1, 2, 3, 4\}$, $D_E = \{1, 2, 3, 4\}$

- Restrições:

$$(B \neq 3) \wedge (C \neq 2) \wedge (A \neq B) \wedge (B \neq C) \wedge (C < D) \wedge (A = D) \wedge (E < A) \wedge (E < B) \wedge (E < C) \wedge (E < D) \wedge (B \neq D):$$

Porque formular um problema como PSR?

- Mais fácil utilizá-lo para resolver um problema do que projetar uma solução personalizada utilizando outra técnica de busca.
- Solucionadores de PSR podem ser mais rápidos do que buscadores de espaços de estados:
  - No ex. da Austrália temos se {AM=azul},
    - Sem restrições: 5 vizinhos e 3 cores possíveis temos $3^5 = 243$ possibilidades;
    - Com restrição: nenhuma das outras 5 vizinhas poderá usar azul (AO, TN, Q, NGS, V} , ou seja, temos $2^5 = 32$ posibilidades

(a)

Tipos de Problemas de Satisfação de Restrições

- A espécie mais simples de PSR envolve variáveis de domínios discretos e finitos*:*
  - Podemos enumerar os valores de cada domínio.
    - ***Exemplo***: coloração de mapas, n-rainhas, PSRs booleanos.
- Variáveis de domínios infinitos.
  - Envolve conjuntos como números inteiros ou *strings.*
    - ***Exemplo***: escalonamento de trabalho.
  - Não é mais possível escrever restrições enumerando todas as combinações de valores permitidos.
  - É necessário empregar uma linguagem de restrições:
    - $\text{IniciarAtividade}_1 + 5 >= \text{IniciarAtividade}_3$

Tipos de Problemas de Satisfação de Restrições

- Variáveis de domínios contínuos
  - Comuns no mundo real e estudadas pela Pesquisa Operacional – Programação Linear.
    - ***Exemplo***: escalonamento de experimentos no telescópio Hubble.
- Restrições quanto à aridade (qtde. de variáveis)
  - Unárias (única variável)- ex.: a T ≠ verde
  - Binárias (duas variáveis) - ex.: AM ≠ Q
  - N-árias - ex.: TodosDiferentes – criptoaritmética, palavras cruzadas
    - Se o domínio for finito estas restrições podem ser reduzidas a restrições binárias).
- Restrições quanto à natureza
  - Absolutas (*hard*)– a solução não viola nenhuma restrição.
  - Preferenciais (*soft*)– a solução pode violar alguma restrição, mas elas devem ser satisfeitas quando possível.

Problema cripto aritmético


```
                    SOLUCAO
  FORTY             29786     com: F = 2
+  TEN                850          O = 9
+  TEN                850          R = 7 etc
--------            ------
  FIFTY             31486
```

http://www.dcc.fc.up.pt/~ines/aulas/1213/SI/outrosmetodos.pdf

Problema cripto aritmético

- estados: conjunto de letras que são substituídas por dígitos
- operadores: substituir todas as ocorrências de uma letra por um dígito que ainda não tenha aparecido
- possíveis regras de seleção dos dígitos: ordem alfabética para evitar repetições, seleção mais restrita respeitando as operações aritméticas
- estado final: jogo contém somente dígitos e representa uma soma correta

Relacionamento com a busca padrão

- O caminho até um objetivo não é importante, somente a solução é importante (semelhante a um problema de otimização).
- Não existem nós predefinidos de início.
- Geralmente estes problemas são enormes, com milhares de variáveis, o que implica que buscar sistematicamente o espaço é inviável.
- Para os problemas de otimização não existem nós objetivos bem definidos.

Benefícios em se tratar um problema de busca como um PSR

- Representação de estados obedece um **padrão definido**.
  - Conjunto de variáveis com valores atribuídos.

- Função de ação e teste de objetivo podem ser **definidos de forma genérica** para todos os PSRs.
  - *Função de ação*: atribuir um valor válido a uma variável.
  - *Teste de objetivo*: todas as variáveis com valores atribuídos.

- Heurísticas efetivas e genéricas podem ser desenvolvidas **sem nenhum conhecimento** sobre o domínio dos problemas.

- A estrutura do grafo de restrições pode ser utilizada para reduzir a complexidade do problema.

Formulação de um PSR como um problema de busca padrão

- Estado inicial: uma associação vazia {} - nenhuma variável atribuída.
- Função sucessor: atribuir um valor a uma variável desde que esta atribuição não entre em conflito com as anteriores.
- Teste de objetivo: atribuição corrente é completa?
- Custo do caminho: constante para cada passo.

Esta formulação é idêntica para qualquer PSR.

Resolução de um PSR como um problema de busca padrão

- Qualquer algoritmo de busca visto até agora pode ser utilizado.
- Toda solução aparece na **profundidade n** = número de variáveis.
- O fator de ramificação ***b*** decresce com a profundidade do nó na árvore de busca.
  - No primeiro nível b = nd, no segundo b = (n-1)d...
  - No último nível teríamos uma árvore com $\mathbf{n!d^n}$ folhas.
- Mas as atribuições são comutativas!
  - AM = verde e AO = vermelho é igual a AO = vermelho e AM = verde, não importa a ordem de atribuição.
  - Então podemos atribuir valores possíveis para uma única variável em cada nível da árvore, gerando $\mathbf{d^n}$ folhas.

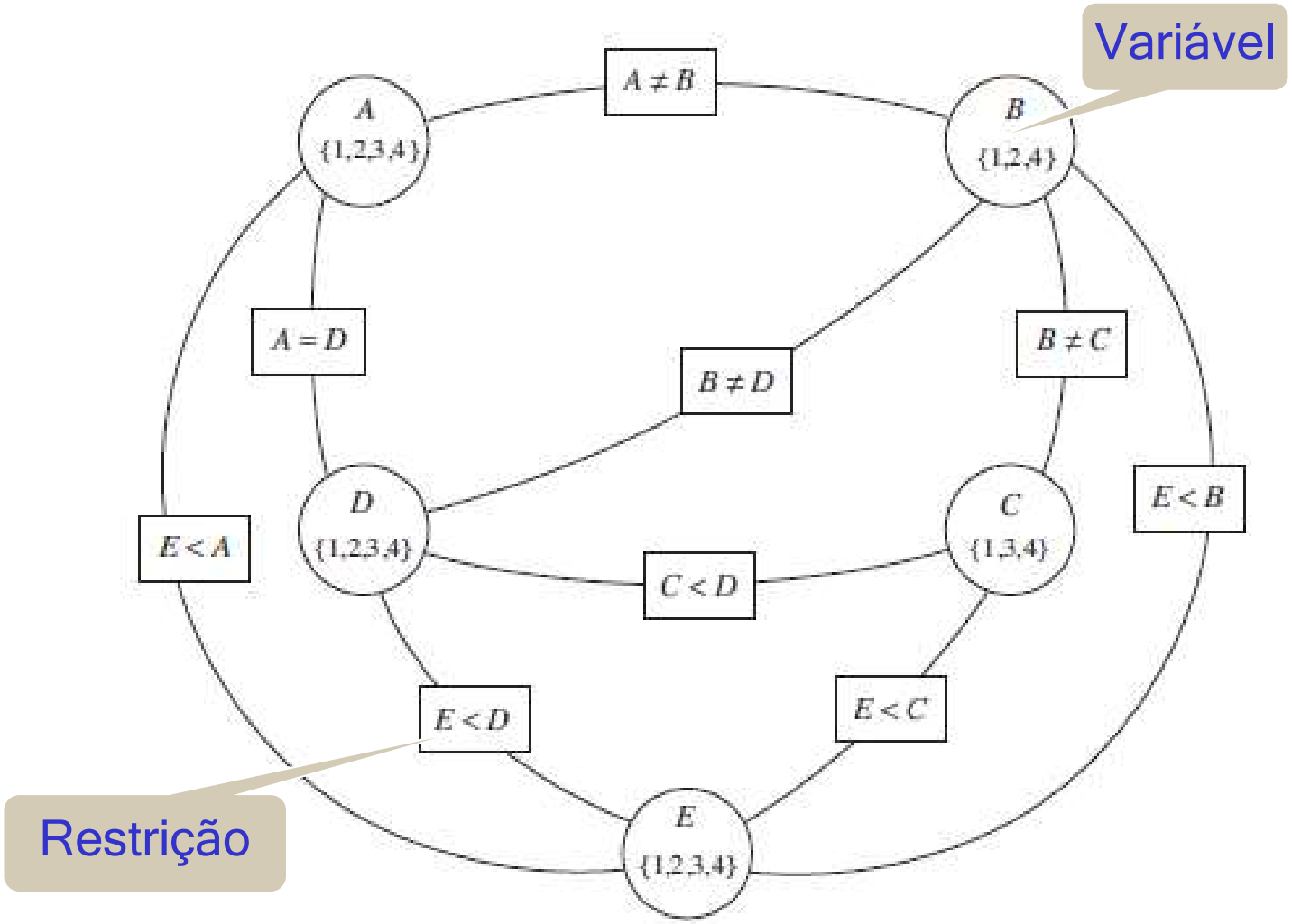
PSR como uma busca em grafo

- Um PSR pode ser representado como um algoritmo de busca em grafos:
  - Um nó é uma atribuição de valores a algumas das variáveis.
  - Suponha que o nó **N** é a atribuição $X_1 = v_1, \ldots, X_k = v_k$. Selecione uma variável **Y** que não está atribuída em **N**. Para cada valor **y** $\in$ dom(Y) existe um vizinho $X_1 = v_1, \ldots, X_k = v_k ; Y = y_i$ se esta atribuição é consistente com as restrições sobre estas variáveis.
  - O nó inicial é a atribuição vazia.
  - Um nó objetivo é uma atribuição total que satisfaz as restrições.

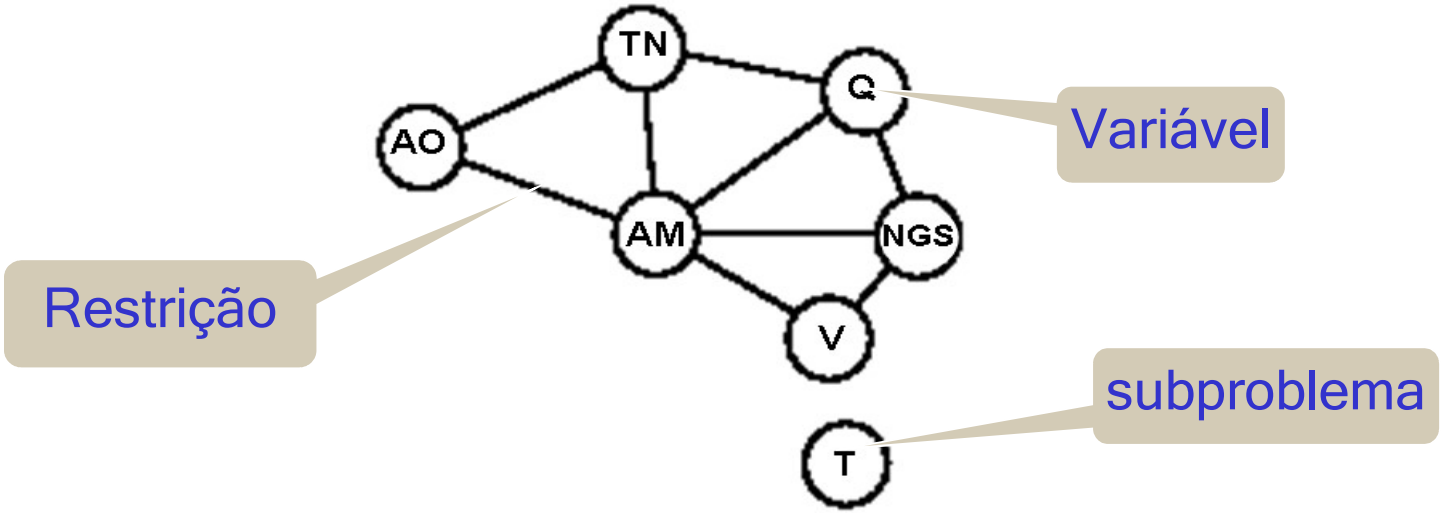
Rede de Restrições

- Existem um nó circular para cada variável.
- Existem um nó retangular para cada restrição.
- Existe um domínio de valores associado com cada nó variável.
- Existe um arco de uma variável X para cada restrição que envolve X.

Exemplo: Rede de Restrições



Exemplo: Rede de Restrições



Algoritmo de geração e teste

- Algoritmo de força bruta (ineficiente)

- Gerar o espaço de atribuição $D = D_{V1} \times D_{V2} \times \ldots \times D_{Vn}$. Testar cada atribuição com as restrições.

- **Exemplo**:

$$
\begin{aligned}
D &= D_A \times D_B \times D_C \times D_D \times D_E \\
&= \{1, 2, 3, 4\} \times \{1, 2, 3, 4\} \times \{1, 2, 3, 4\} \times \{1, 2, 3, 4\} \times \{1, 2, 3, 4\} \\
&= \{<1, 1, 1, 1, 1>, <1, 1, 1, 1, 2>, \ldots , <4, 4, 4, 4, 4>\}:
\end{aligned}
$$

- Geração e teste é sempre **exponencial no número de variáveis**.

Algoritmo de retrocesso

- Explorar sistematicamente o domínio ***D*** pela instanciação de variáveis, uma de cada vez.

- Avaliar cada predicado de restrição tão logo todas suas variáveis estejam ligadas.

- Qualquer atribuição parcial que não satisfaz a restrição pode ser podada.

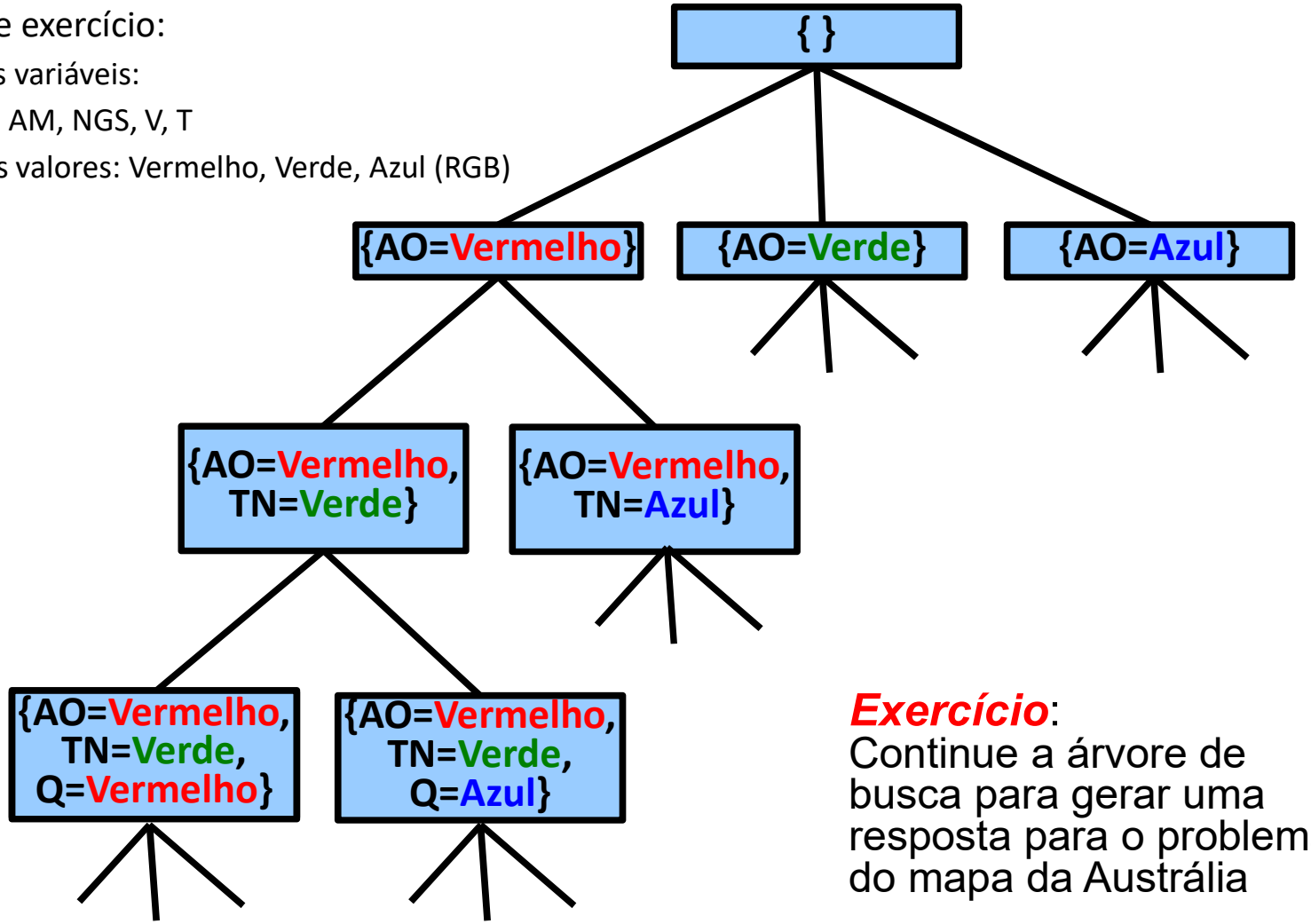
- A atribuição exemplo A = 1 ^ B = 1 é inconsistente com a restrição A ≠ B independente do valor das outras variáveis.

Busca com retrocesso para PSRs

- Busca em profundidade com atribuição simples de variáveis (uma em cada nível da árvore).
  - É um algoritmo uniformizado básico para PSRs.
- Explora o espaço de associações sistematicamente.
  - Associa as variáveis aos seus valores em alguma ordem e avalia as restrições que envolvem as variáveis já atribuídas.
  - Qualquer associação que não satisfaça as restrições pode ser podada
- Exemplo:
  - Associar AO=R e TN=R viola a restrição AO≠TN, independente dos valores das outras variáveis
- Pode resolver n-rainhas para n ≈ 25

Busca com retrocesso para problema do mapa da Austrália

- Para efeito de exercício:
  - Ordem das variáveis:
  - AO, TN, Q, AM, NGS, V, T
  - Ordem dos valores: Vermelho, Verde, Azul (RGB)



Exercício:
Continue a árvore de busca para gerar uma resposta para o problema do mapa da Austrália

Melhorando a eficiência do algoritmo com retrocesso

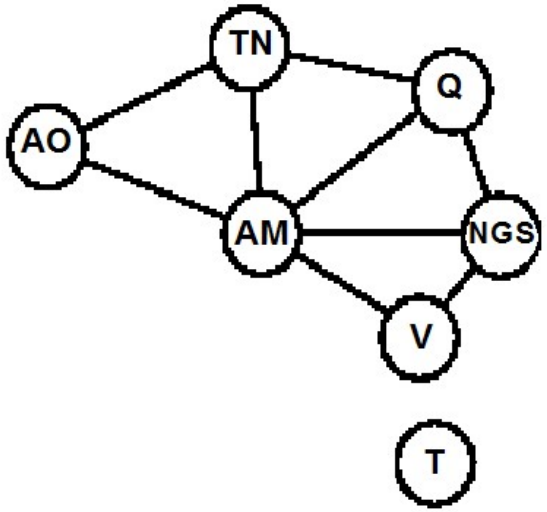
- Heurísticas de propósito geral podem obter imensos ganhos em velocidade.
- Ideia: tentar gerar uma falha logo no início da busca, podando os galhos que não terão sucesso
  - Em que ordem atribuir as variáveis?
  - Em que ordem testar os valores?
  - Pode-se detectar falhas inevitáveis antes do início da busca?
  - Podemos tirar vantagens da estrutura do problema?
- R: Pode-se usar as várias abordagens em conjunto.

Em que ordem testar as variáveis?

- **Heurística de variável mais restrita (valores restantes mínimos** (VRM) – reduzir o número de nós.
  - Testar as variáveis em ordem crescente do tamanho do domínio.

1 2 4 3

Variáveis =>	AO	TN	Q	AM	NGS	V	T
Domínio =>	RGB	RGB	RGB	RGB	RGB	RGB	RGB
AO = R	R	GB	RGB	GB	RGB	RGB	RGB
TN = G		G	RB	B	RGB	RGB	RGB
AM = B			R	B	RG	RG	RGB
Q = R			R		G	RG	RGB
NGS = G					G	R	RGB
V = R						R	RGB
T = R							R



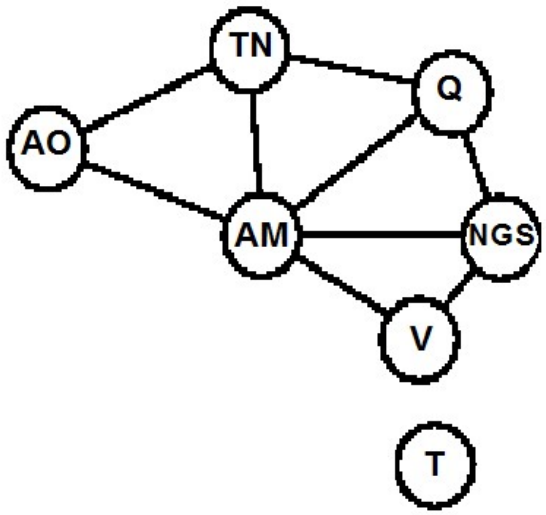
1 - começa de alguma variável (normalmente a primeira);
2, 3 ... - a sequencia vai pela variável mais **restrita** (de domínio)**.**

Em que ordem testar as variáveis?

- **Heurística de grau (de restrição)** – qual variável será escolhida primeiro.
  - Testar as variáveis em ordem decrescente pela quantidade de restrições que cada variável está envolvida.

2 3 1 4

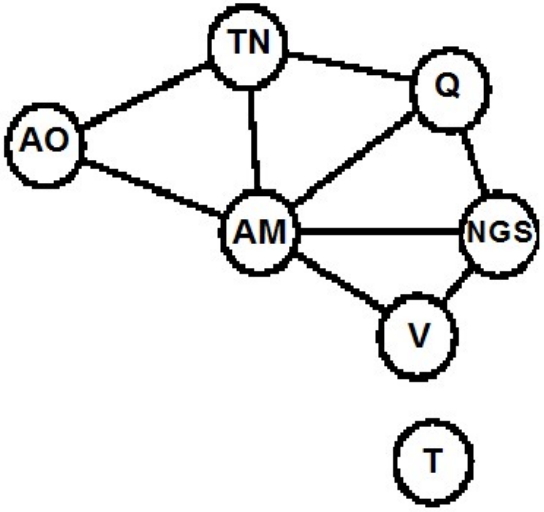
Variáveis =>	AO (2)	TN (3)	Q (3)	AM (5)	NGS (3)	V (2)	T (0)
Domínio =>	RGB	RGB	RGB	RGB	RGB	RGB	RGB
AM = R	GB	GB	GB	R	GB	GB	RGB
TN = G	B	G	B		GB	GB	RGB
Q = B	B		B		G	GB	RGB
NGS = G	B				G	B	RGB
AO = B	B					B	RGB
V = B						B	RGB
							R



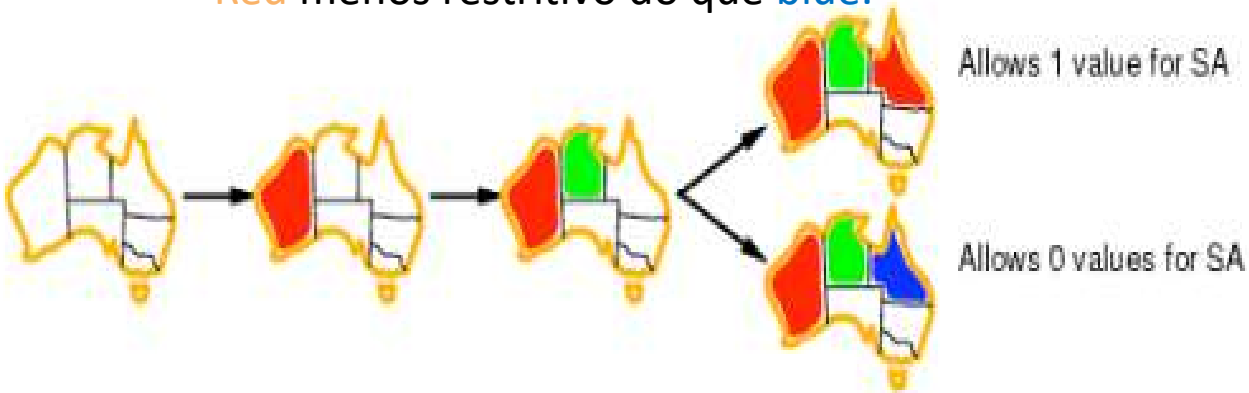
Em que ordem testar os valores?

- **Heurística de valor menos restritivo** (VMR) – para não gerar uma falha precoce.
  - Testar primeiro os valores que restringem menos as atribuições futuras (de seus vizinhos imediatos).

Variáveis =>	AO	TN	Q	AM	NGS	V	T
Domínio =>	**RGB**	RGB	RGB	RGB	RGB	RGB	RGB
AO = R	R	**GB**	RGB	GB	RGB	RGB	RGB
TN = G		G	**RB**	B	RGB	RGB	RGB
Q = R			R	**B**	GB	RGB	RGB
AM = B				B	**GB**	RGB	RGB
NGS = G					G	RGB	RGB
V = R						R	**RGB**
T = R							R

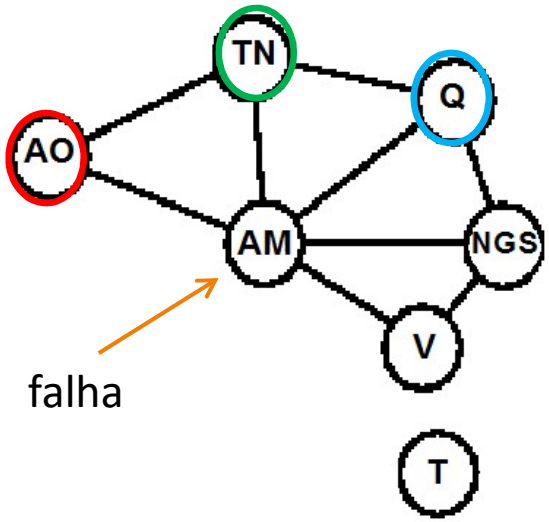


Red menos restritivo do que blue.



Fonte: https://fenix.tecnico.ulisboa.pt/downloadFile/3779572071285/cap5-restricoes.pdf

- AO = red e TN = green
- Q = blue or red?
  - mas se Q = blue restringe o vizinho AM a não poder usar nenhuma cor e o algoritmo falha.

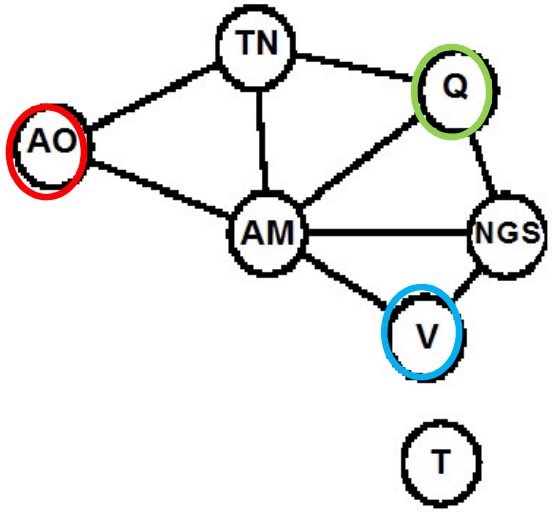


- então o algoritmo escolhe Q = red.

Utilizando verificação à frente e VRM

- **Verificação à frente**
  - Eliminar dos domínios das variáveis que estão envolvidas nas restrições os valores que não poderão mais serem utilizados devido às atribuições anteriores.

	AO	TN	Q	NGS	V	AM	T
DOMÍNIO	RGB	RGB	RGB	RGB	RGB	RGB	RGB
AO=R	**R**	GB	RGB	RGB	RGB	GB	RGB
Q=G	**R**	**B**	**G**	R B	RGB	**B**	RGB
V=B	**R**	**B**	**G**	**R**	**B**		RGB



- Verificação à frente não propaga todas as restrições.

Pode-se detectar falhas inevitáveis antes do início da busca?

- **Algoritmos de consistência de arco**
  - Tentam propagar as restrições até níveis mais profundos do que a verificação prévia pode chegar.
  - Trabalham sobre uma rede de restrições que representa o PSR.

Consistência de Arco

- Ideia: podar os domínios o máximo possível antes de selecionar valores deles.
- Um variável é **consistente no domínio** se nenhum valor do domínio do nó é tornado impossível por qualquer das restrições.
- Exemplo:
  - $D_B = \{1, 2, 3, 4\}$ não é consistente no domínio pois B = 3 viola a restrição B $\neq$ 3.

Consistência de Arco

relação

- Um arco <X, r(X, $\bar{Y}$)> é arco consistente se, para cada valor x ∈ dom(X), existe algum valor $\bar{y}$ ∈ dom($\bar{Y}$) tal que r(x, $\bar{y}$) é satisfeita.

- Uma **rede de restrições** é arco consistente se todos seus arcos são consistentes.

- Se um arco <X, r(X, $\bar{Y}$)> não é arco consistente, todos os valores de X no dom(X) para o qual não há valor correspondente no dom($\bar{Y}$) podem ser apagados do dom(X) para tornar o arco <X, r(X, $\bar{Y}$)> consistente.

Algoritmo de Consistência de arco

- Os arcos podem ser analisados um por vez tornando cada arco consistente.
- Um arco <X, r(X, *Y*)> precisa ser revisitado se o domínio de um dos *Y* for reduzido.
- Existem três possíveis resultados (quando todos os arcos forem consistentes):
  - Um **domínio está vazio** → Não há solução
  - Cada **domínio tem um valor único** → solução única
  - Alguns domínios tem mais de um valor → pode ou não existir uma solução

1-Consistência

- 1-Consistência ou consistência de domínio ou consistência de nó: uma variável é domínio consistente se nenhum valor do domínio é impossível por qualquer uma das restrições
  - Exemplo:
    - $D_x = \{1, 2, 3\}$ e a restrição $X \neq 2$
    - X não é domínio consistente pois o valor 2 de D_x viola a restrição
      - Para tornar D_x domínio consistente temos que retirar o valor 2 de D_x
      - $D_x = \{1, 3\}$ torna x domínio consistente

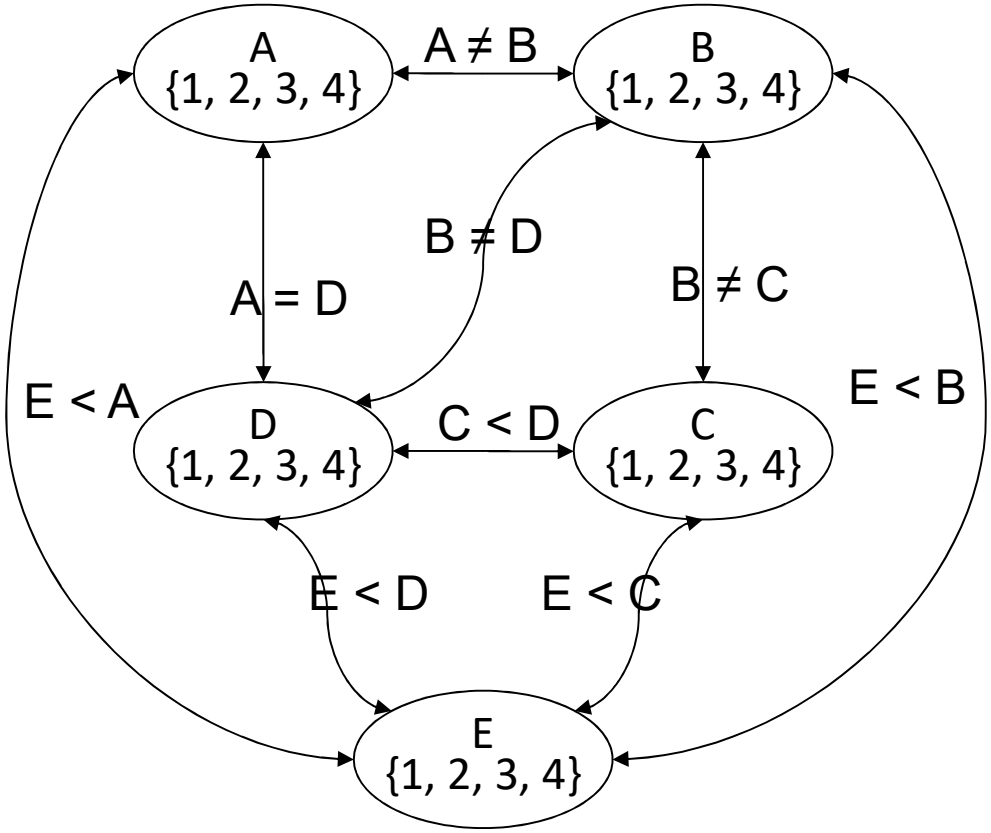
2-Consistência

- 2-Consistência ou arco consistência: Um arco <X, Y> é um arco consistente se para cada valor de X em D_X, existe algum valor de Y em D_y, para o qual a restrição P(X, Y) é satisfeita
  - Exemplo:
    - D_x = {1, 2, 3}, D_y ={2} e a restrição X ≠ Y
    - O arco <Y, X> é consistente, pois existe pelo menos um valor em X diferente de 2 em Y
    - O arco <X, Y> não é consistente, pois não existe nenhum valor em Y diferente de 2 em X
      - Para tornar o arco <X, Y> consistente devemos retirar o valor 2 de D_x
      - D_x = {1, 3} torna o arco <X, Y> consistente

Exemplo de PSR para escalonamento de atividades

- Variáveis: A, B, C, D e E que representam o tempo de início das várias atividades
- Domínios: $D_A = D_B = D_C = D_D = D_E = \{1, 2, 3, 4\}$
- Restrições:
  - $(B \neq 3) \wedge (C \neq 2) \wedge (A \neq B) \wedge (B \neq C) \wedge (B \neq D) \wedge (C < D) \wedge (A = D) \wedge (E < A) \wedge (E < B) \wedge (E < C) \wedge (E < D)$

Rede de restrições para o problema de escalonamento de atividades



$(B \neq 3) \wedge (C \neq 2) \wedge (A \neq B) \wedge (B \neq C) \wedge (B \neq D) \wedge (C < D) \wedge (A = D) \wedge (E < A) \wedge (E < B) \wedge (E < C) \wedge (E < D)$

Algoritmo de consistência de arcos AC-3

- Batizado pelo autor como AC-3 por ser a terceira versão do algoritmo.

1. Se um arco <X,Y> não é arco consistente
   - todos os valores de X em D_X para os quais não existe valor correspondente em D_y podem ser apagados de D_X para fazer <X,Y> arco consistente.
2. Os arcos são considerados um a um tornando cada um deles consistente.
3. Um arco <X,Y> necessita ser revisado se o domínio de Y for reduzido.

Entrada:

- um conjunto de variáveis
- um domínio Dx para cada variável X
- relações Px sobre a variável X que deve ser satisfeita
- relações Pxy sobre as variáveis X e Y que deve ser satisfeita

Saída:

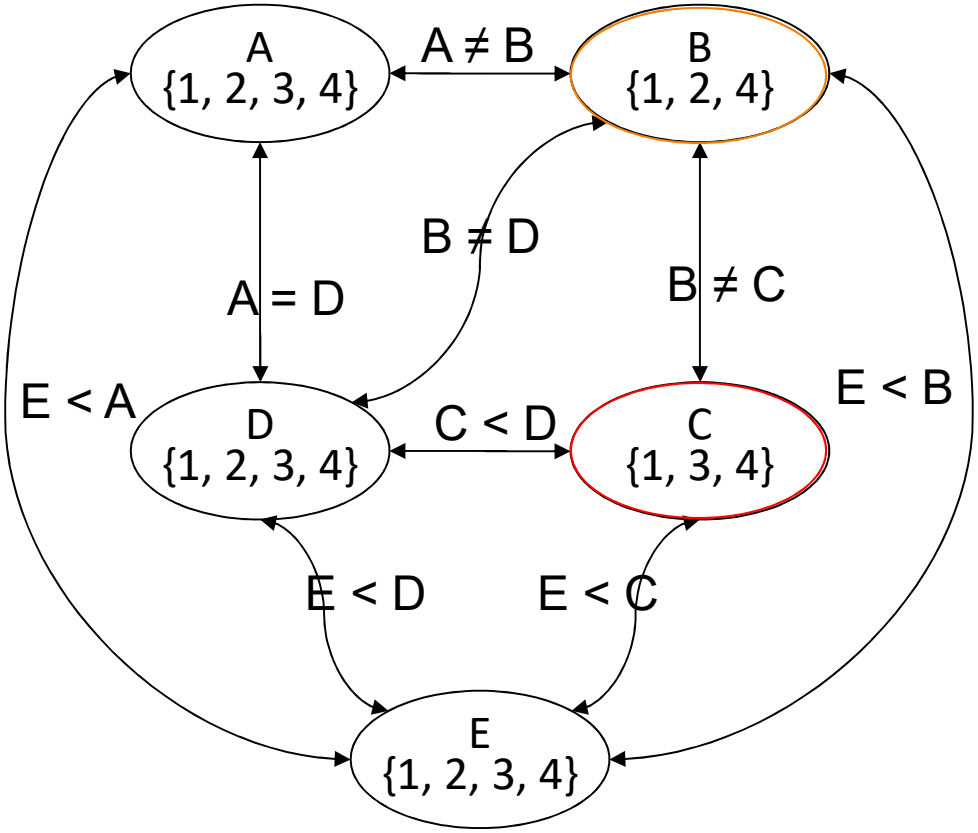
- domínios arco consistentes para cada variável

```
para cada variável X
   torne o Dx consistente de acordo com Px
TDA = conjunto de arcos a fazer - para cada Pxy, <X, Y> e <Y,
   X> fazem parte do conjunto
repita
   selecione qualquer arco de TDA;
   exclua este arco de TDA;
   retire todo x do Dx que não satisfaça Pxy
   se o Dx foi alterado
      então todos os outros arcos <Z, X> devem ser inseridos em
        TDA
até que TDA seja vazio
```

Algoritmo de consistência de arcos AC-3

Tornando as variáveis domínio consistentes

- Px:
  - B ≠ 3
  - C ≠ 2



Entrada:

- um conjunto de variáveis
- um domínio Dx para cada variável X
- relações Px sobre a variável X que deve ser satisfeita
- relações Pxy sobre as variáveis X e Y que deve ser satisfeita

Saída:

- domínios arco consistentes para cada variável

para cada variável X

torne o Dx consistente de acordo com Px

TDA = conjunto de arcos a fazer – para cada Pxy, <X, Y> e <Y, X> fazem parte do conjunto

repita

selecione qualquer arco de TDA;

exclua este arco de TDA;

retire todo x do Dx que não satisfaça Pxy

se o Dx foi alterado

então todos os outros arcos <Z, X> devem ser inseridos em TDA

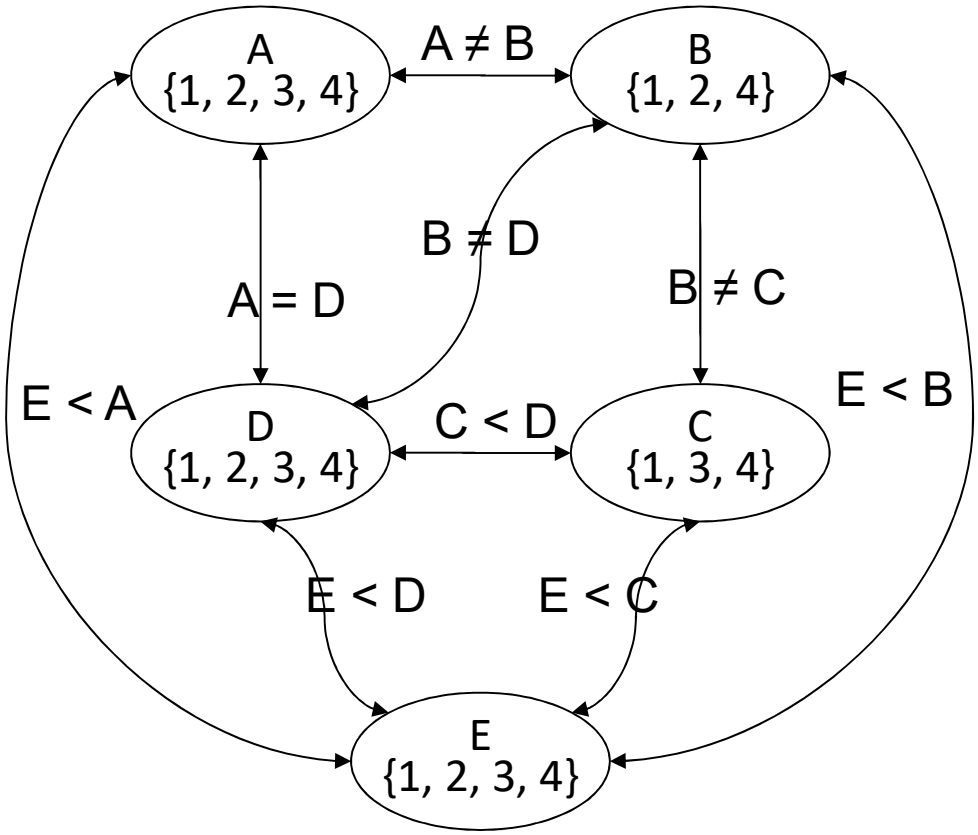
até que TDA seja vazio

Algoritmo de consistência de arcos AC-3

Construção da lista de arcos TDA (*To-Do-Arcs*)

- TDA
  - <A, B>
  - <B, A>
  - <A, D>
  - <D, A>
  - <A, E>
  - <E, A>
  - <B, D>
  - <D, B>
  - <B, C>
  - <C, B>
  - <B, E>
  - <E, B>
  - <D, C>
  - <C, D>
  - <D, E>
  - <E, D>
  - <C, E>
  - <E, C>

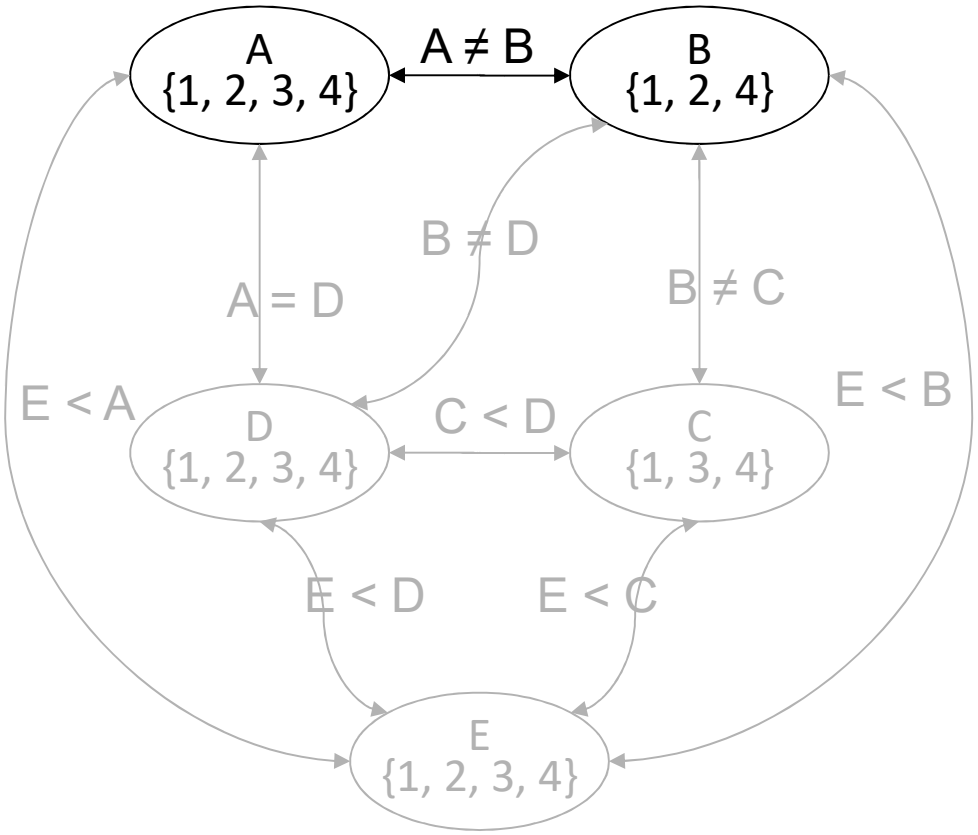
Lista (não rep.)
(todos os arcos)



Algoritmo de consistência de arcos AC-3

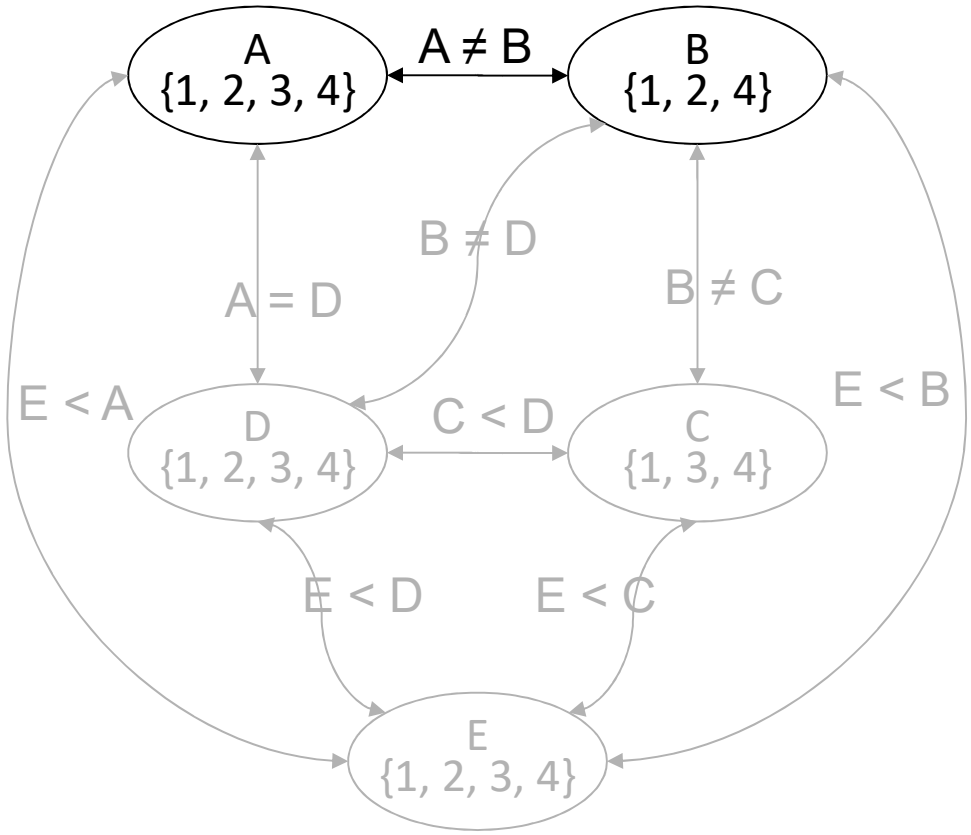
Tornando o grafo arco consistente

- TDA
  - **<A, B>**
  - <B, A>
  - <A, D>
  - <D, A>
  - <A, E>
  - <E, A>
  - <B, D>
  - <D, B>
  - <B, C>
  - <C, B>
  - <B, E>
  - <E, B>
  - <D, C>
  - <C, D>
  - <D, E>
  - <E, D>
  - <C, E>
  - <E, C>



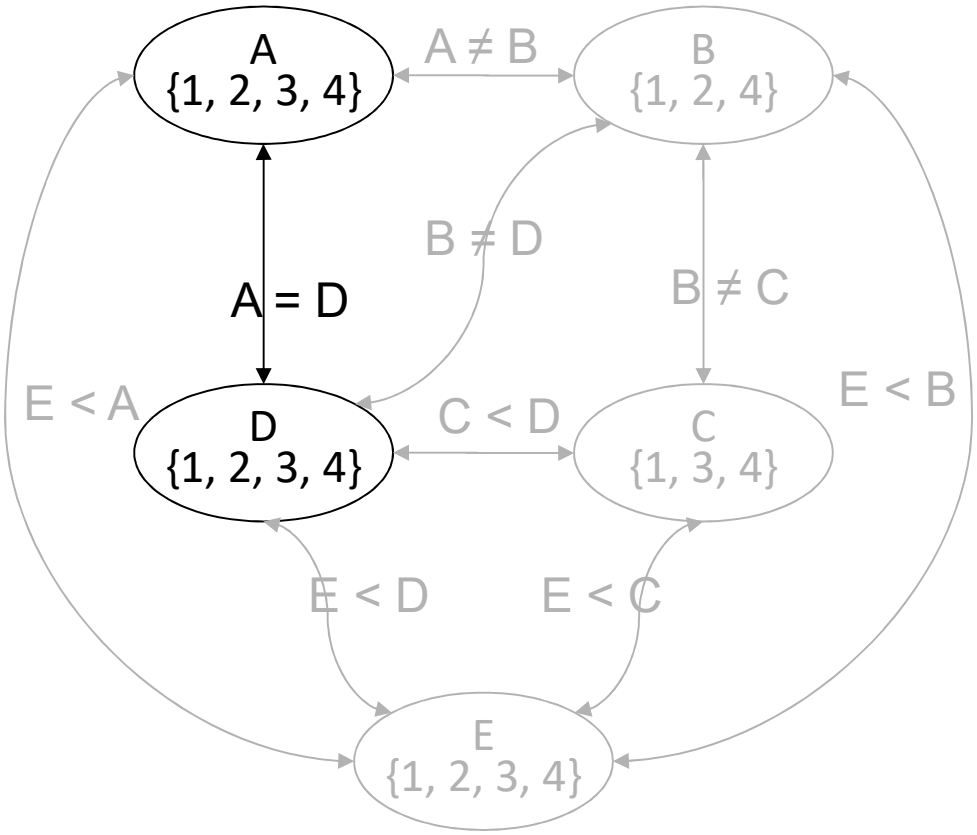
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <A, B> OK
  - **<B, A>**
  - <A, D>
  - <D, A>
  - <A, E>
  - <E, A>
  - <B, D>
  - <D, B>
  - <B, C>
  - <C, B>
  - <B, E>
  - <E, B>
  - <D, C>
  - <C, D>
  - <D, E>
  - <E, D>
  - <C, E>
  - <E, C>



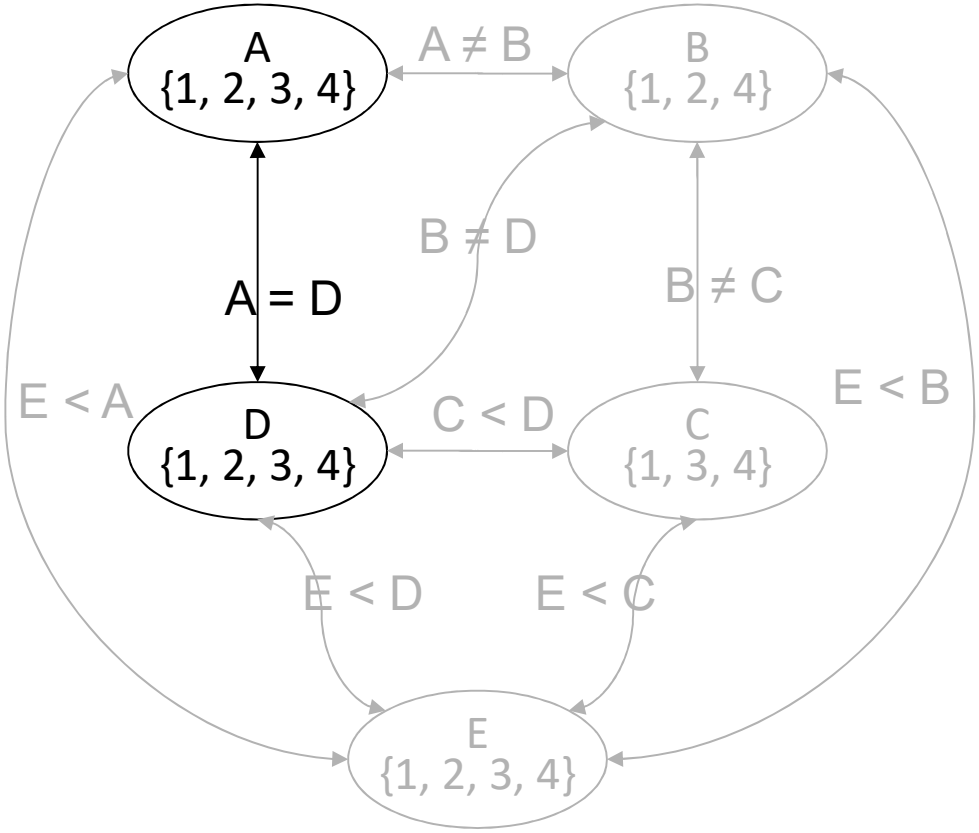
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <A, B> Ok
  - ✓ <B, A> Ok
  - **<A, D>**
  - <D, A>
  - <A, E>
  - <E, A>
  - <B, D>
  - <D, B>
  - <B, C>
  - <C, B>
  - <B, E>
  - <E, B>
  - <D, C>
  - <C, D>
  - <D, E>
  - <E, D>
  - <C, E>
  - <E, C>



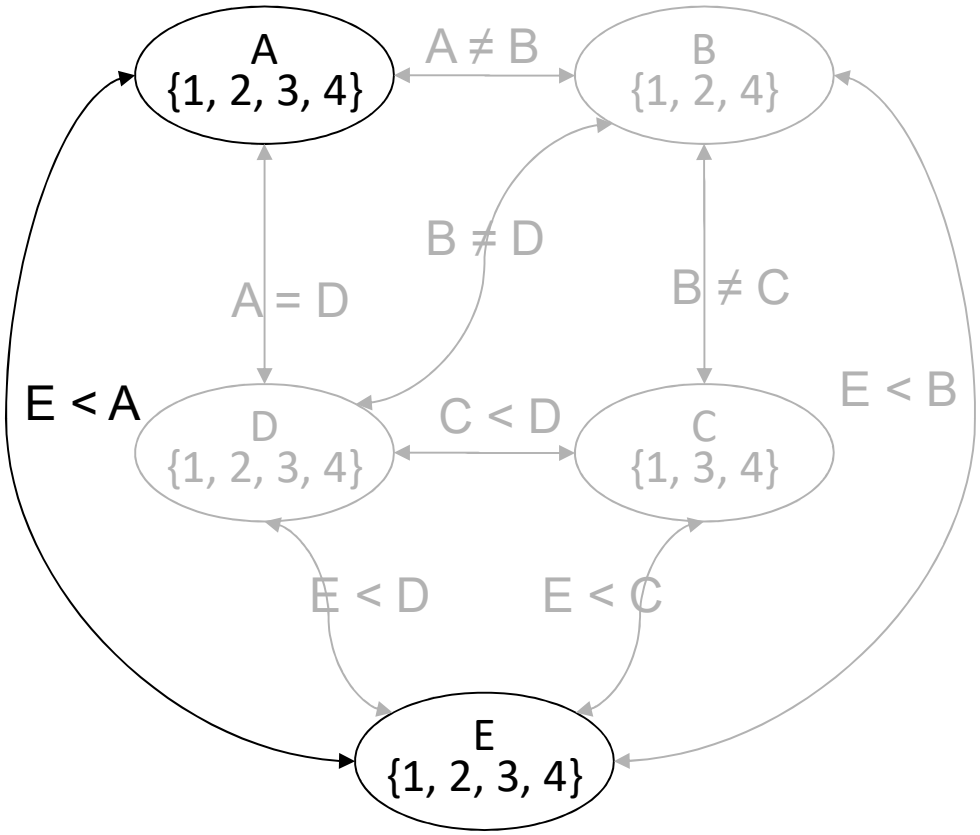
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <A, B> Ok
  - ✓ <B, A> Ok
  - ✓ <A, D> Ok
  - **<D, A>**
  - <A, E>
  - <E, A>
  - <B, D>
  - <D, B>
  - <B, C>
  - <C, B>
  - <B, E>
  - <E, B>
  - <D, C>
  - <C, D>
  - <D, E>
  - <E, D>
  - <C, E>
  - <E, C>



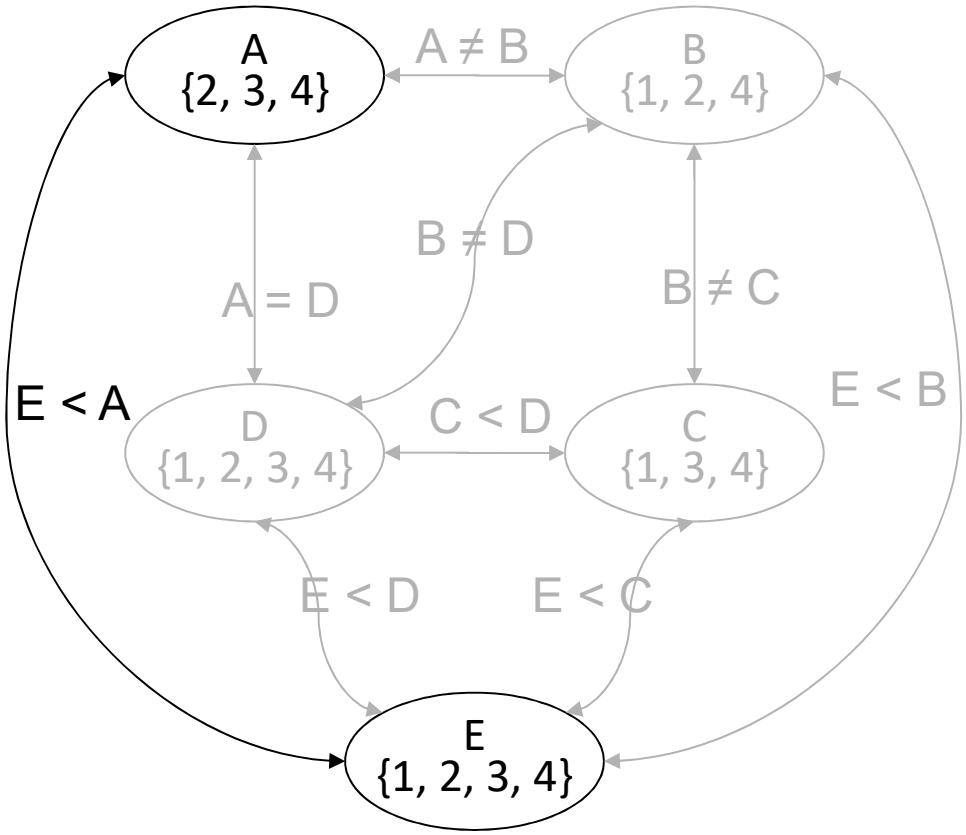
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, A> Ok
  - **<A, E>**
  - <E, A>
  - <B, D>
  - <D, B>
  - <B, C>
  - <C, B>
  - <B, E>
  - <E, B>
  - <D, C>
  - <C, D>
  - <D, E>
  - <E, D>
  - <C, E>
  - <E, C>



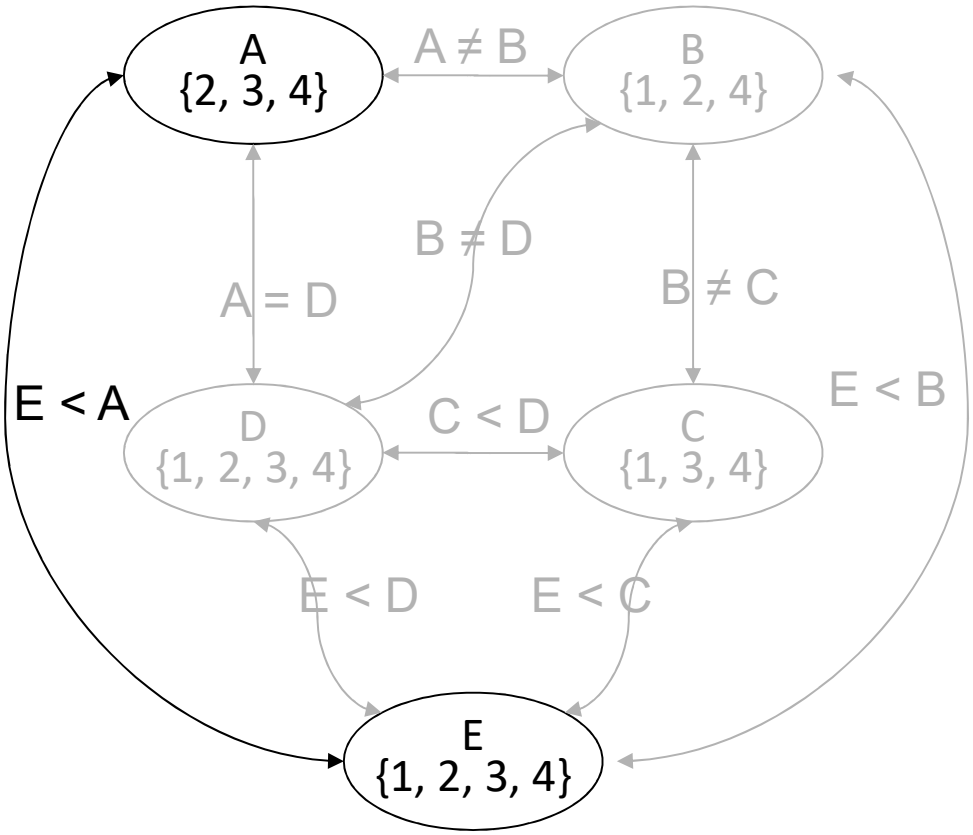
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, A> Ok
  - **<A, E> retira 1 de A insere os arcos <B, A>, <D, A>**
  - <E, A>
  - <B, D>
  - <D, B>
  - <B, C>
  - <C, B>
  - <B, E>
  - <E, B>
  - <D, C>
  - <C, D>
  - <D, E>
  - <E, D>
  - <C, E>
  - <E, C>
  - **<B, A>**
  - **<D, A>**



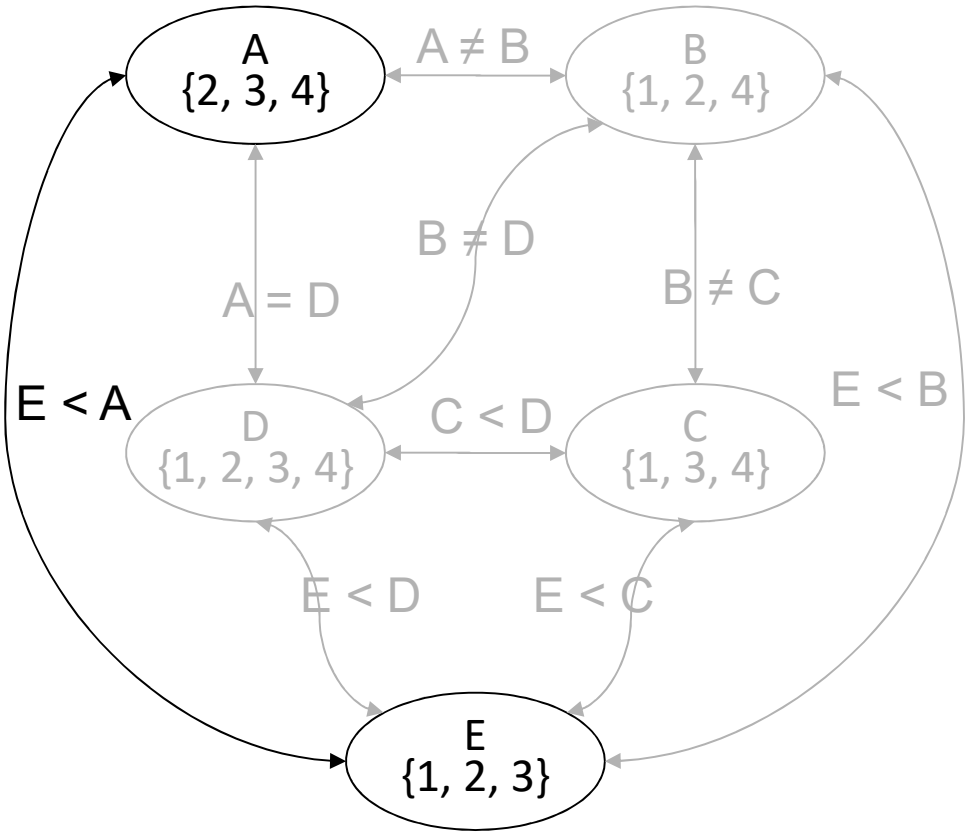
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <A, E> retira 1 de A insere os arcos <B, A>, <D, A>
  - **<E, A>**
  - <B, D>
  - <D, B>
  - <B, C>
  - <C, B>
  - <B, E>
  - <E, B>
  - <D, C>
  - <C, D>
  - <D, E>
  - <E, D>
  - <C, E>
  - <E, C>
  - <B, A>
  - <D, A>



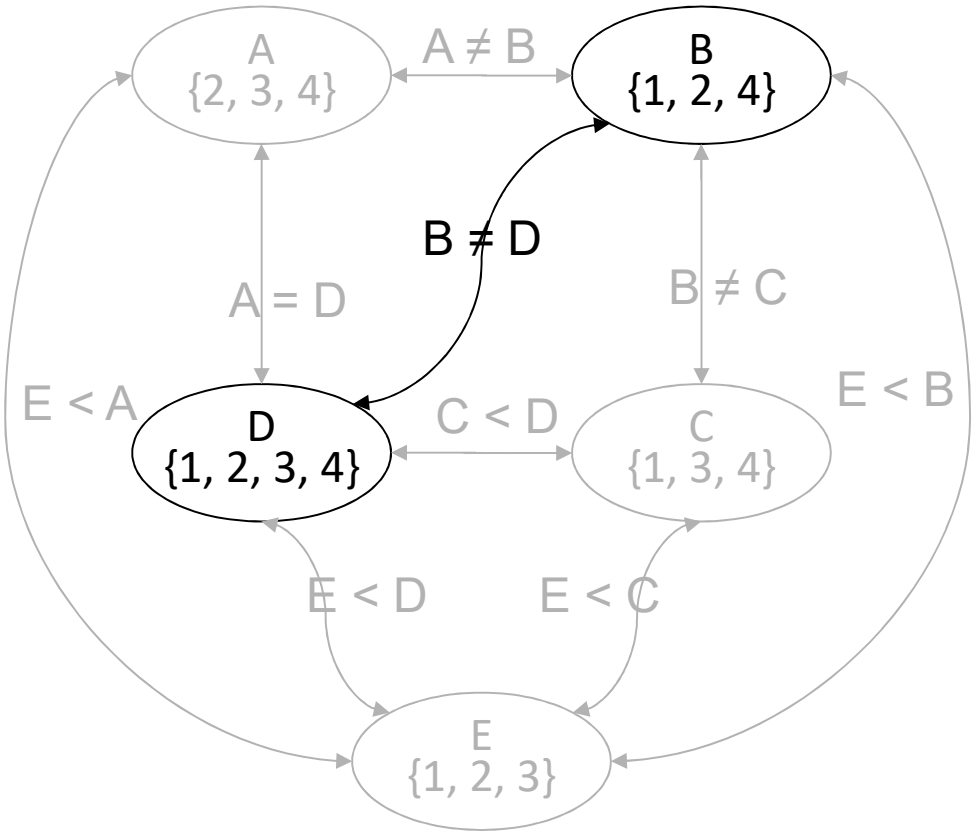
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <A, E> retira 1 de A insere os arcos <B, A>, <D, A>
  - **<E, A> retira 4 de E insere os arcos <B, E>, <C, E> e <D, E>**
  - <B, D>
  - <D, B>
  - <B, C>
  - <C, B>
  - **<B, E>***
  - <E, B>
  - <D, C>
  - <C, D>
  - **<D, E>***
  - <E, D>
  - **<C, E>***
  - <E, C>
  - <B, A>
  - <D, A>



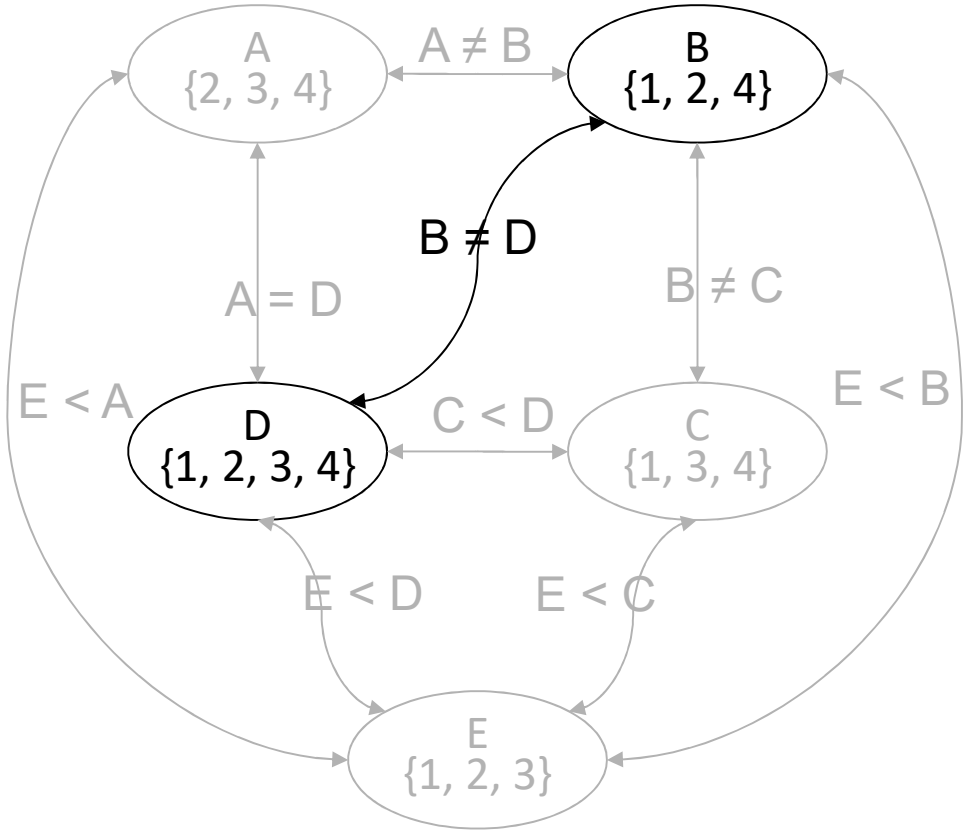
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <A, E> retira 1 de A insere os arcos <B, A>, <D, A>
  - ✓ <E, A> retira 4 de E insere os arcos <B, E>, <C, E> e <D, E>
  - **<B, D>**
  - <D, B>
  - <B, C>
  - <C, B>
  - <B, E>
  - <E, B>
  - <D, C>
  - <C, D>
  - <D, E>
  - <E, D>
  - <C, E>
  - <E, C>
  - <B, A>
  - <D, A>



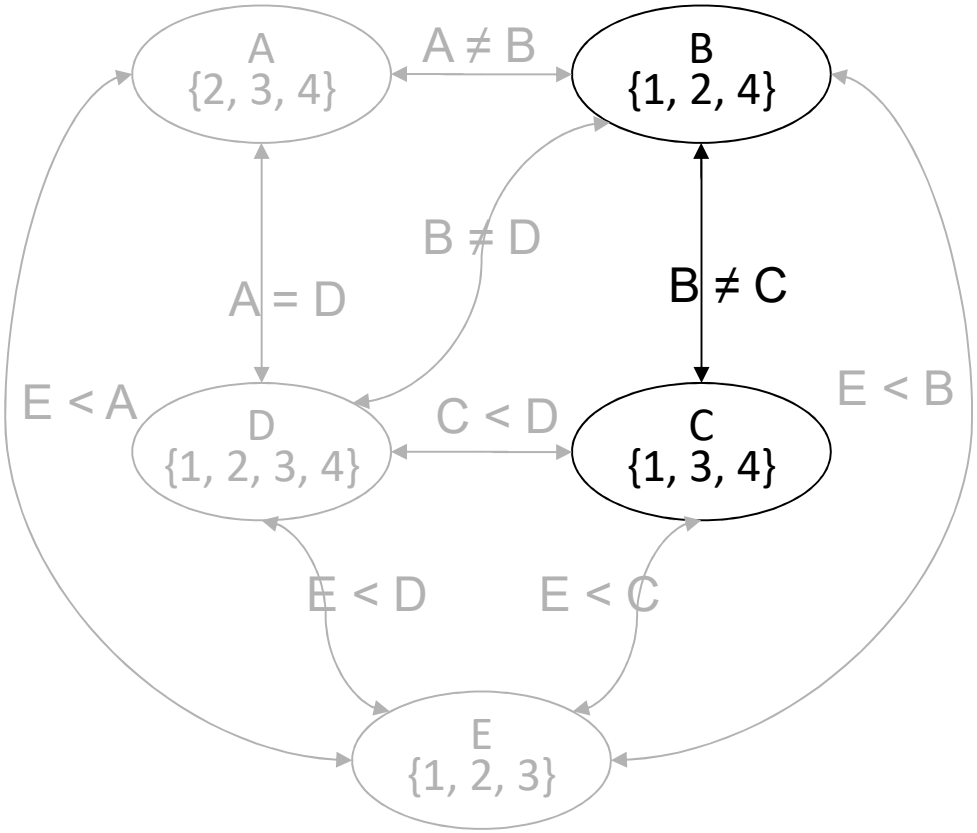
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <A, E> retira 1 de A insere os arcos <B, A>, <D, A>
  - ✓ <E, A> retira 4 de E insere os arcos <B, E>, <C, E> e <D, E>
  - ✓ <B, D> Ok
  - **<D, B>**
  - <B, C>
  - <C, B>
  - <B, E>
  - <E, B>
  - <D, C>
  - <C, D>
  - <D, E>
  - <E, D>
  - <C, E>
  - <E, C>
  - <B, A>
  - <D, A>



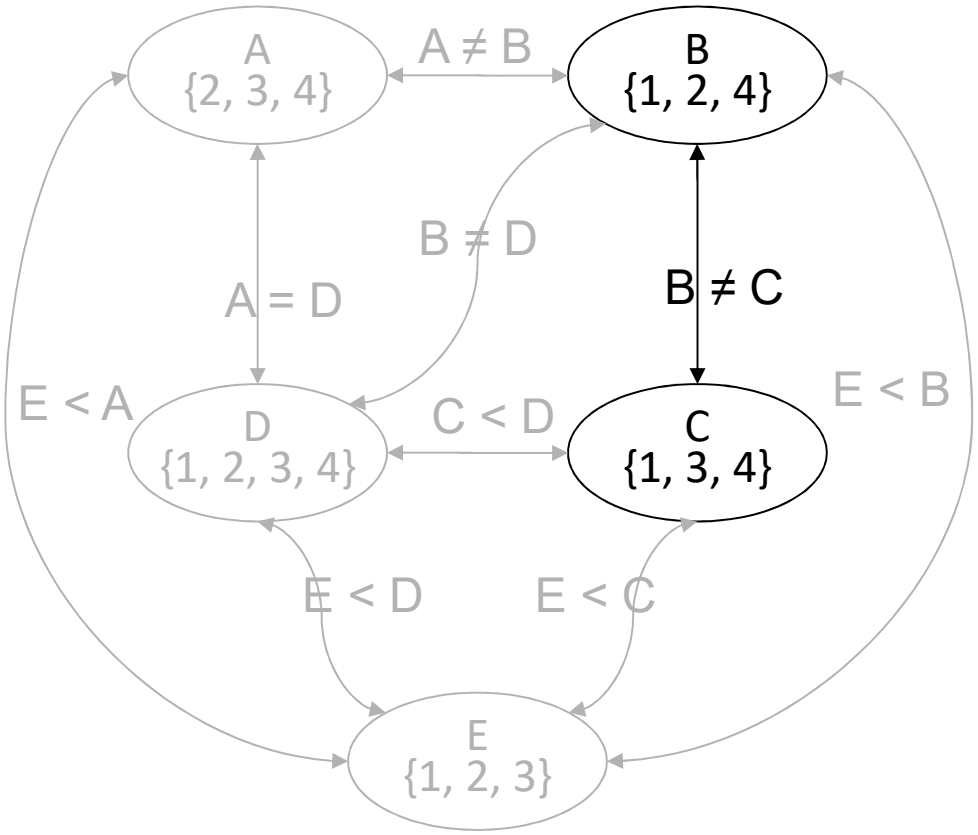
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <A, E> retira 1 de A insere os arcos <B, A>, <D, A>
  - ✓ <E, A> retira 4 de E insere os arcos <B, E>, <C, E> e <D, E>
  - ✓ <B, D> Ok
  - ✓ <D, B> Ok
  - **<B, C>**
  - <C, B>
  - <B, E>
  - <E, B>
  - <D, C>
  - <C, D>
  - <D, E>
  - <E, D>
  - <C, E>
  - <E, C>
  - <B, A>
  - <D, A>



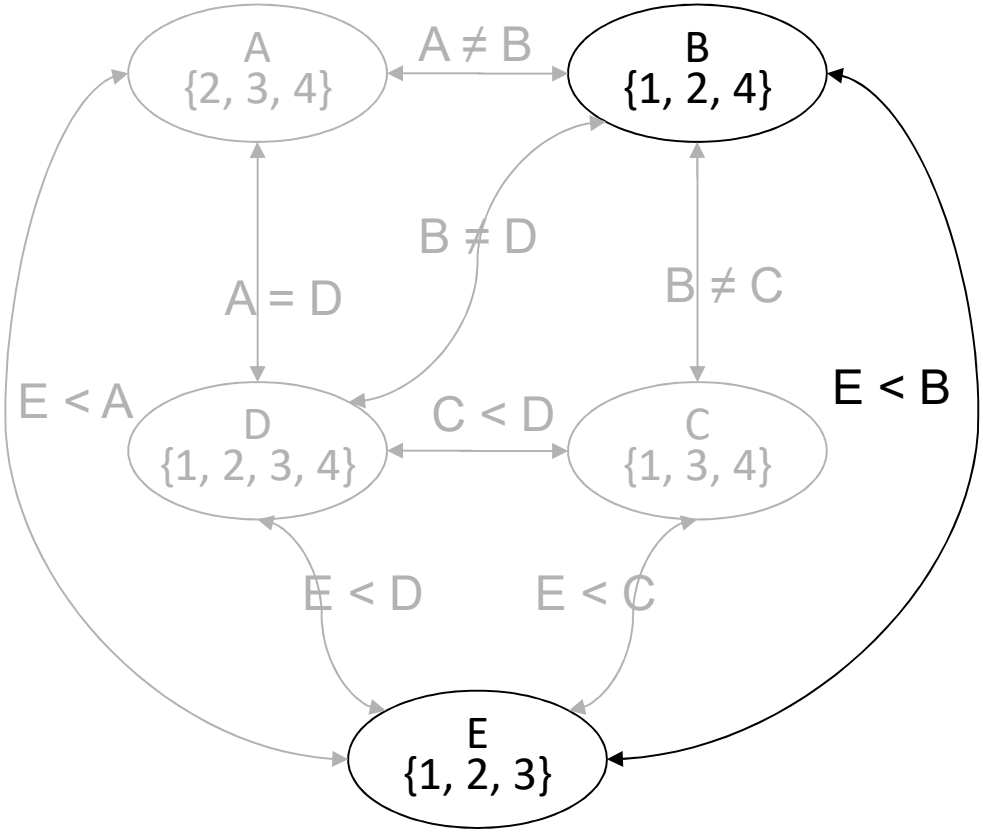
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <A, E> retira 1 de A insere os arcos <B, A>, <D, A>
  - ✓ <E, A> retira 4 de E insere os arcos <B, E>, <C, E> e <D, E>
  - ✓ <B, D> Ok
  - ✓ <D, B> Ok
  - ✓ <B, C> Ok
  - **<C, B>**
  - <B, E>
  - <E, B>
  - <D, C>
  - <C, D>
  - <D, E>
  - <E, D>
  - <C, E>
  - <E, C>
  - <B, A>
  - <D, A>



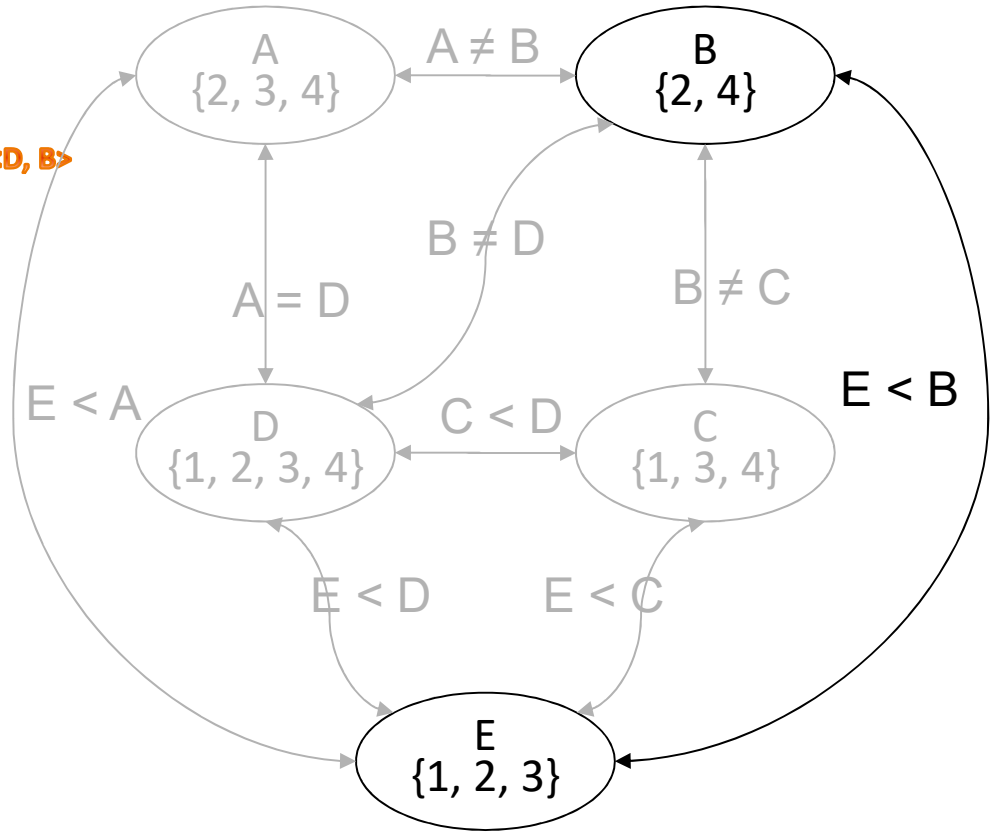
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <B, D> Ok
  - ✓ <D, B> Ok
  - ✓ <B, C> Ok
  - ✓ <C, B> Ok
  - **<B, E>**
  - <E, B>
  - <D, C>
  - <C, D>
  - <D, E>
  - <E, D>
  - <C, E>
  - <E, C>
  - <B, A>
  - <D, A>



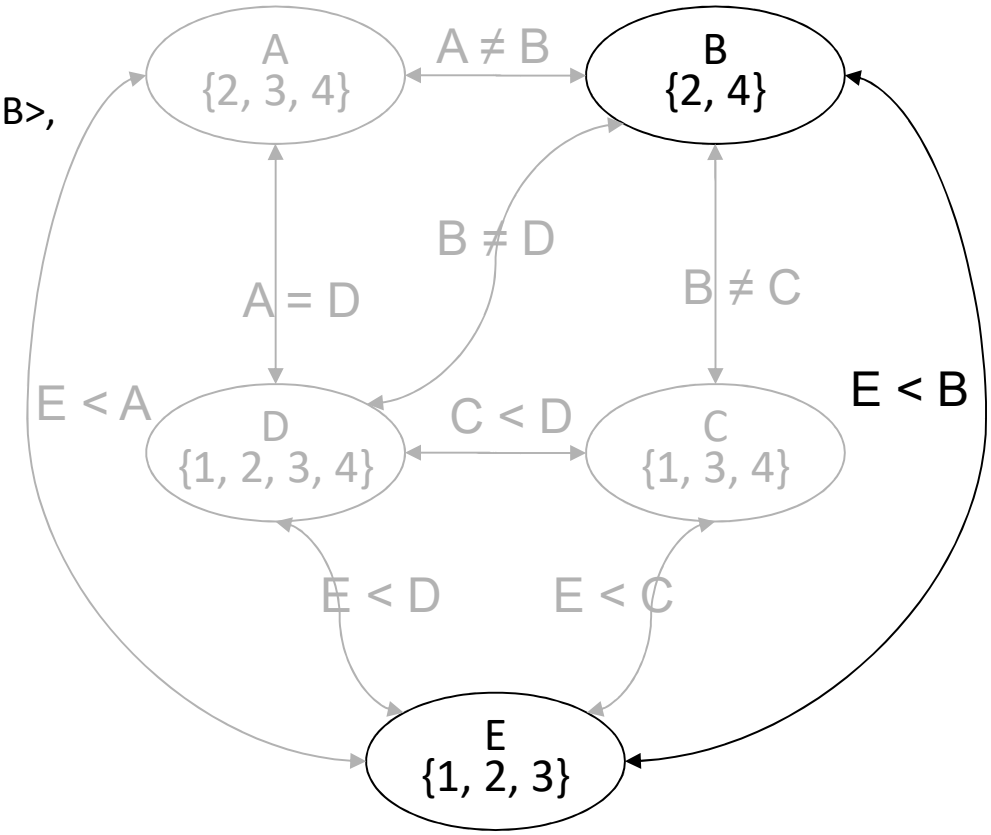
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <B, D> Ok
  - ✓ <D, B> Ok
  - ✓ <B, C> Ok
  - ✓ <C, B> Ok
  - **<B, E> retira 1 de B insere os arcos <A, B>, <C, B>, <D, B>**
  - <E, B>
  - <D, C>
  - <C, D>
  - <D, E>
  - <E, D>
  - <C, E>
  - <E, C>
  - <B, A>
  - <D, A>
  - **<A, B>**
  - **<C, B>**
  - **<D, B>**



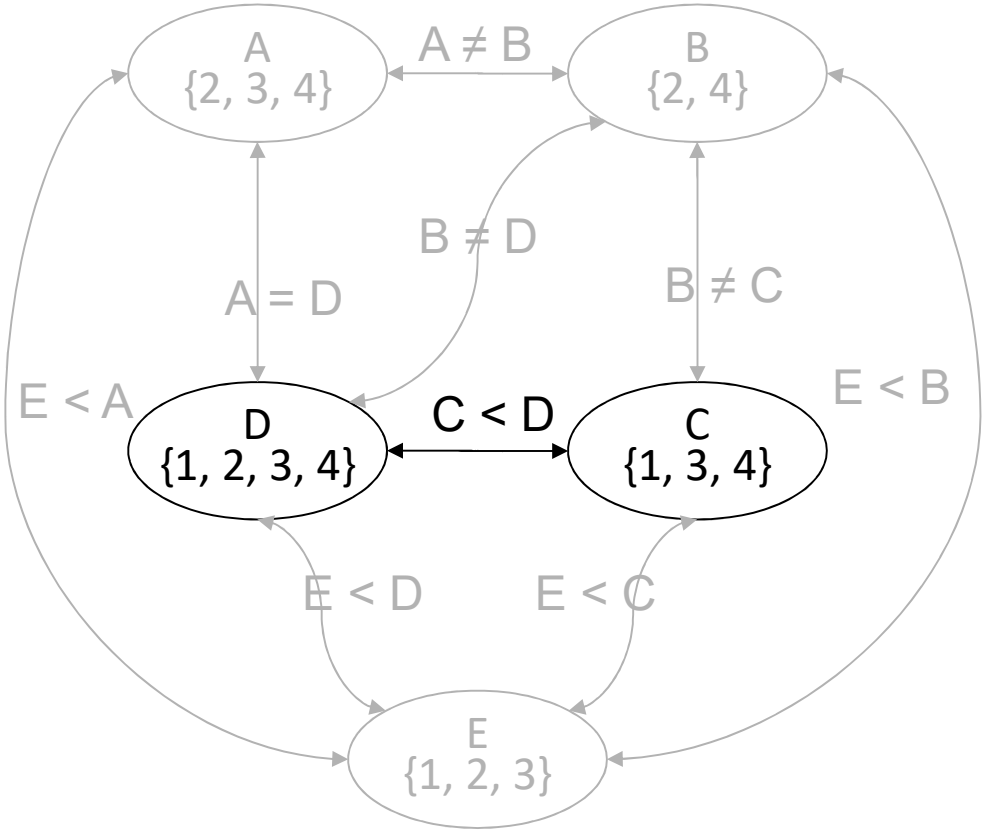
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <B, D> Ok
  - ✓ <D, B> Ok
  - ✓ <B, C> Ok
  - ✓ <C, B> Ok
  - ✓ <B, E> retira 1 de B insere os arcos <A, B>, <C, B>, <D, B>
  - **<E, B>**
  - <D, C>
  - <C, D>
  - <D, E>
  - <E, D>
  - <C, E>
  - <E, C>
  - <B, A>
  - <D, A>
  - <A, B>
  - <C, B>
  - <D, B>



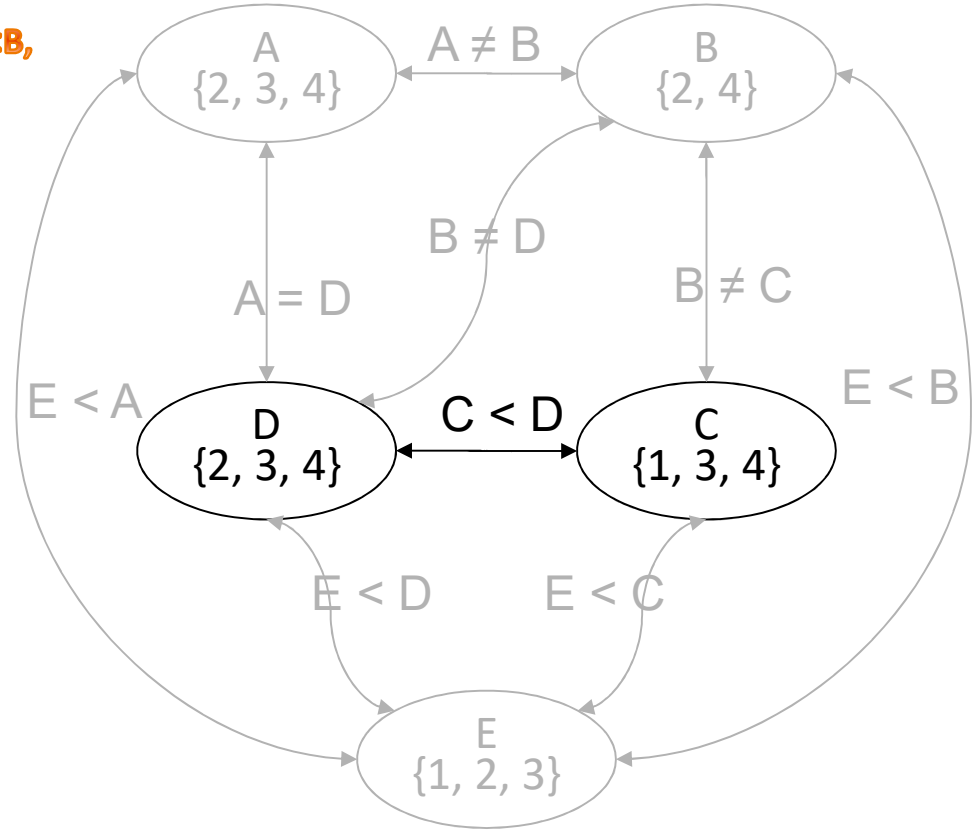
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <B, E> retira 1 de B insere os arcos <A, B>, <C, B>, <D, B>
  - ✓ <E, B> Ok
  - **<D, C>**
  - <C, D>
  - <D, E>
  - <E, D>
  - <C, E>
  - <E, C>
  - <B, A>
  - <D, A>
  - <A, B>
  - <C, B>
  - <D, B>
  - <A, D>
  - <B, D>



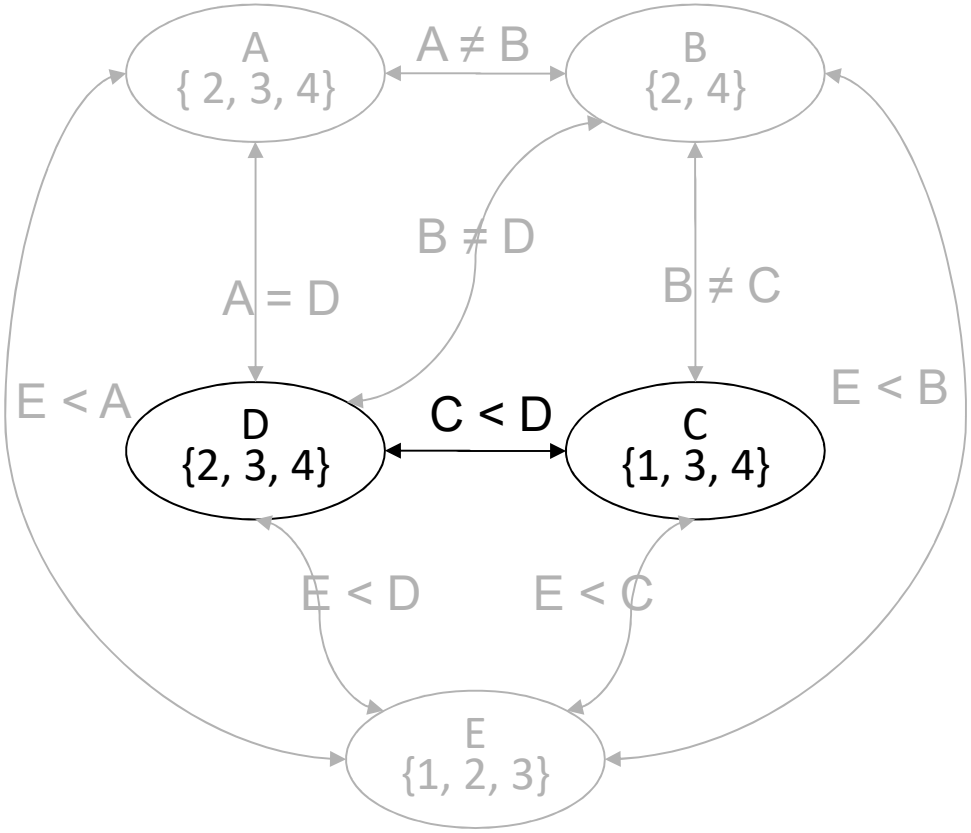
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <B, E> retira 1 de B insere os arcos <A, B>, <C, B>, <D, B>
  - ✓ <E, B> Ok
  - **<D, C> retira 1 de D insere os arcos <A, D>, <B, D>, <E, D>**
  - <C, D>
  - <D, E>
  - **<E, D>***
  - <C, E>
  - <E, C>
  - <B, A>
  - <D, A>
  - <A, B>
  - <C, B>
  - <D, B>
  - **<A, D>**
  - **<B, D>**



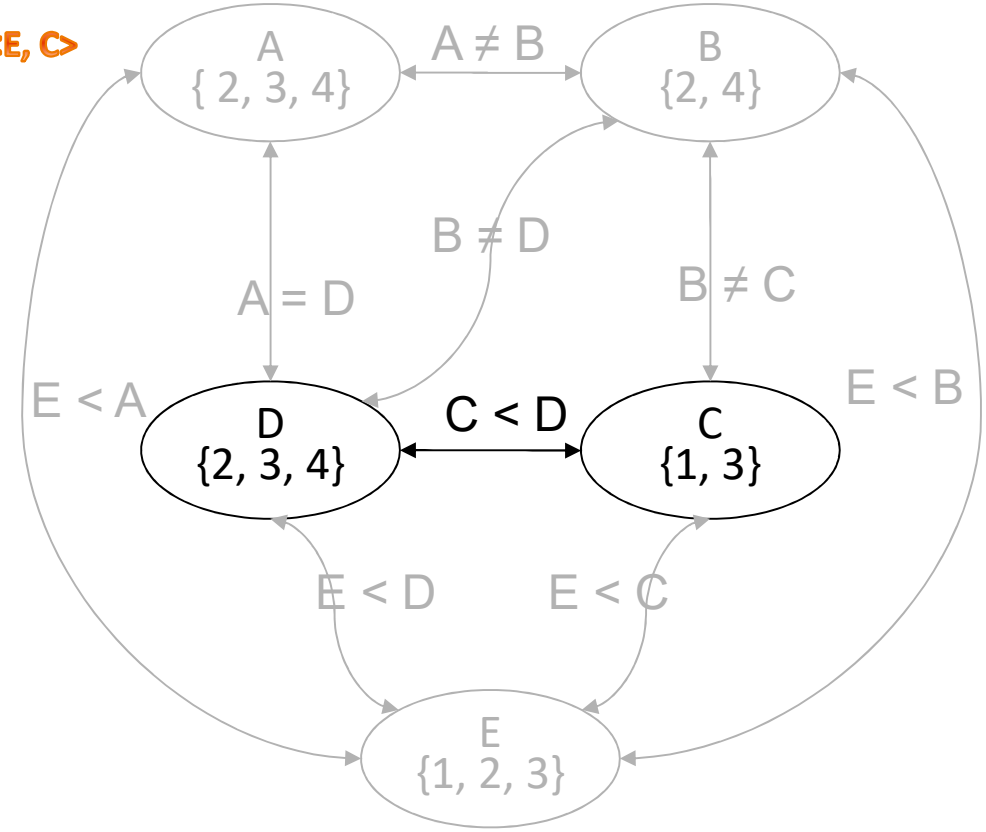
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <E, B> Ok
  - ✓ <D, C> retira 1 de D insere os arcos <A, D>, <B, D>, <E, D>
  - **<C, D>**
  - <D, E>
  - <E, D>
  - <C, E>
  - <E, C>
  - <B, A>
  - <D, A>
  - <A, B>
  - <C, B>
  - <D, B>
  - <A, D>
  - <B, D>
  - <B, C>



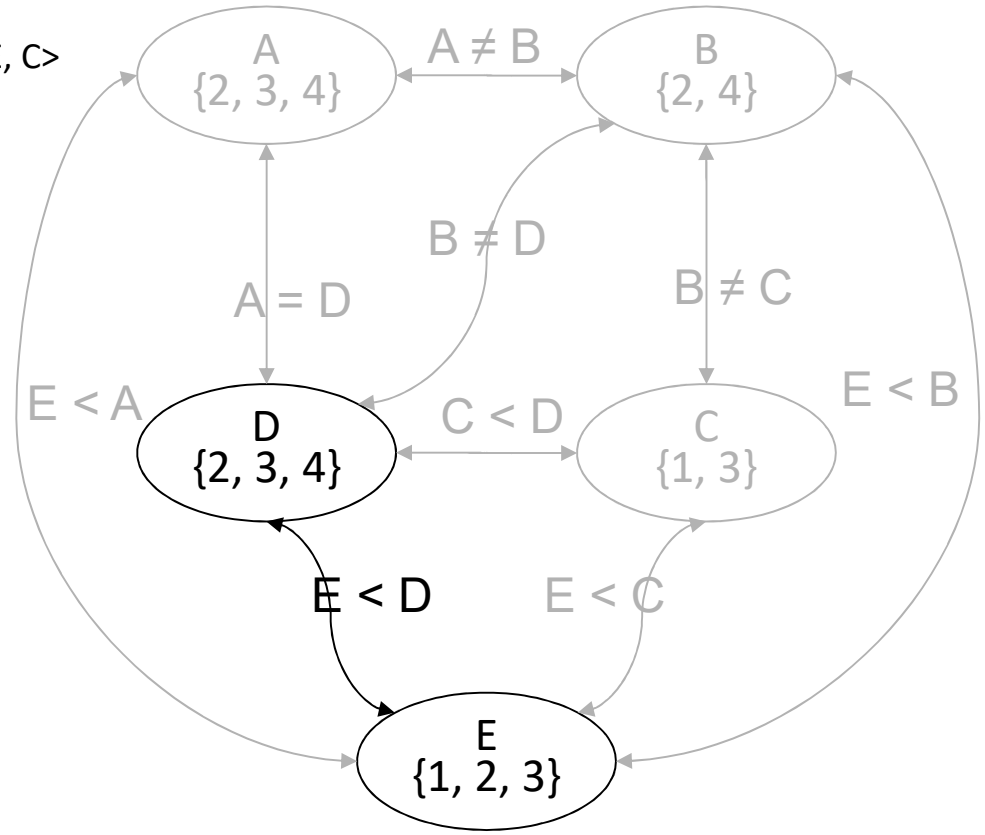
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <E, B> Ok
  - ✓ <D, C> retira 1 de D insere os arcos <A, D>, <B, D>, <E, D>
  - **<C, D> retira 4 de C insere os arcos <B, C>, <E, C>**
  - <D, E>
  - <E, D>
  - <C, E>
  - **<E, C>***
  - <B, A>
  - <D, A>
  - <A, B>
  - <C, B>
  - <D, B>
  - <A, D>
  - <B, D>
  - **<B, C>**



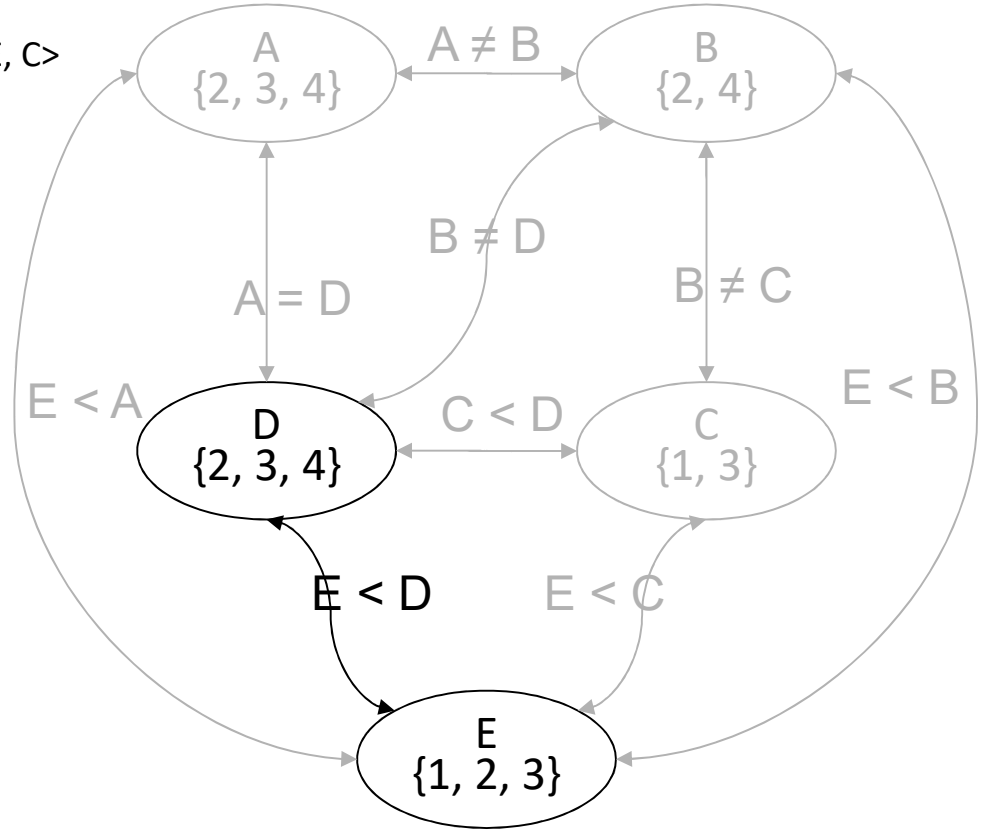
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <E, B> Ok
  - ✓ <D, C> retira 1 de D insere os arcos <A, D>, <B, D>, <E, D>
  - ✓ <C, D> retira 4 de C insere os arcos <B, C>, <E, C>
  - **<D, E>**
  - <E, D>
  - <C, E>
  - <E, C>
  - <B, A>
  - <D, A>
  - <A, B>
  - <C, B>
  - <D, B>
  - <A, D>
  - <B, D>
  - <B, C>



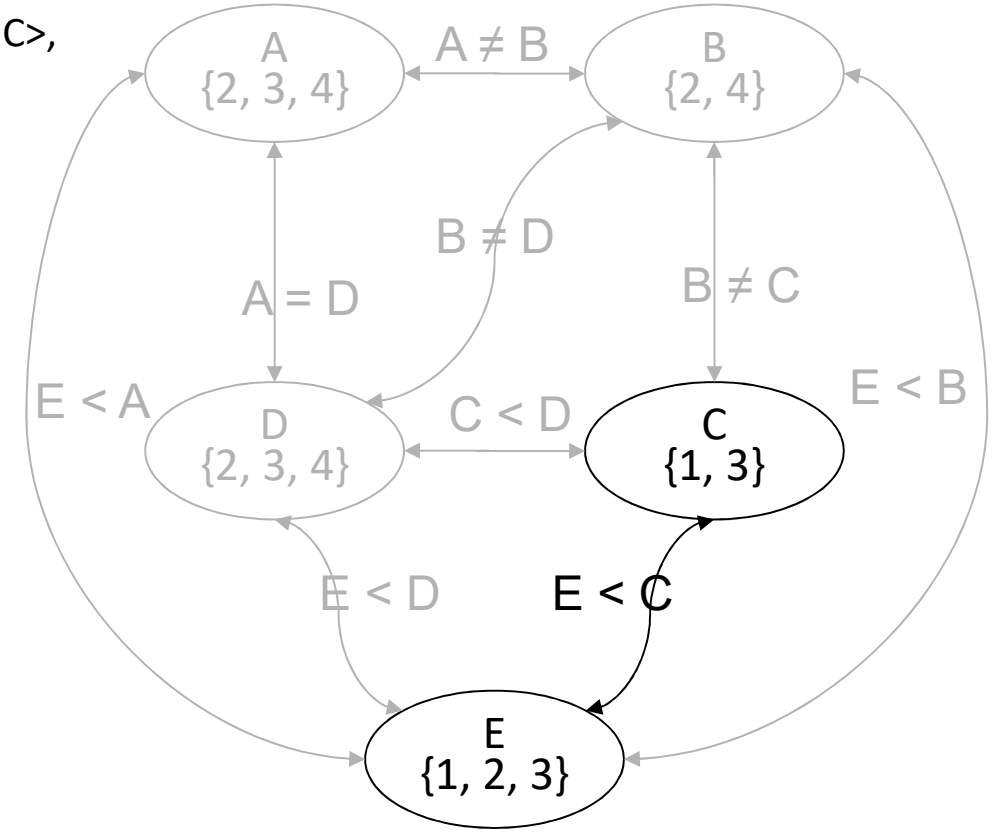
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <E, B> Ok
  - ✓ <D, C> retira 1 de D insere os arcos <A, D>, <B, D>, <E, D>
  - ✓ <C, D> retira 4 de C insere os arcos <B, C>, <E, C>
  - ✓ <D, E> Ok
  - **<E, D>**
  - <C, E>
  - <E, C>
  - <B, A>
  - <D, A>
  - <A, B>
  - <C, B>
  - <D, B>
  - <A, D>
  - <B, D>
  - <B, C>



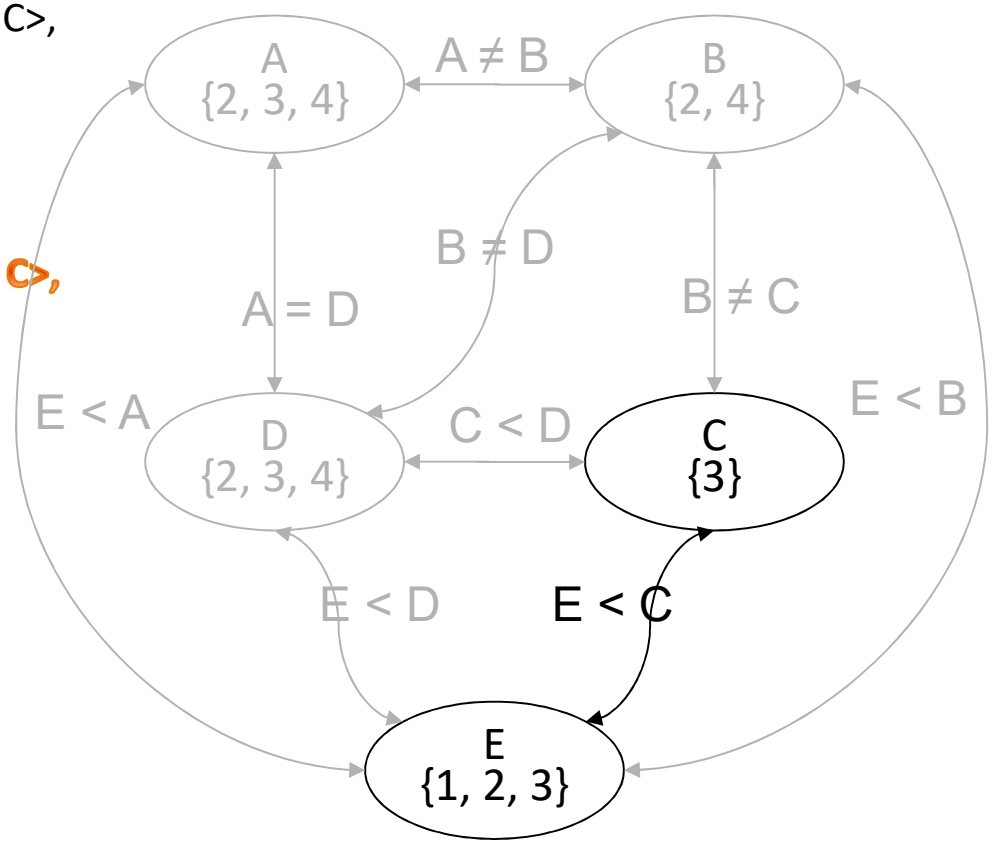
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <E, B> Ok
  - ✓ <D, C> retira 1 de D insere os arcos <A, D>, <B, D>, <E, D>
  - ✓ <C, D> retira 4 de C insere os arcos <B, C>, <E, C>
  - ✓ <D, E> Ok
  - ✓ <E, D> Ok
  - **<C, E>**
  - <E, C>
  - <B, A>
  - <D, A>
  - <A, B>
  - <C, B>
  - <D, B>
  - <A, D>
  - <B, D>
  - <B, C>
  - <D, C>



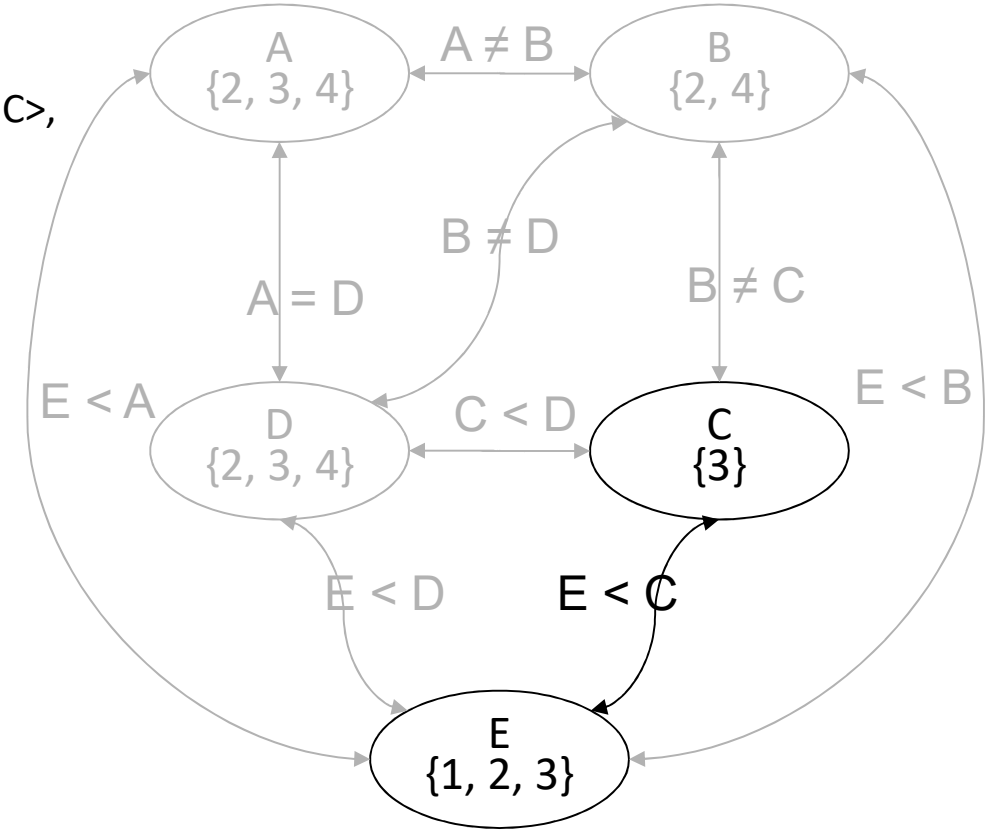
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <E, B> Ok
  - ✓ <D, C> retira 1 de D insere os arcos <A, D>, <B, D>, <E, D>
  - ✓ <C, D> retira 4 de C insere os arcos <B, C>, <E, C>
  - ✓ <D, E> Ok
  - ✓ <E, D> Ok
  - **<C, E> retira 1 de C insere os arcos <B, C>, <D, C>**
  - <E, C>
  - <B, A>
  - <D, A>
  - <A, B>
  - <C, B>
  - <D, B>
  - <A, D>
  - <B, D>
  - **<B, C>***
  - **<D, C>**



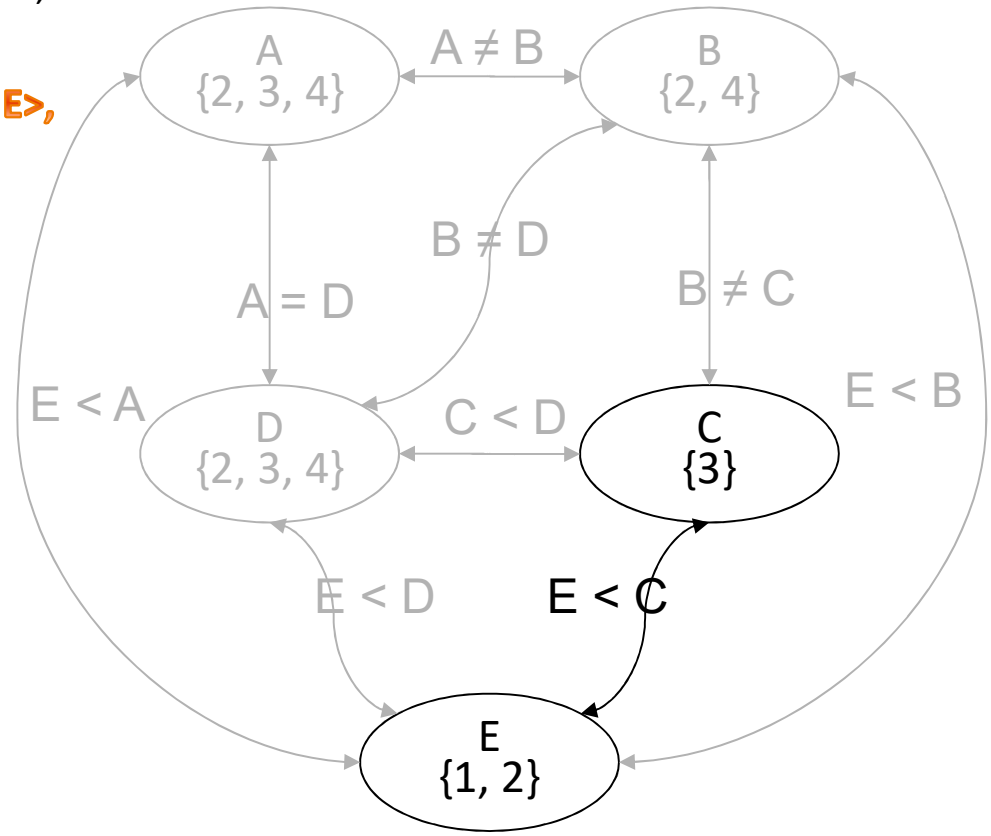
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, E> Ok
  - ✓ <E, D> Ok
  - ✓ <C, E> retira 1 de C insere os arcos <B, C>, <D, C>
  - **<E, C>**
  - <B, A>
  - <D, A>
  - <A, B>
  - <C, B>
  - <D, B>
  - <A, D>
  - <B, D>
  - <B, C>
  - <D, C>



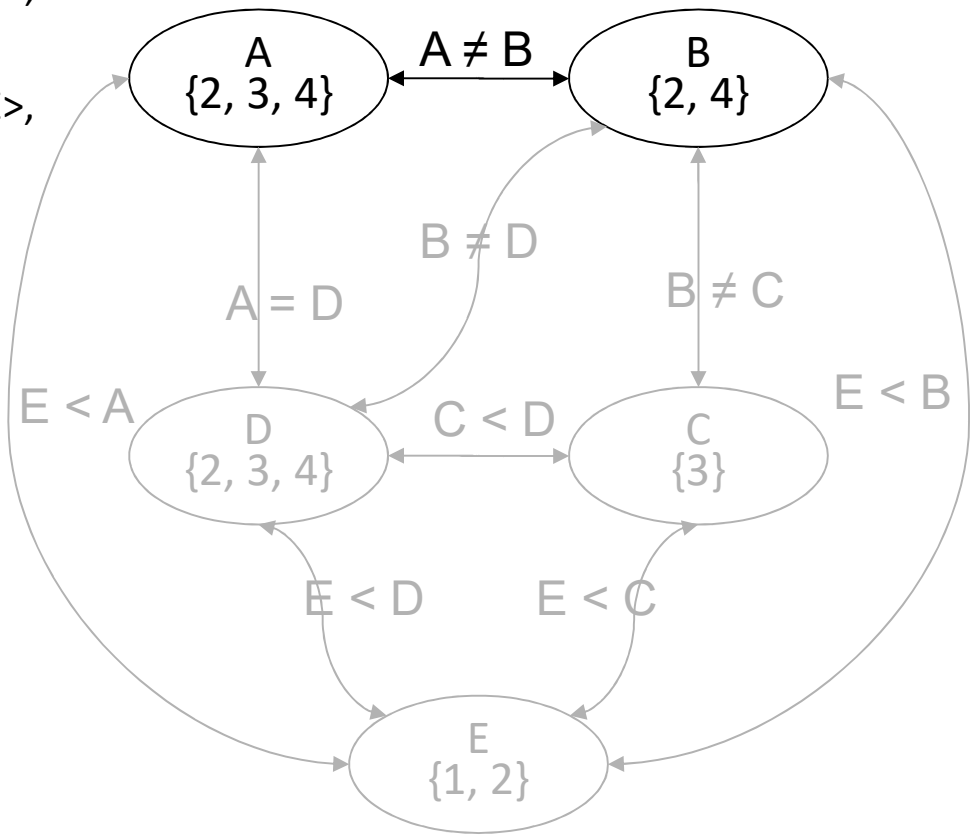
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, E> Ok
  - ✓ <E, D> Ok
  - ✓ <C, E> retira 1 de C insere os arcos <B, C>, <D, C>
  - **<E, C> retira 3 de E insere os arcos <A, E>, <B, E>, <D, E>**
  - <B, A>
  - <D, A>
  - <A, B>
  - <C, B>
  - <D, B>
  - <A, D>
  - <B, D>
  - <B, C>
  - <D, C>
  - **<A, E>**
  - **<B, E>**
  - **<D, E>**



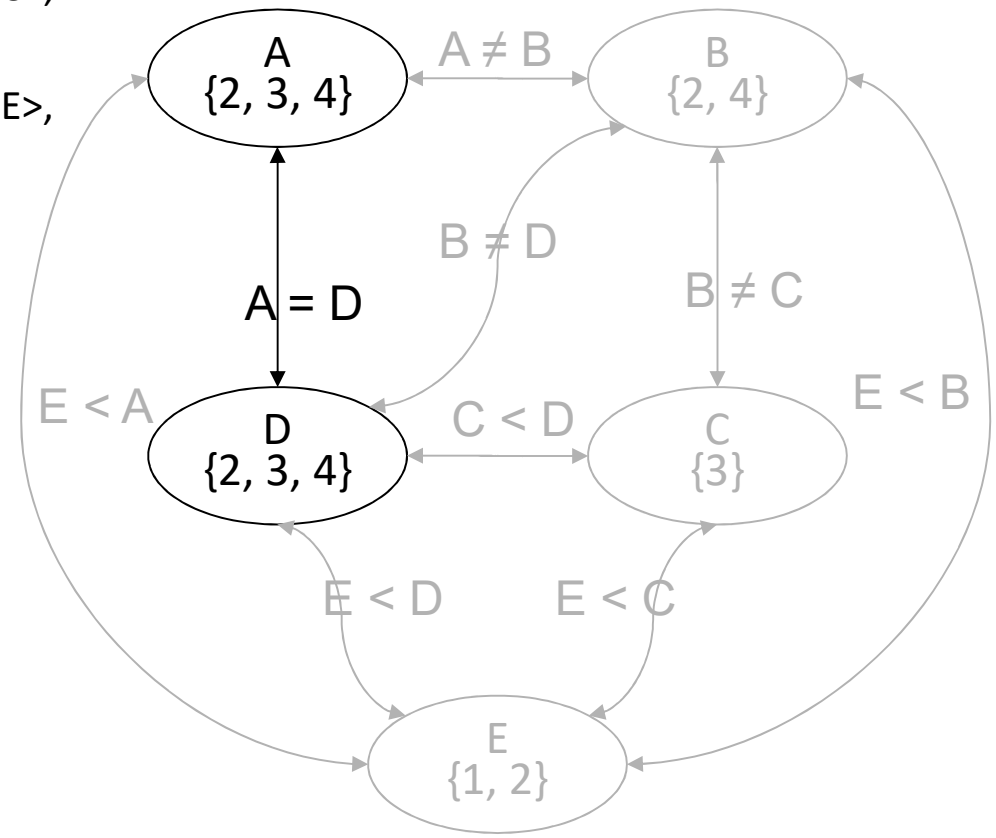
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, E> Ok
  - ✓ <E, D> Ok
  - ✓ <C, E> retira 1 de C insere os arcos <B, C>, <D, C>
  - ✓ <E, C> retira 3 de E insere os arcos <A, E>, <B, E>, <D, E>
  - **<B, A>**
  - <D, A>
  - <A, B>
  - <C, B>
  - <D, B>
  - <A, D>
  - <B, D>
  - <B, C>
  - <D, C>
  - <A, E>
  - <B, E>
  - <D, E>



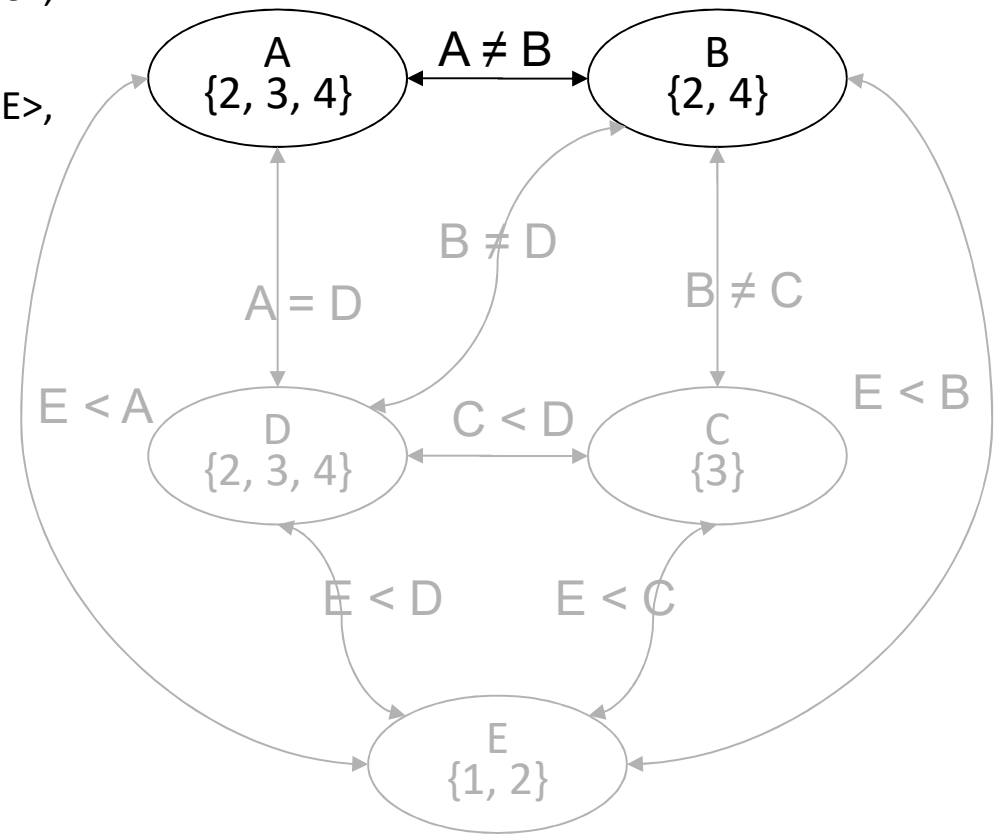
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, E> Ok
  - ✓ <E, D> Ok
  - ✓ <C, E> retira 1 de C insere os arcos <B, C>, <D, C>
  - ✓ <E, C> retira 3 de E insere os arcos <A, E>, <B, E>, <D, E>
  - ✓ <B, A> Ok
  - **<D, A>**
  - <A, B>
  - <C, B>
  - <D, B>
  - <A, D>
  - <B, D>
  - <B, C>
  - <D, C>
  - <A, E>
  - <B, E>
  - <D, E>



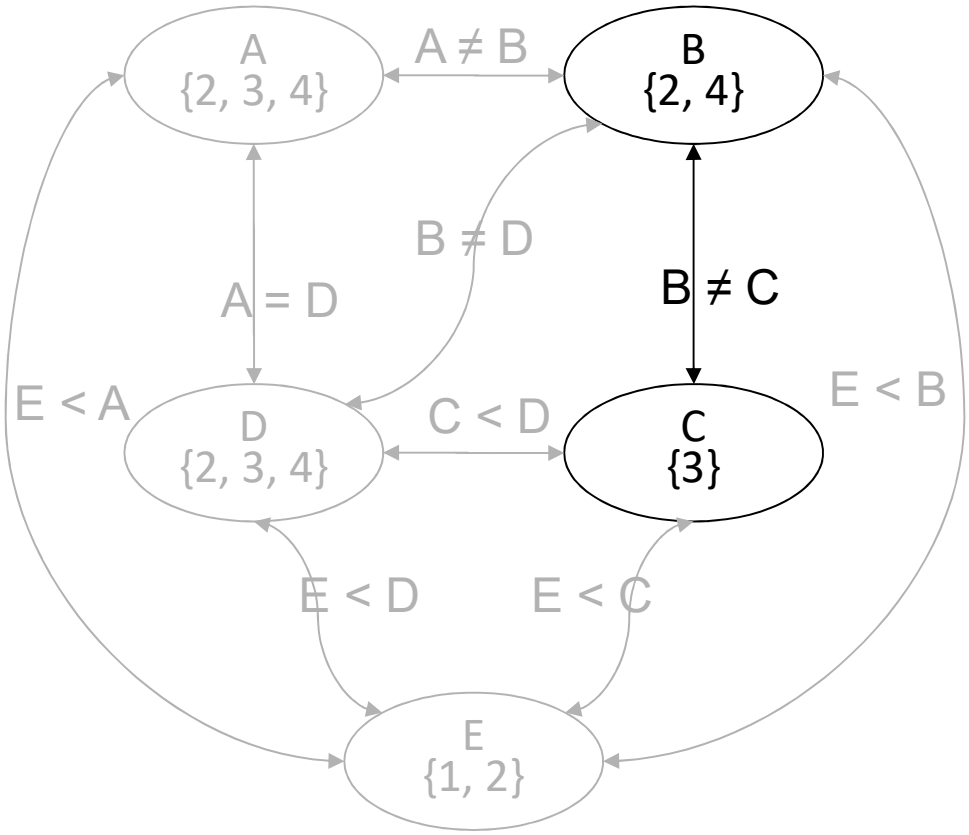
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, E> Ok
  - ✓ <E, D> Ok
  - ✓ <C, E> retira 1 de C insere os arcos <B, C>, <D, C>
  - ✓ <E, C> retira 3 de E insere os arcos <A, E>, <B, E>, <D, E>
  - ✓ <B, A> Ok
  - ✓ <D, A> Ok
  - **<A, B>**
  - <C, B>
  - <D, B>
  - <A, D>
  - <B, D>
  - <B, C>
  - <D, C>
  - <A, E>
  - <B, E>
  - <D, E>



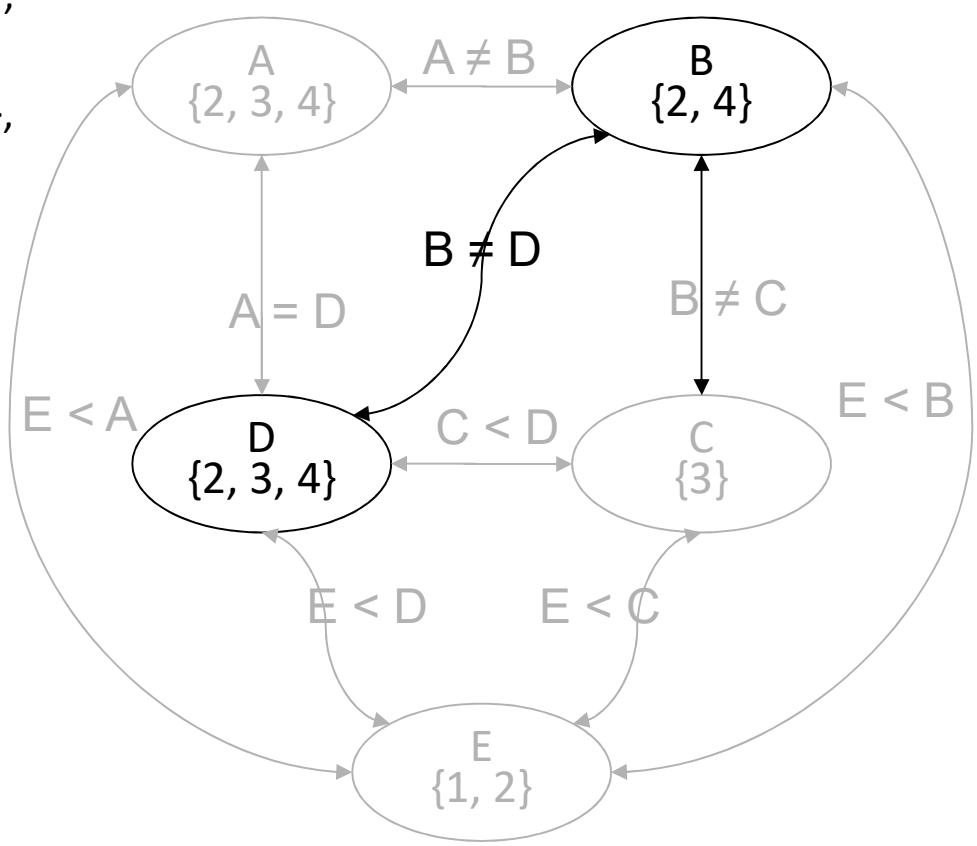
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, E> Ok
  - ✓ <E, D> Ok
  - ✓ <C, E> retira 1 de C insere os arcos <B, C>, <D, C>
  - ✓ <E, C> retira 3 de E insere os arcos <A, E>, <B, E>, <D, E>
  - ✓ <B, A> Ok
  - ✓ <D, A> Ok
  - ✓ <A, B> Ok
  - **<C, B>**
  - <D, B>
  - <A, D>
  - <B, D>
  - <B, C>
  - <D, C>
  - <A, E>
  - <B, E>
  - <D, E>



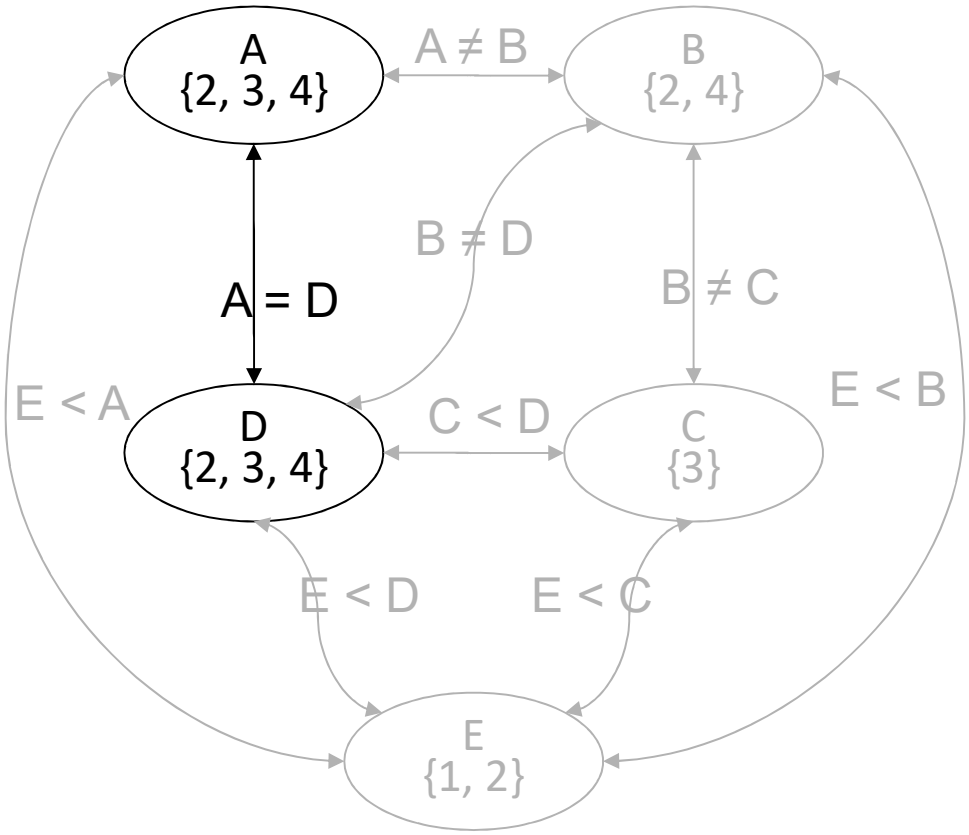
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, E> Ok
  - ✓ <E, D> Ok
  - ✓ <C, E> retira 1 de C insere os arcos <B, C>, <D, C>
  - ✓ <E, C> retira 3 de E insere os arcos <A, E>, <B, E>, <D, E>
  - ✓ <B, A> Ok
  - ✓ <D, A> Ok
  - ✓ <A, B> Ok
  - ✓ <C, B> Ok
  - **<D, B>**
  - <A, D>
  - <B, D>
  - <B, C>
  - <D, C>
  - <A, E>
  - <B, E>
  - <D, E>



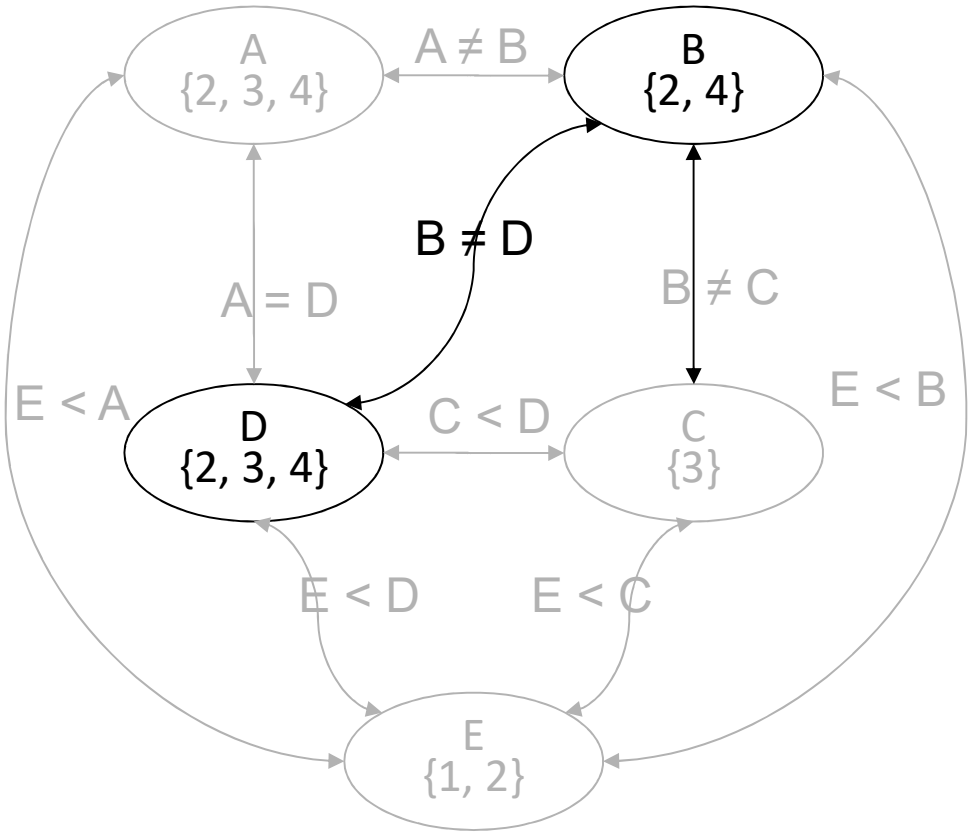
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, E> Ok
  - ✓ <E, D> Ok
  - ✓ <C, E> retira 1 de C insere os arcos <B, C>, <D, C>
  - ✓ <E, C> retira 3 de E insere os arcos <A, E>, <B, E>, <D, E>
  - ✓ <B, A> Ok
  - ✓ <D, A> Ok
  - ✓ <A, B> Ok
  - ✓ <C, B> Ok
  - ✓ <D, B> Ok
  - **<A, D>**
  - <B, D>
  - <B, C>
  - <D, C>
  - <A, E>
  - <B, E>
  - <D, E>



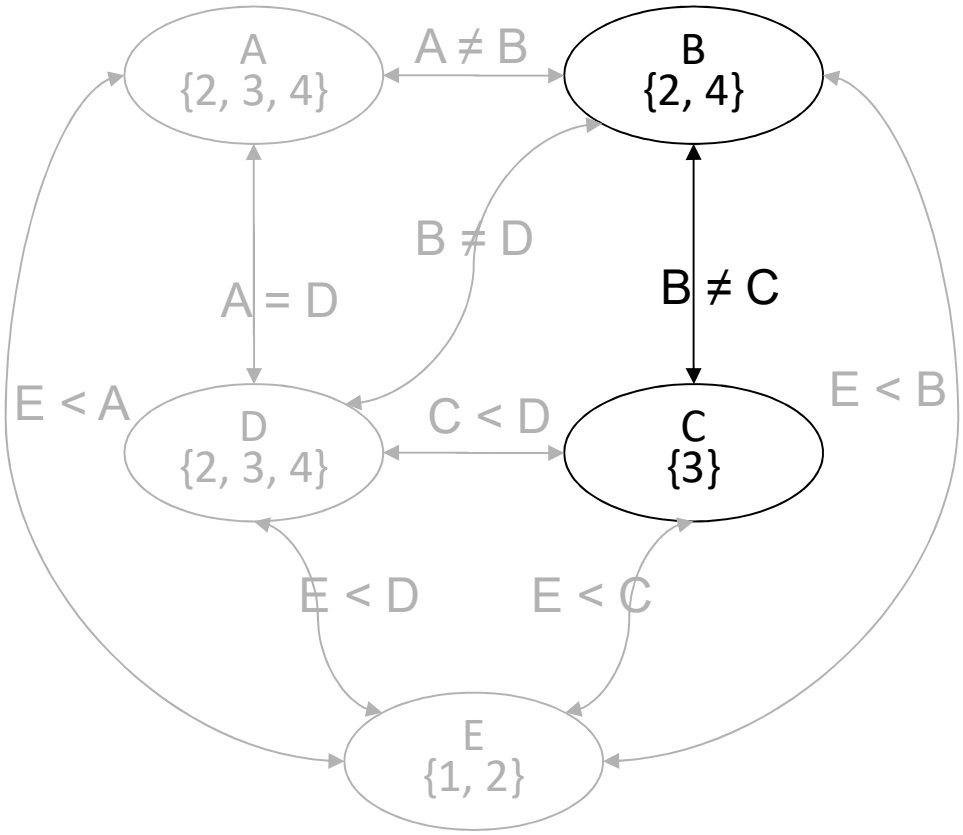
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, E> Ok
  - ✓ <E, D> Ok
  - ✓ <C, E> retira 1 de C insere os arcos <B, C>, <D, C>
  - ✓ <E, C> retira 3 de E insere os arcos <A, E>, <B, E>, <D, E>
  - ✓ <B, A> Ok
  - ✓ <D, A> Ok
  - ✓ <A, B> Ok
  - ✓ <C, B> Ok
  - ✓ <D, B> Ok
  - ✓ <A, D> Ok
  - **<B, D>**
  - <B, C>
  - <D, C>
  - <A, E>
  - <B, E>
  - <D, E>



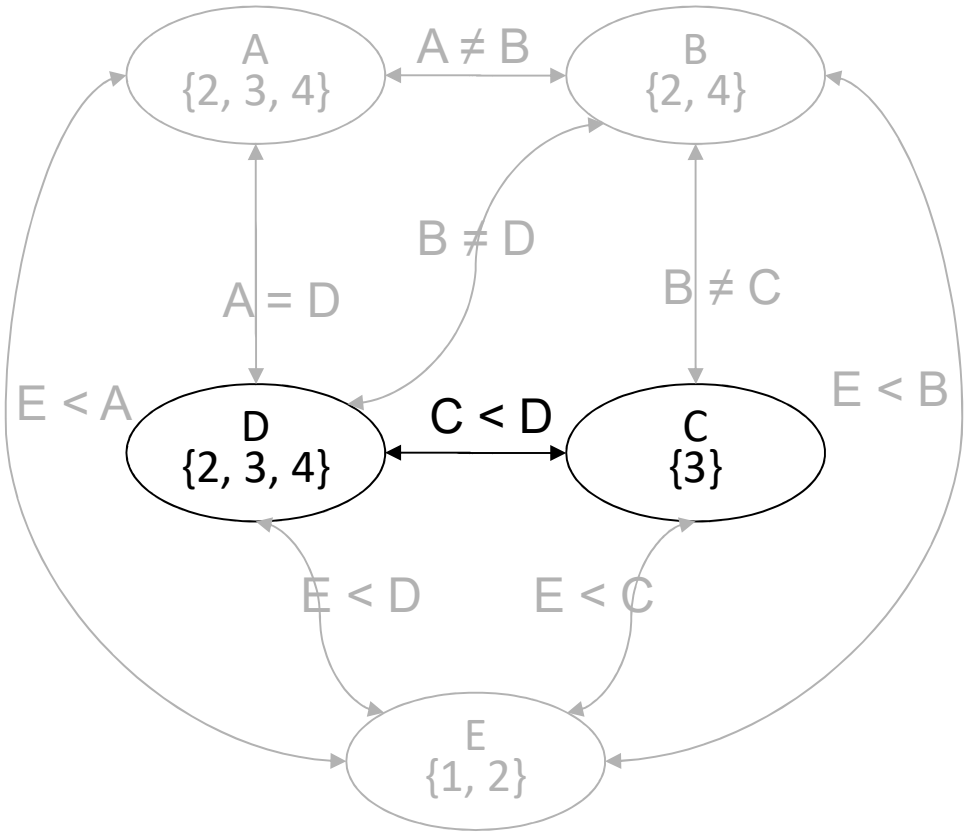
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, E> Ok
  - ✓ <E, D> Ok
  - ✓ <C, E> retira 1 de C insere os arcos <B, C>, <D, C>
  - ✓ <E, C> retira 3 de E insere os arcos <A, E>, <B, E>, <D, E>
  - ✓ <B, A> Ok
  - ✓ <D, A> Ok
  - ✓ <A, B> Ok
  - ✓ <C, B> Ok
  - ✓ <D, B> Ok
  - ✓ <A, D> Ok
  - ✓ <B, D> Ok
  - **<B, C>**
  - <D, C>
  - <A, E>
  - <B, E>
  - <D, E>



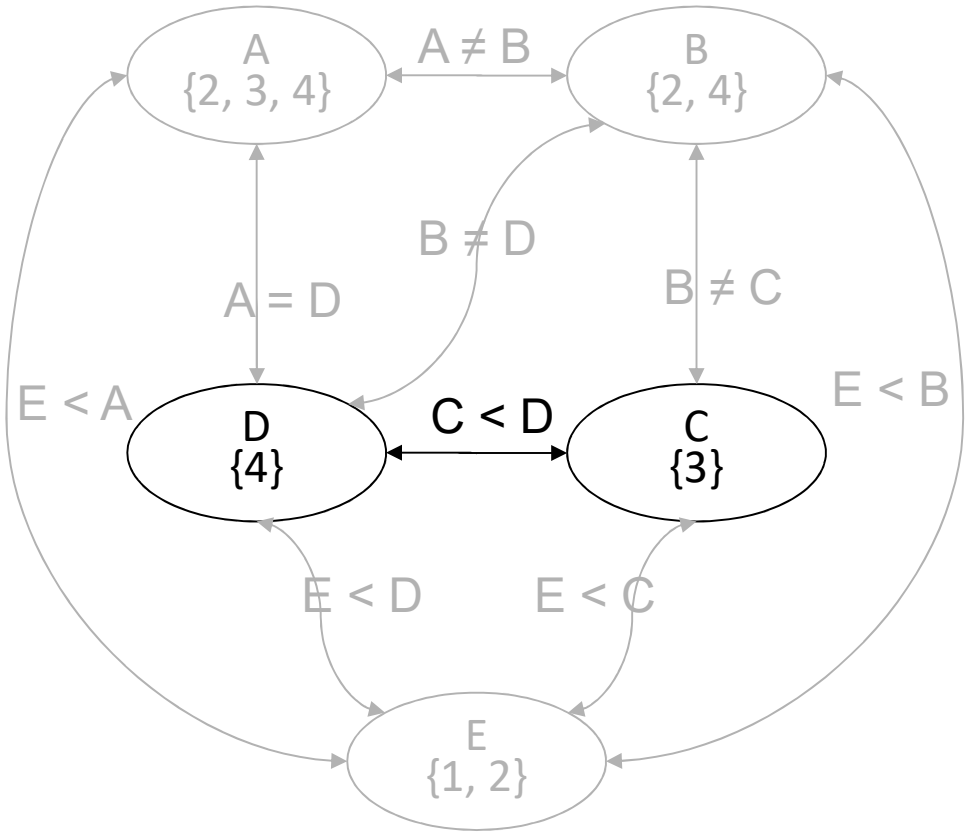
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <B, A> Ok
  - ✓ <D, A> Ok
  - ✓ <A, B> Ok
  - ✓ <C, B> Ok
  - ✓ <D, B> Ok
  - ✓ <A, D> Ok
  - ✓ <B, D> Ok
  - ✓ <B, C> Ok
  - **<D, C>**
  - <A, E>
  - <B, E>
  - <D, E>



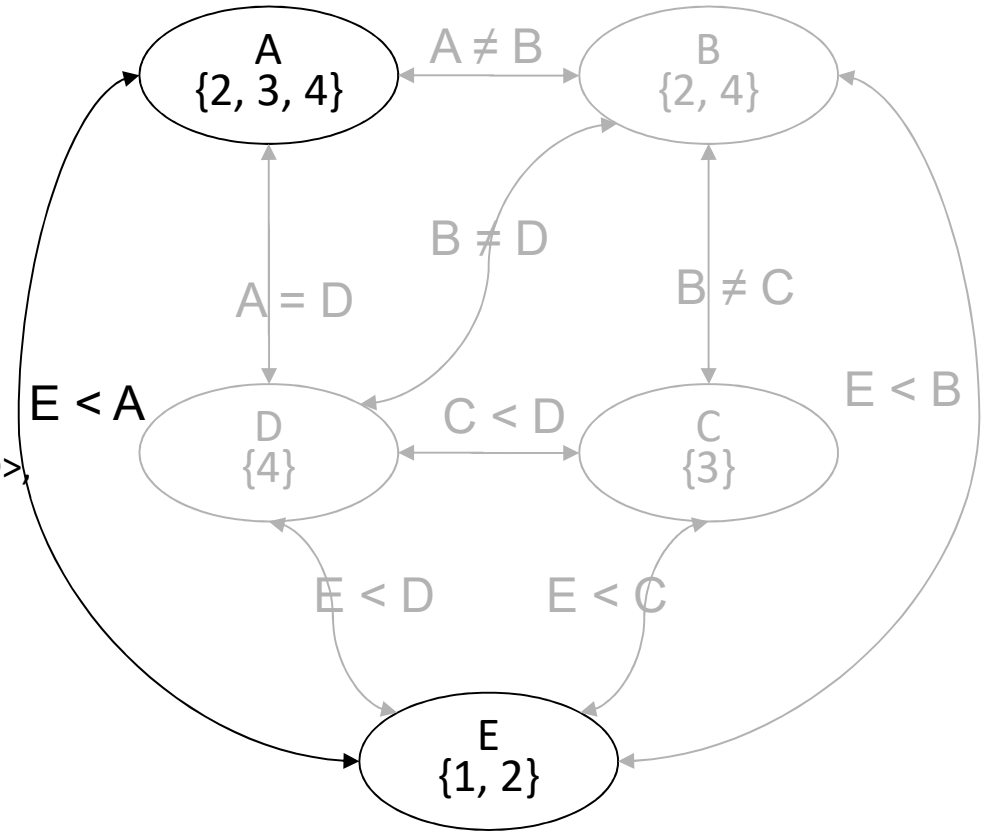
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <B, A> Ok
  - ✓ <D, A> Ok
  - ✓ <A, B> Ok
  - ✓ <C, B> Ok
  - ✓ <D, B> Ok
  - ✓ <A, D> Ok
  - ✓ <B, D> Ok
  - ✓ <B, C> Ok
  - **<D, C> retira 2 e 3 de D insere os arcos <A,D>, <B,D>, <E,D>**
  - <A, E>
  - <B, E>
  - <D, E>
  - **<A, D>**
  - **<B, D>**
  - **<E, D>**



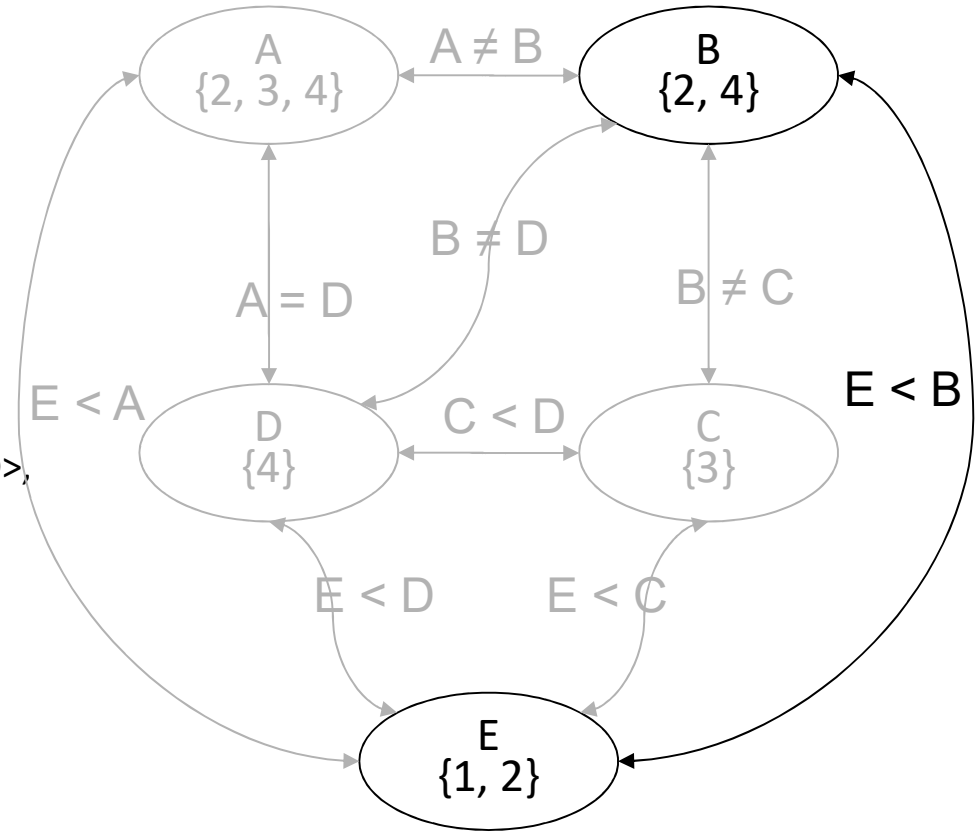
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <B, A> Ok
  - ✓ <D, A> Ok
  - ✓ <A, B> Ok
  - ✓ <C, B> Ok
  - ✓ <D, B> Ok
  - ✓ <A, D> Ok
  - ✓ <B, D> Ok
  - ✓ <B, C> Ok
  - ✓ <D, C> retira 2 e 3 de D insere os arcos <A,D>, <B,D>, <E,D>
  - **<A, E>**
  - <B, E>
  - <D, E>
  - <A, D>
  - <B, D>
  - <E, D>



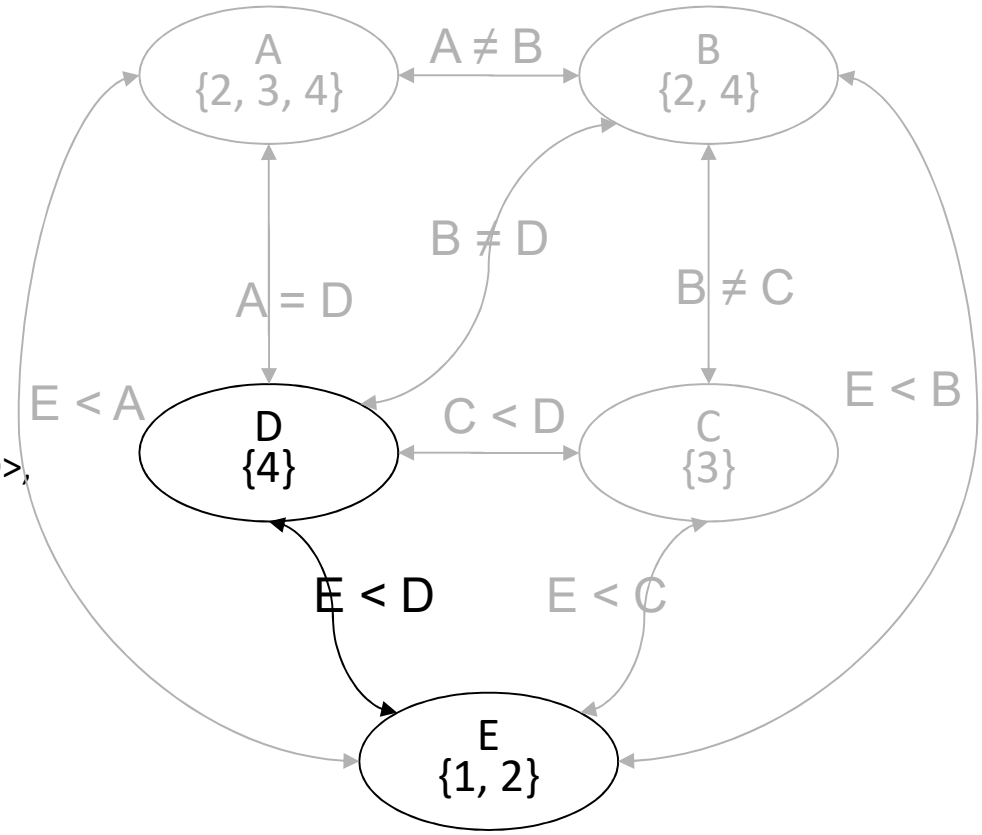
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <B, A> Ok
  - ✓ <D, A> Ok
  - ✓ <A, B> Ok
  - ✓ <C, B> Ok
  - ✓ <D, B> Ok
  - ✓ <A, D> Ok
  - ✓ <B, D> Ok
  - ✓ <B, C> Ok
  - ✓ <D, C> retira 2 e 3 de D insere os arcos <A,D>, <B,D>, <E,D>
  - ✓ <A, E> Ok
  - **<B, E>**
  - <D, E>
  - <A, D>
  - <B, D>
  - <E, D>



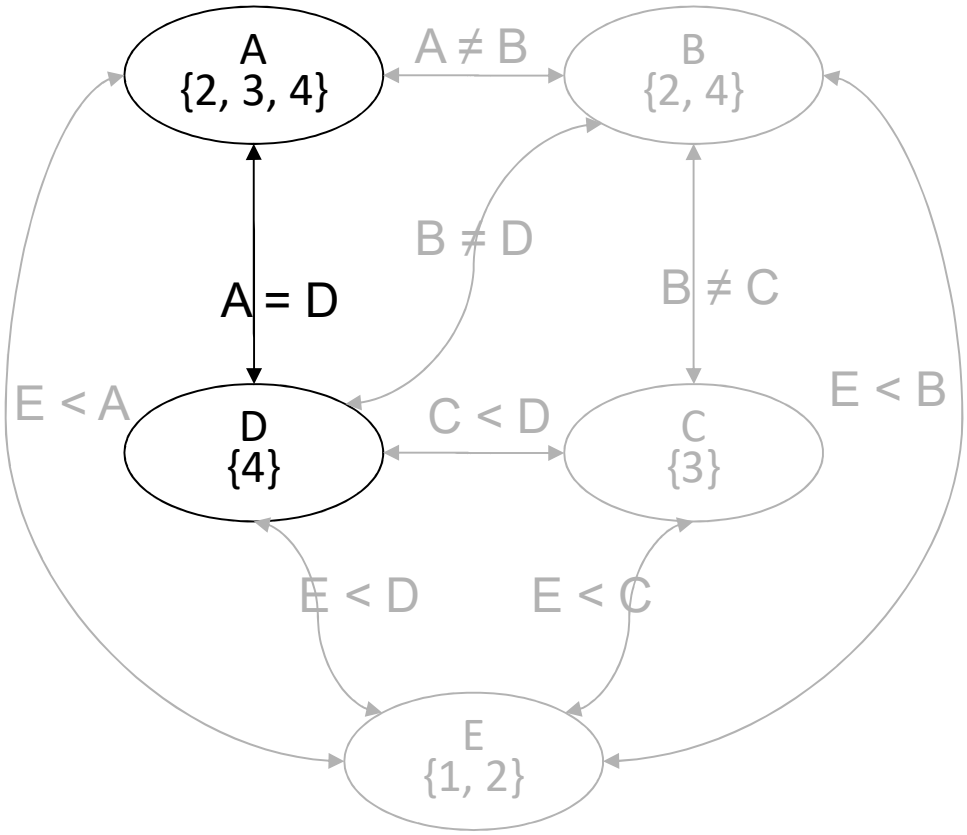
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <B, A> Ok
  - ✓ <D, A> Ok
  - ✓ <A, B> Ok
  - ✓ <C, B> Ok
  - ✓ <D, B> Ok
  - ✓ <A, D> Ok
  - ✓ <B, D> Ok
  - ✓ <B, C> Ok
  - ✓ <D, C> retira 2 e 3 de D insere os arcos <A,D>, <B,D>, <E,D>
  - ✓ <A, E> Ok
  - ✓ <B, E> Ok
  - **<D, E>**
  - <A, D>
  - <B, D>
  - <E, D>



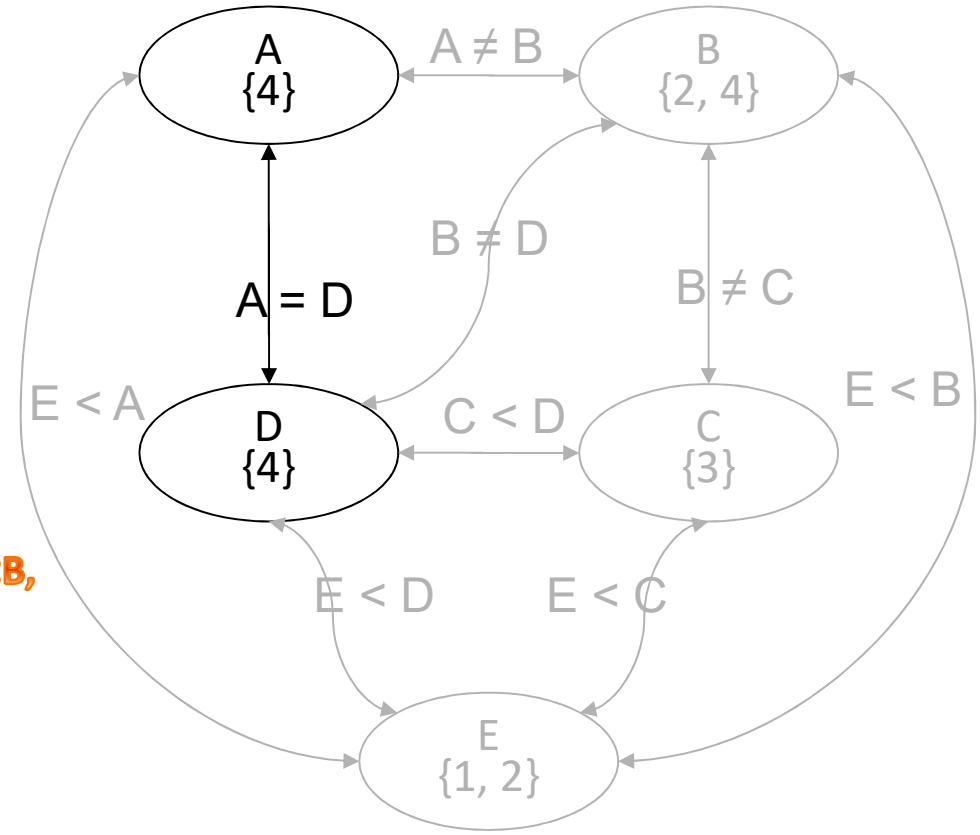
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <A, D> Ok
  - ✓ <B, D> Ok
  - ✓ <B, C> Ok
  - ✓ <D, C> retira 2 e 3 de D insere os arcos <A,D>, <B,D>, <E,D>
  - ✓ <A, E> Ok
  - ✓ <B, E> Ok
  - ✓ <D, E> Ok
  - **<A, D>**
  - <B, D>
  - <E, D>



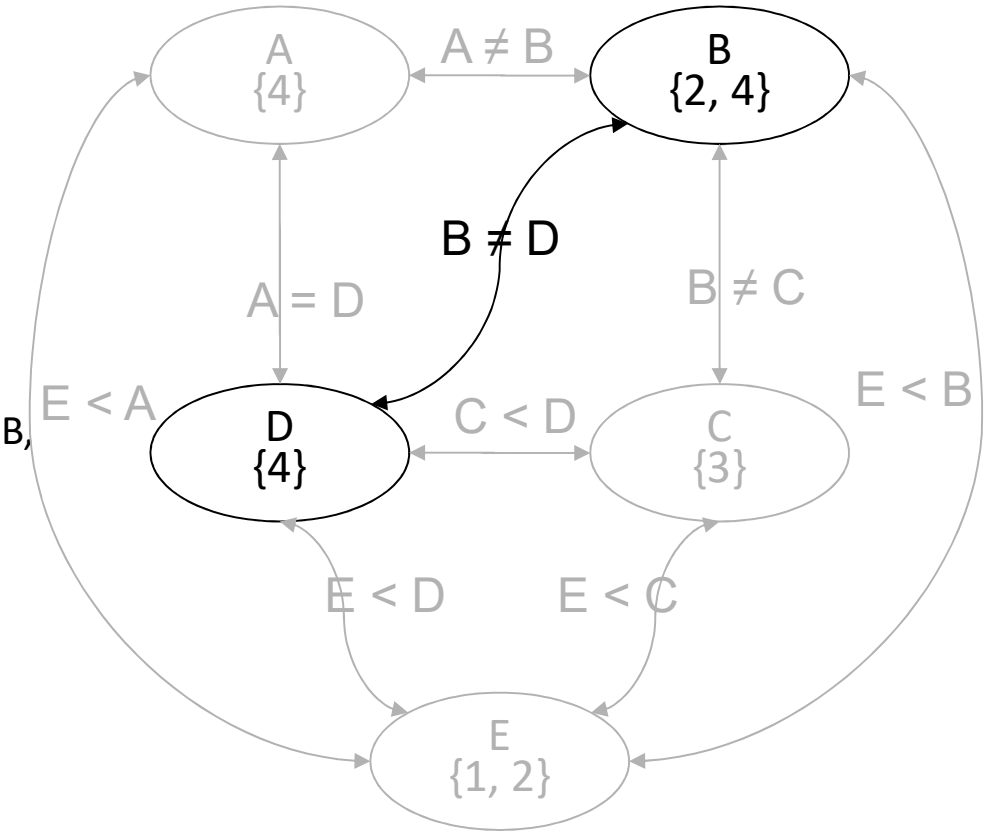
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <A, D> Ok
  - ✓ <B, D> Ok
  - ✓ <B, C> Ok
  - ✓ <D, C> retira 2 e 3 de D insere os arcos <A,D>, <B,D>, <E,D>
  - ✓ <A, E> Ok
  - ✓ <B, E> Ok
  - ✓ <D, E> Ok
  - **<A, D> retira 2 e 3 de A insere os arcos <B, A>, <E, A>**
  - <B, D>
  - <E, D>
  - **<B, A>**
  - **<E, A>**



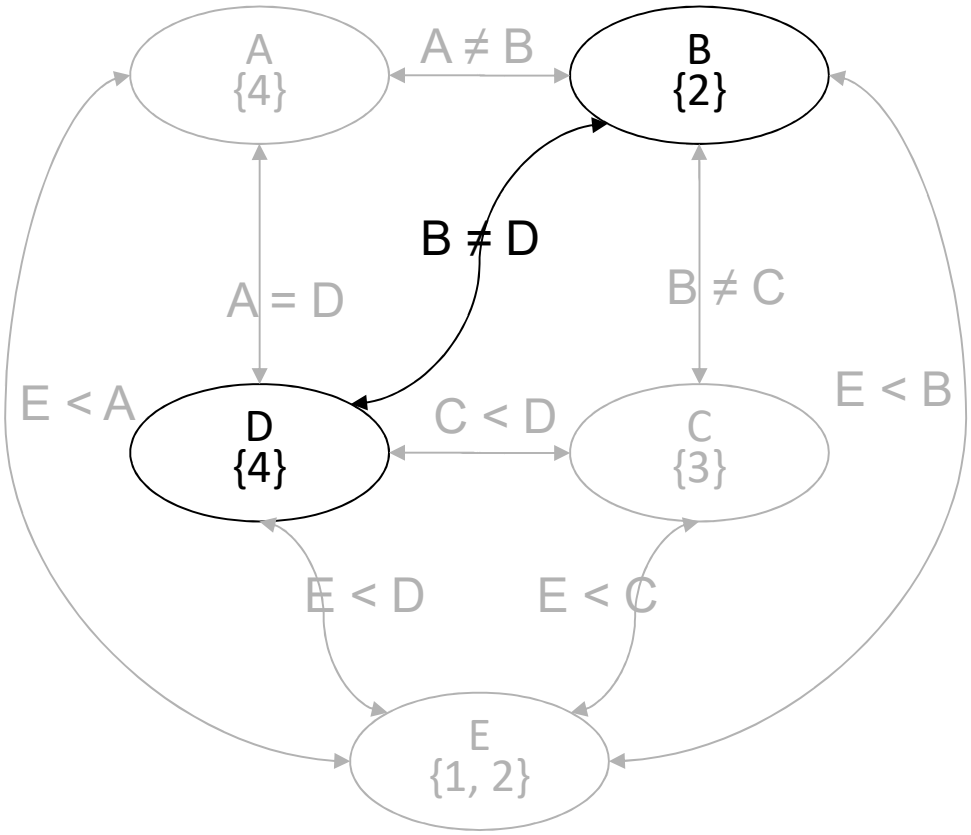
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, C> retira 2 e 3 de D insere os arcos <A,D>, <B,D>, <E,D>
  - ✓ <A, E> Ok
  - ✓ <B, E> Ok
  - ✓ <D, E> Ok
  - ✓ <A, D> retira 2 e 3 de A insere os arcos <B, A>, <E, A>
  - **<B, D>**
  - <E, D>
  - <B, A>
  - <E, A>



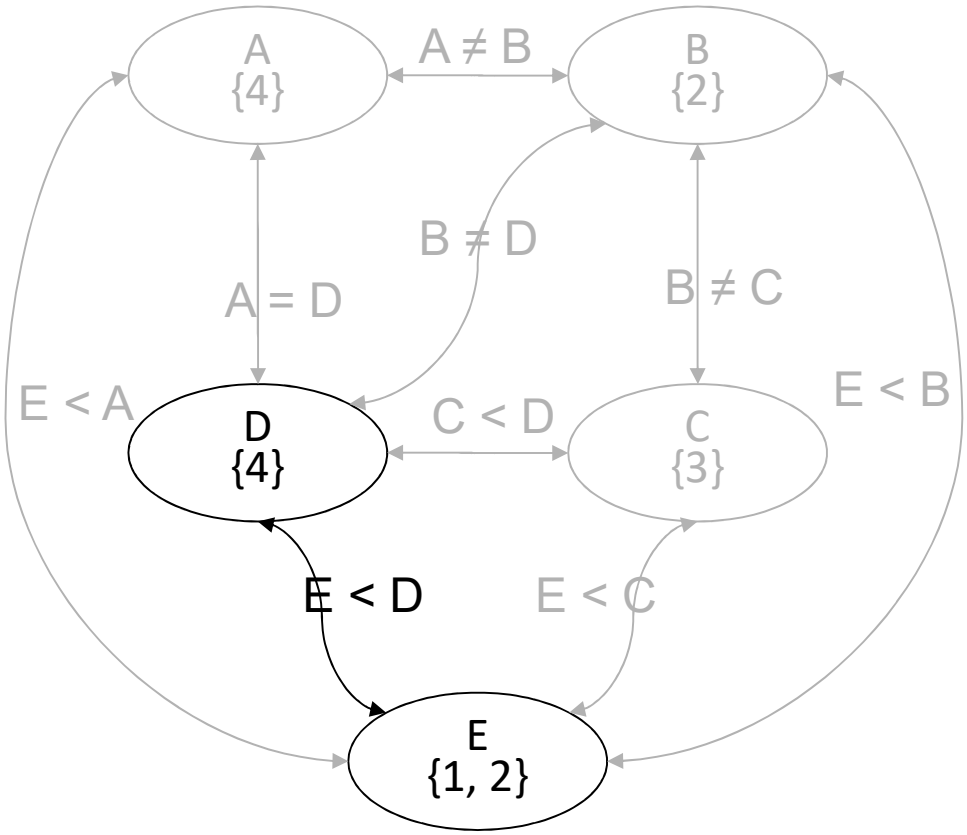
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, C> retira 2 e 3 de D insere os arcos <A,D>, <B,D>, <E,D>
  - ✓ <A, E> Ok
  - ✓ <B, E> Ok
  - ✓ <D, E> Ok
  - ✓ <A, D> retira 2 e 3 de A insere os arcos <B,A>, <E,A>
  - **<B, D> retira de 4 de B insere os arcos <A,B>, <C,B>, <E,B>**
  - <E, D>
  - <B, A>
  - <E, A>
  - **<A, B>**
  - **<C, B>**
  - **<E, B>**



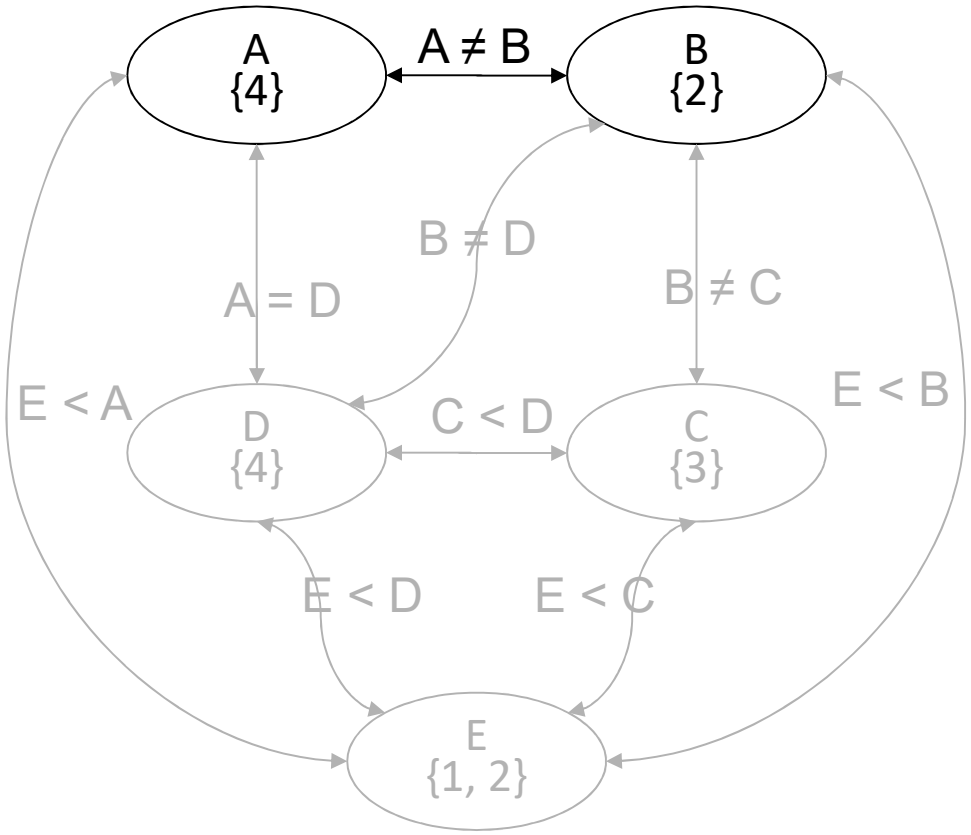
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, C> retira 2 e 3 de D insere os arcos <A,D>, <B,D>, <E,D>
  - ✓ <A, E> Ok
  - ✓ <B, E> Ok
  - ✓ <D, E> Ok
  - ✓ <A, D> retira 2 e 3 de A insere os arcos <B,A>, <E,A>
  - ✓ <B, D> retira de 4 de B insere os arcos <A,B>, <C,B>, <E,B>
  - **<E, D>**
  - <B, A>
  - <E, A>
  - <A, B>
  - <C, B>
  - <E, B>



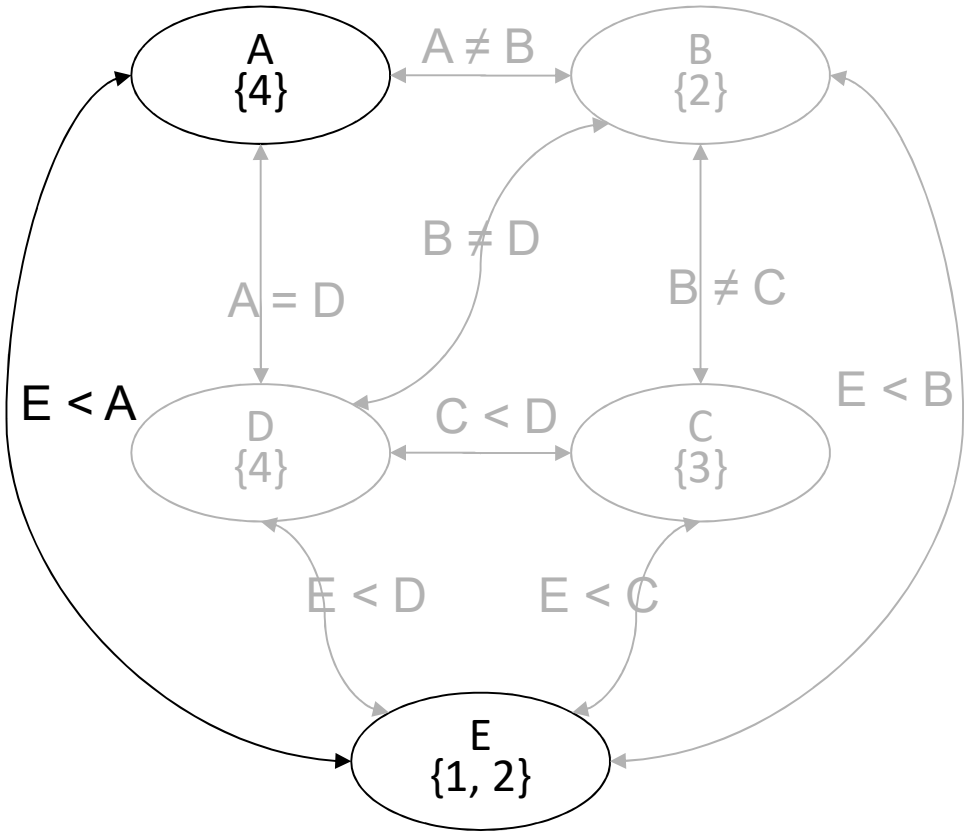
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, C> retira 2 e 3 de D insere os arcos <A,D>, <B,D>, <E,D>
  - ✓ <A, E> Ok
  - ✓ <B, E> Ok
  - ✓ <D, E> Ok
  - ✓ <A, D> retira 2 e 3 de A insere os arcos <B,A>, <E,A>
  - ✓ <B, D> retira de 4 de B insere os arcos <A,B>, <C,B>, <E,B>
  - ✓ <E, D> Ok
  - **<B, A>**
  - <E, A>
  - <A, B>
  - <C, B>
  - <E, B>



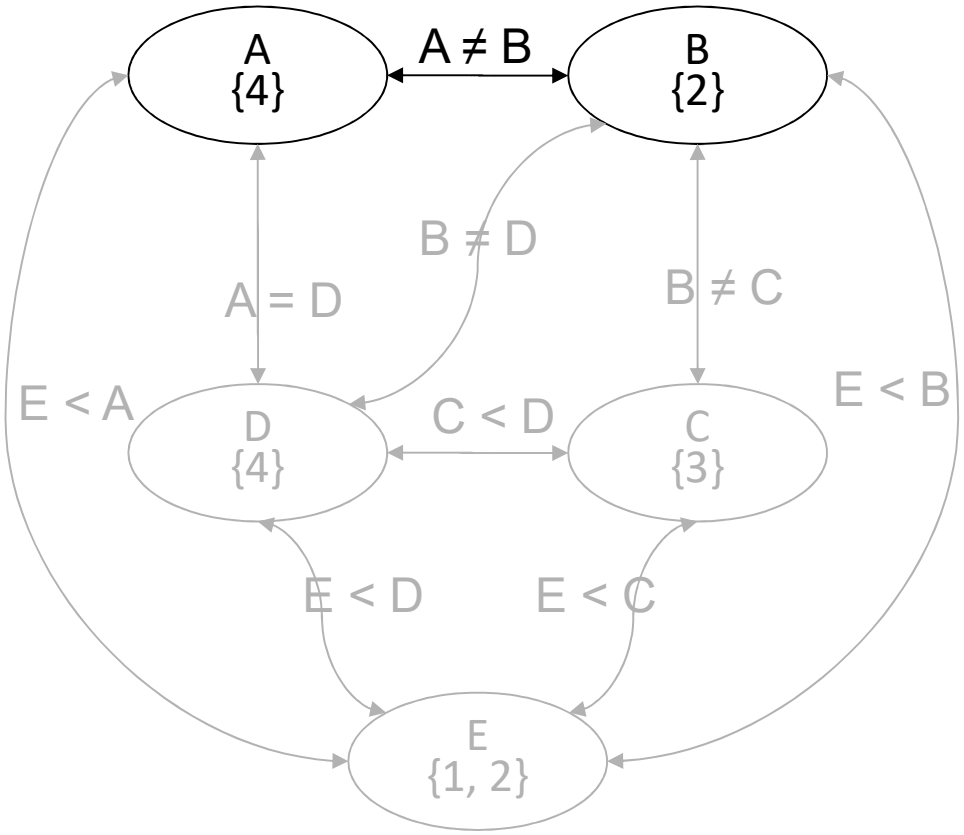
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, C> retira 2 e 3 de D insere os arcos <A,D>, <B,D>, <E,D>
  - ✓ <A, E> Ok
  - ✓ <B, E> Ok
  - ✓ <D, E> Ok
  - ✓ <A, D> retira 2 e 3 de A insere os arcos <B,A>, <E,A>
  - ✓ <B, D> retira de 4 de B insere os arcos <A,B>, <C,B>, <E,B>
  - ✓ <E, D> Ok
  - ✓ <B, A> Ok
  - **<E, A>**
  - <A, B>
  - <C, B>
  - <E, B>



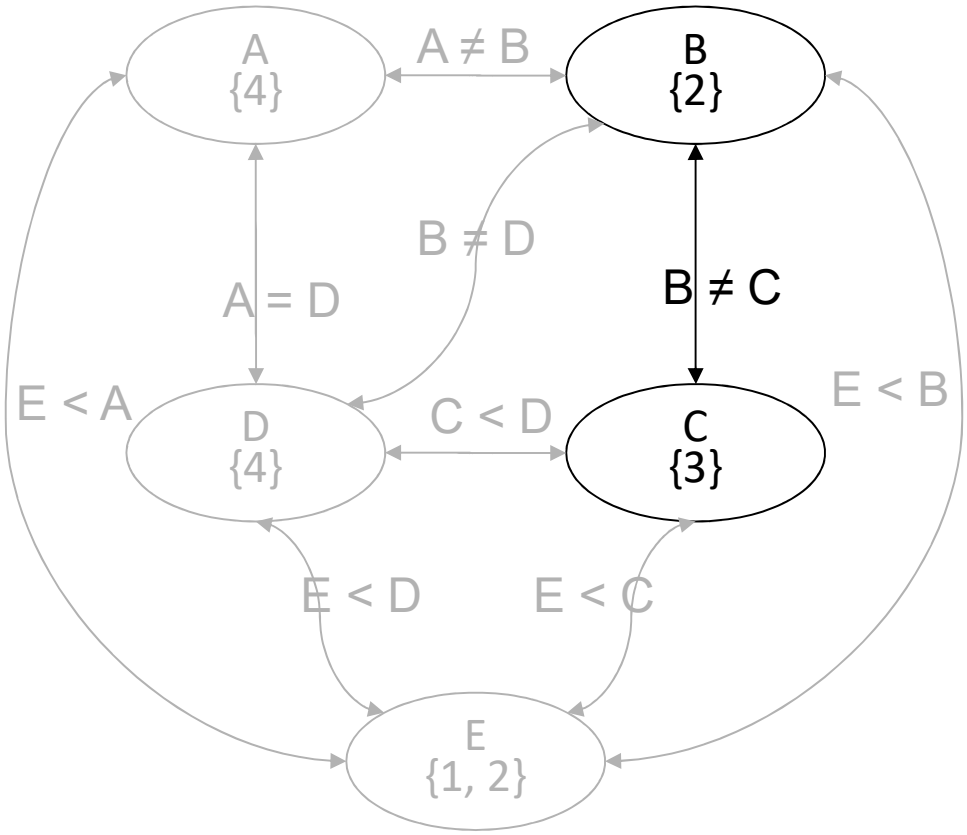
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, C> retira 2 e 3 de D insere os arcos <A,D>, <B,D>, <E,D>
  - ✓ <A, E> Ok
  - ✓ <B, E> Ok
  - ✓ <D, E> Ok
  - ✓ <A, D> retira 2 e 3 de A insere os arcos <B,A>, <E,A>
  - ✓ <B, D> retira de 4 de B insere os arcos <A,B>, <C,B>, <E,B>
  - ✓ <E, D> Ok
  - ✓ <B, A> Ok
  - ✓ <E, A> Ok
  - **<A, B>**
  - <C, B>
  - <E, B>



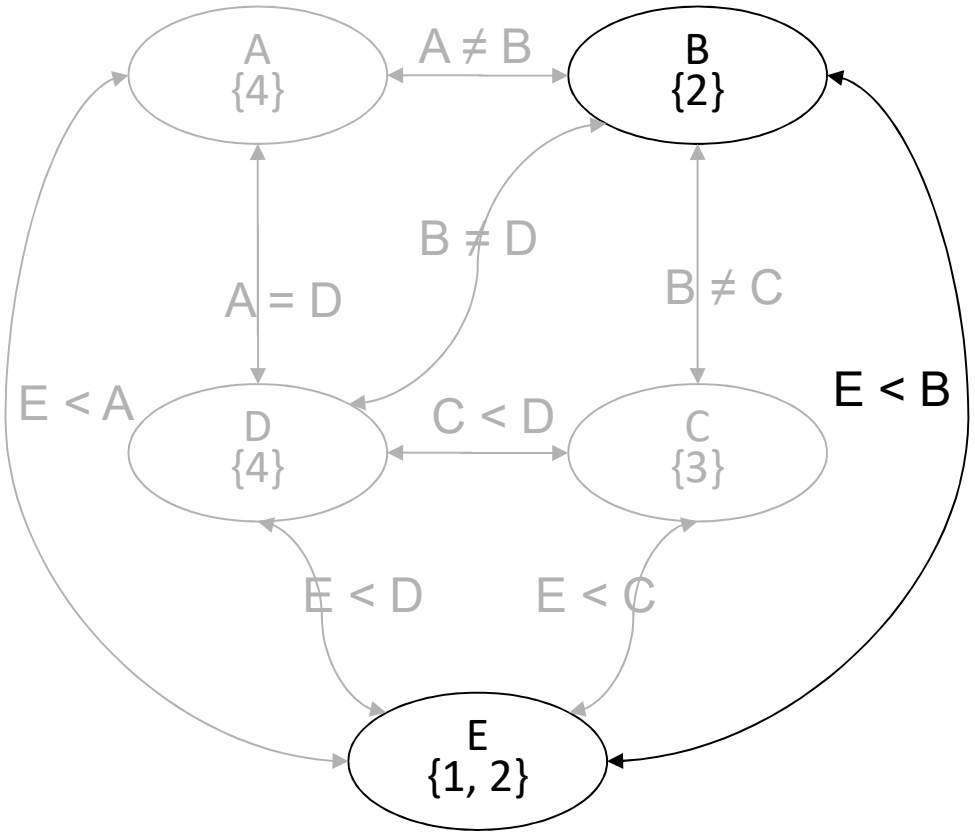
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <D, C> retira 2 e 3 de D insere os arcos <A,D>, <B,D>, <E,D>
  - ✓ <A, E> Ok
  - ✓ <B, E> Ok
  - ✓ <D, E> Ok
  - ✓ <A, D> retira 2 e 3 de A insere os arcos <B,A>, <E,A>
  - ✓ <B, D> retira de 4 de B insere os arcos <A,B>, <C,B>, <E,B>
  - ✓ <E, D> Ok
  - ✓ <B, A> Ok
  - ✓ <E, A> Ok
  - ✓ <A, B> Ok
  - **<C, B>**
  - <E, B>



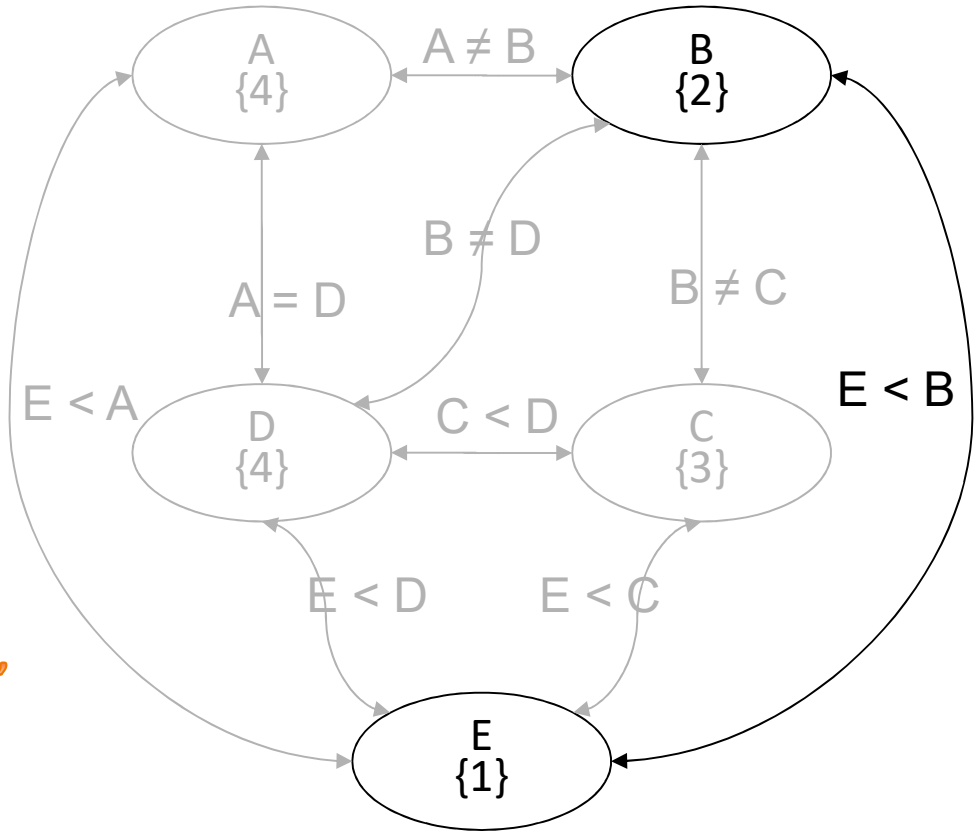
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <A, D> retira 2 e 3 de A insere os arcos <B,A>, <E,A>
  - ✓ <B, D> retira de 4 de B insere os arcos <A,B>, <C,B>, <E,B>
  - ✓ <E, D> Ok
  - ✓ <B, A> Ok
  - ✓ <E, A> Ok
  - ✓ <A, B> Ok
  - ✓ <C, B> Ok
  - **<E, B>**



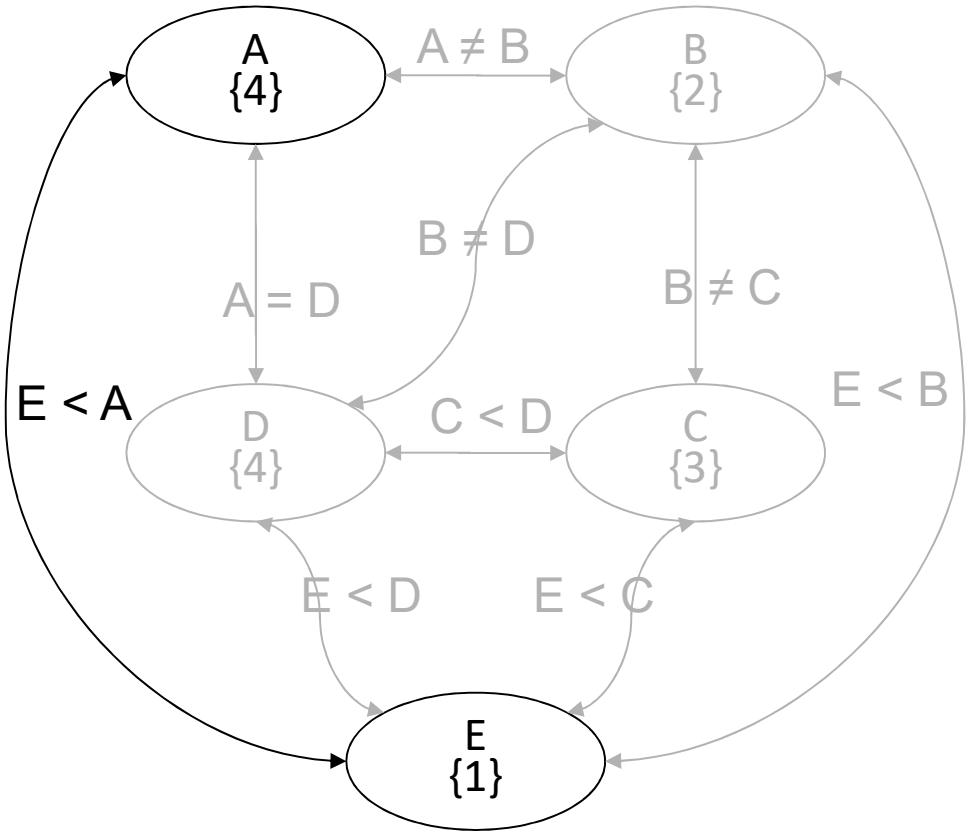
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <A, D> retira 2 e 3 de A insere os arcos <B,A>, <E,A>
  - ✓ <B, D> retira de 4 de B insere os arcos <A,B>, <C,B>, <E,B>
  - ✓ <E, D> Ok
  - ✓ <B, A> Ok
  - ✓ <E, A> Ok
  - ✓ <A, B> Ok
  - ✓ <C, B> Ok
  - **<E, B> retira 2 de E insere os arcos <A,E>, <C,E>, <D,E>**
  - **<A, E>**
  - **<C, E>**
  - **<D ,E>**



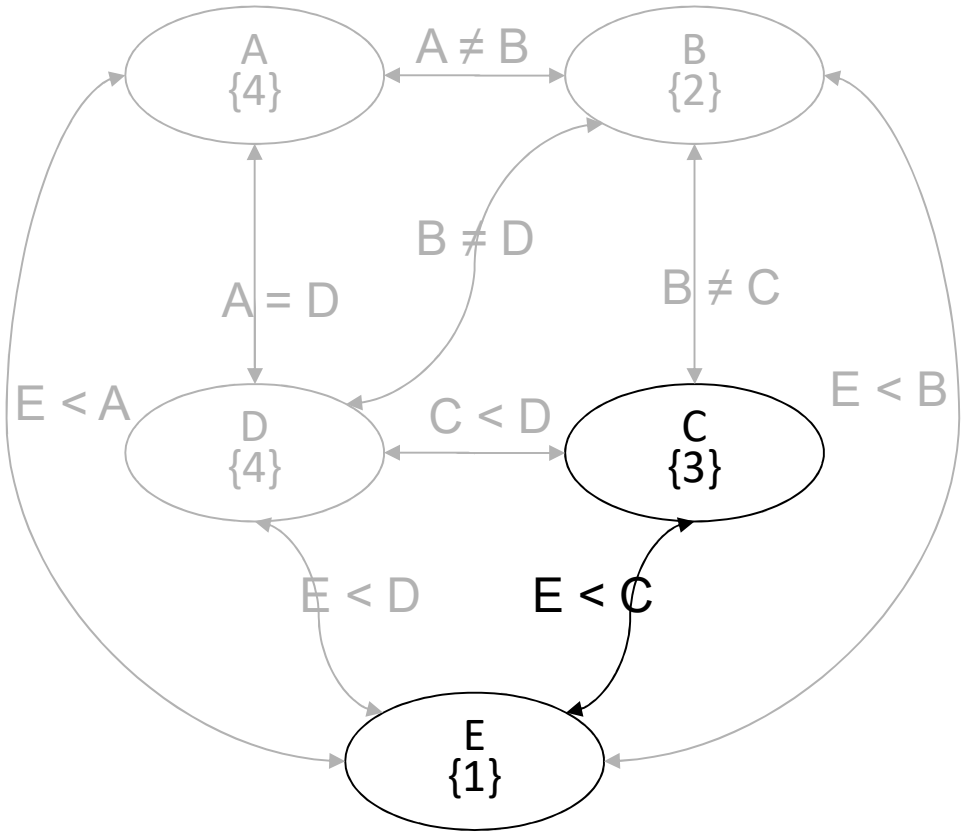
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <A, D> retira 2 e 3 de A insere os arcos <B,A>, <E,A>
  - ✓ <B, D> retira de 4 de B insere os arcos <A,B>, <C,B>, <E,B>
  - ✓ <E, D> Ok
  - ✓ <B, A> Ok
  - ✓ <E, A> Ok
  - ✓ <A, B> Ok
  - ✓ <C, B> Ok
  - ✓ <E, B> retira 2 de E insere os arcos <A,E>, <C,E>, <D,E>
  - **<A, E>**
  - <C, E>
  - <D ,E>



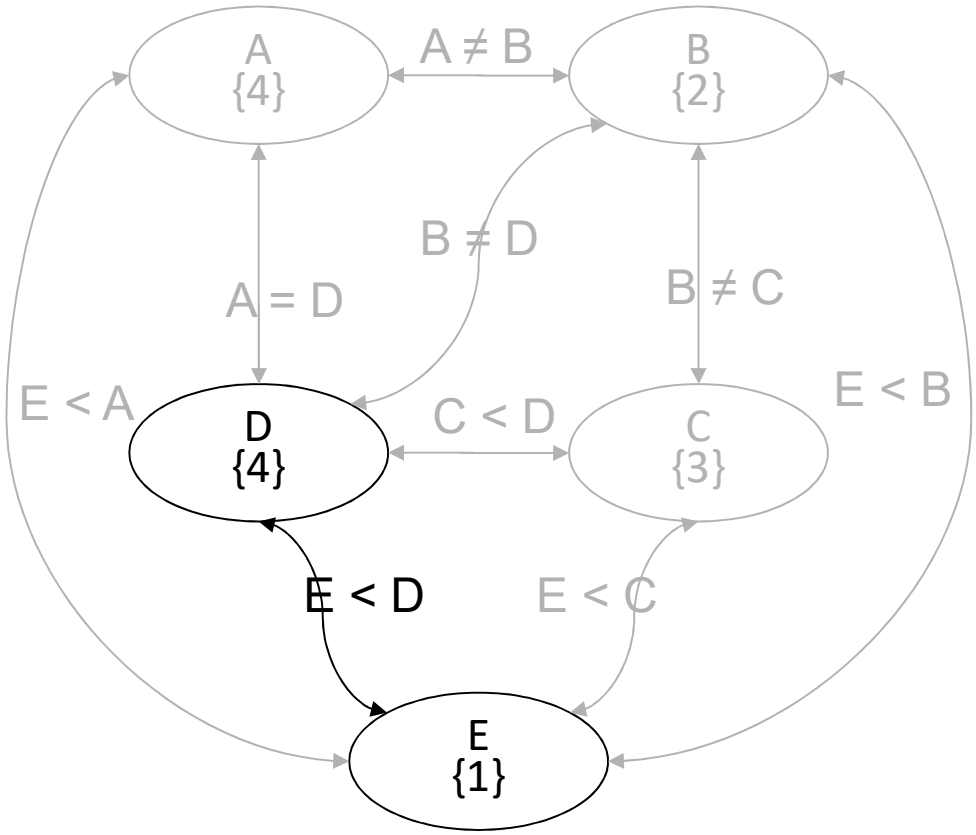
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <A, D> retira 2 e 3 de A insere os arcos <B,A>, <E,A>
  - ✓ <B, D> retira de 4 de B insere os arcos <A,B>, <C,B>, <E,B>
  - ✓ <E, D> Ok
  - ✓ <B, A> Ok
  - ✓ <E, A> Ok
  - ✓ <A, B> Ok
  - ✓ <C, B> Ok
  - ✓ <E, B> retira 2 de E insere os arcos <A,E>, <C,E>, <D,E>
  - ✓ <A, E> Ok
  - **<C, E>**
  - <D ,E>



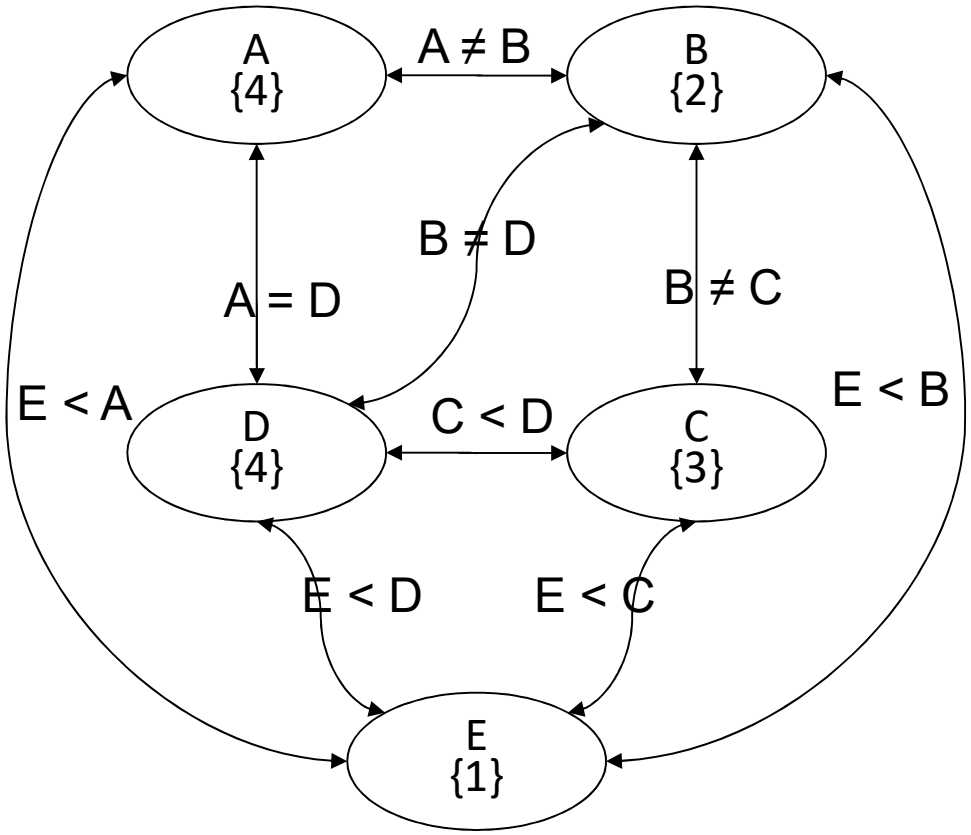
Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <A, D> retira 2 e 3 de A insere os arcos <B,A>, <E,A>
  - ✓ <B, D> retira de 4 de B insere os arcos <A,B>, <C,B>, <E,B>
  - ✓ <E, D> Ok
  - ✓ <B, A> Ok
  - ✓ <E, A> Ok
  - ✓ <A, B> Ok
  - ✓ <C, B> Ok
  - ✓ <E, B> retira 2 de E insere os arcos <A,E>, <C,E>, <D,E>
  - ✓ <A, E> Ok
  - ✓ <C, E> Ok
  - **<D ,E>**



Algoritmo de consistência de arcos AC-3

- TDA
  - ✓ <A, D> retira 2 e 3 de A insere os arcos <B,A>, <E,A>
  - ✓ <B, D> retira de 4 de B insere os arcos <A,B>, <C,B>, <E,B>
  - ✓ <E, D> Ok
  - ✓ <B, A> Ok
  - ✓ <E, A> Ok
  - ✓ <A, B> Ok
  - ✓ <C, B> Ok
  - ✓ <E, B> retira 2 de E insere os arcos <A,E>, <C,E>, <D,E>
  - ✓ <A, E> Ok
  - ✓ <C, E> Ok
  - ✓ <D ,E> Ok



Quando aplicar o AC-3

- Antes de iniciar a busca, ou seja, antes de atribuir qualquer variável.
  - Se algum domínio se tornar vazio em qualquer momento da aplicação do AC-3, então não existe solução para o problema.
- A cada atribuição (como na verificação prévia) para tornar novamente os arcos consistentes com última atribuição
  - Se algum domínio se tornar vazio em qualquer momento da aplicação do AC-3, então temos que fazer retrocesso e testar um outro valor para alguma variável já atribuída.

Resultados do AC-3

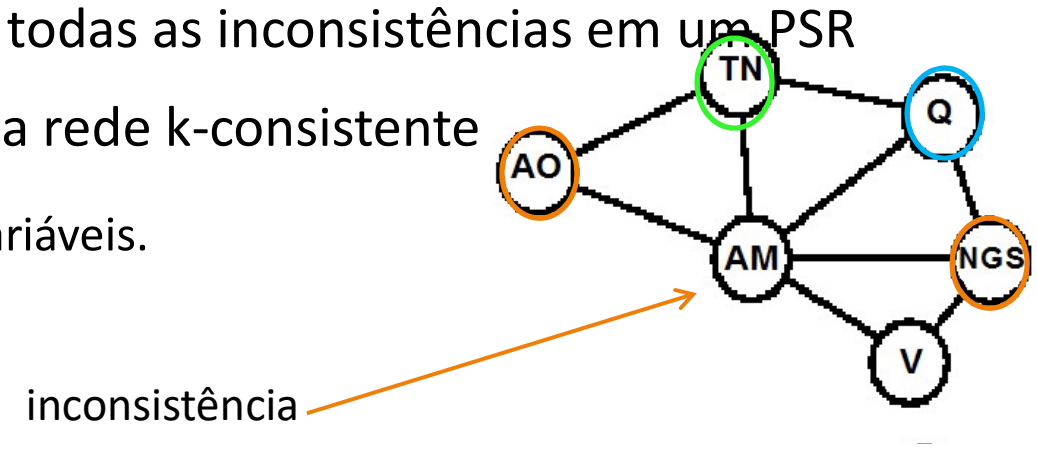
- Se ao final da aplicação do AC-3
  - Cada domínio tiver somente um valor, então existe somente uma solução para o problema e ela já foi encontrada pela aplicação do AC-3
  - Alguns domínios tem mais de um valor, então pode existir mais de uma solução para o problema
    - Devemos atribuir um valor para uma variável e aplicar AC-3 novamente

Domínios que terminam com mais de um valor

- Se um domínio terminar com os valores x e y
  - Não basta simplesmente fazer uma solução contendo o valor x e outra contendo o valor y
  - A soluções podem não ser válidas quando o domínio for dividido
    - Um grafo contendo somente x (ou y) pode não ser consistente
- Solução: Dividir um domínio com mais de um valor e aplicar recursivamente o AC-3.
  - É melhor dividir o menor domínio (heurística de valores restantes mínimos)
  - Inicialmente todos os arcos são arco-consistente exceto aqueles que apontam para o domínio que foi dividido
    - TDA inicial = arcos que apontam para o domínio divido

Desempenho do AC-3

- AC-3 **não consegue detectar todas as inconsistências**
  - Por exemplo, a atribuição parcial {AO = R, NGS = R} para o problema do mapa da Austrália é inconsistente, mas o AC-3 não a encontrará.
- AC-3 verifica a consistência de variáveis duas a duas
  - Por isso, ele não detecta todas as inconsistências.
- Uma forma de detectarmos todas as inconsistências em um PSR com k variáveis seria tornar a rede k-consistente
  - Consistente entre todas as variáveis.



Complexidade do AC-3

- Se cada variável tem domínio de tamanho ***d*** e ***e*** relações a serem testadas
  - AC-3 tem complexidade $O(ed^3)$
  - Redes de restrições, na forma de árvore AC-3, são lineares no número de variáveis

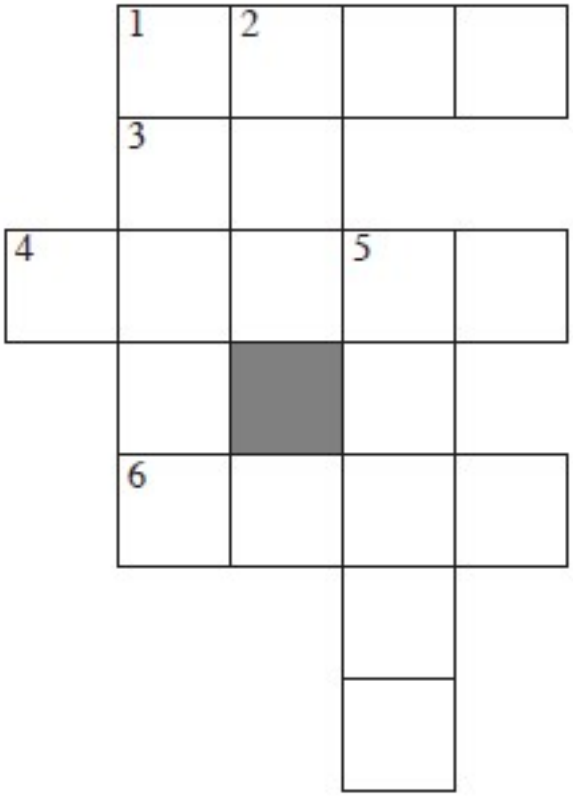
Revisitando a Satisfação de Restrição

- Um problema de satisfação de restrição consiste de:
  - Um conjunto de variáveis
  - Um conjunto de valores possíveis, um domínio para cada variável
  - Um conjunto de restrições entre subconjuntos das variáveis

- O objetivo é encontrar um conjunto de atribuições que satisfaça todas as restrições, ou encontrar todos as atribuições.

Exemplo: Palavras cruzadas

- at, be, he, it, on,
- eta, hat, her, him,
- one,
- desk, dove, easy,
- else, help, kind,
- soon, this,
- dance, rst, fuels,
- given, haste, loses,
- sense, sound, think,
- usage



Representações para palavras cruzadas

- Duas maneiras de representar as palavras cruzadas como um PSR:
  - Primeira representação:
    - Nós representam posições da palavra: 1-para_baixo... 6-para_lado
    - Domínios são as palavras
    - Restrições especificam que as letras nas interseções devem ser as mesmas.
  - Segunda representação:
    - Nós representam os quadrados individuais.
    - Domínios são as letras
    - Restrições especificam que as palavras devem se adequar ao espaço

Eliminação de Variáveis

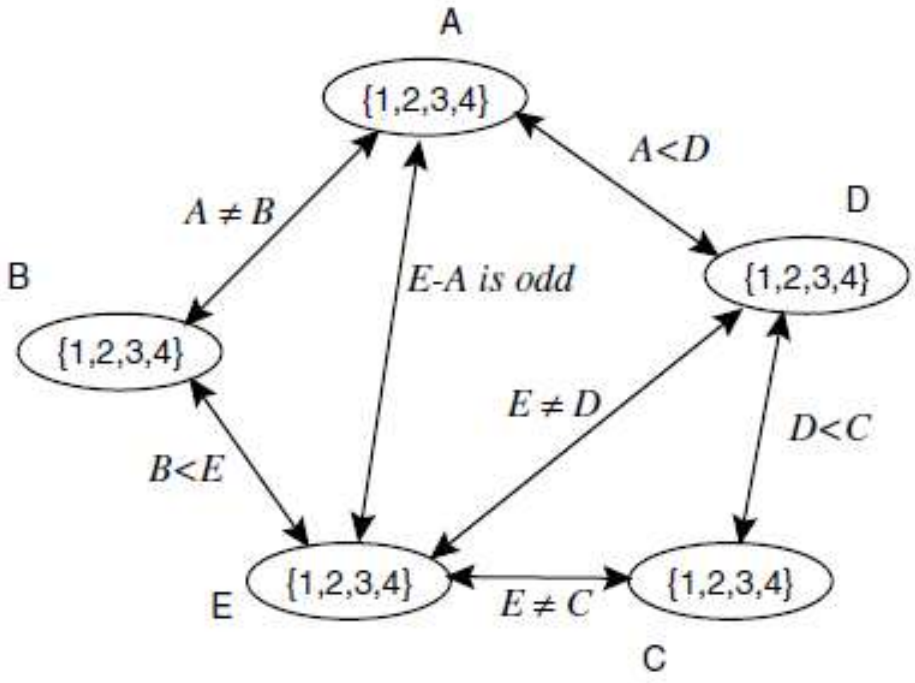
- Idéia: eliminar as variáveis uma por uma passando suas restrições aos seus vizinhos.

- **Algoritmo de eliminação de variável**:
  - Se há apenas uma variável, retorne a interseção das restrições (unárias) que a contêm.
  - Selecionar uma variável X.
  - Fundir as restrições na qual X aparece, formando a restrição R_1.
  - Projetar R1 para suas variáveis com exceção de X, formando R_2.
  - Substituir todas as restrições na qual X_i aparece por R_2.
  - Recursivamente, resolver o problema simplificado, formando R_3.
  - Retornar R_1 fundida com R_3.

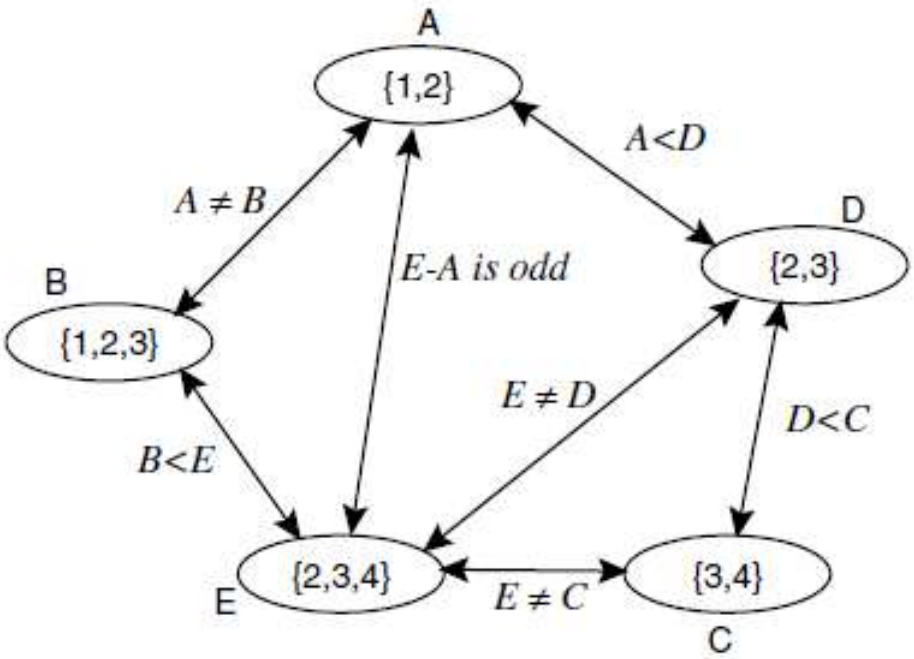
Eliminação de Variáveis (cont.)

- Quando há somente uma única variável restante, se ela não tiver nenhum valor, a rede é inconsistente.
- As variáveis são eliminadas de acordo com alguma ordem de eliminação.
- Ordens de eliminação diferentes resultam em restrições intermediárias de diferentes tamanhos.

Exemplo: Rede de restrições



Exemplo: Rede Arco-consistente (após AC3)



Domínios reduzidos

Exemplo: eliminando C

$r_2 : C > D$	C	D
	3	2
	4	2
	4	3

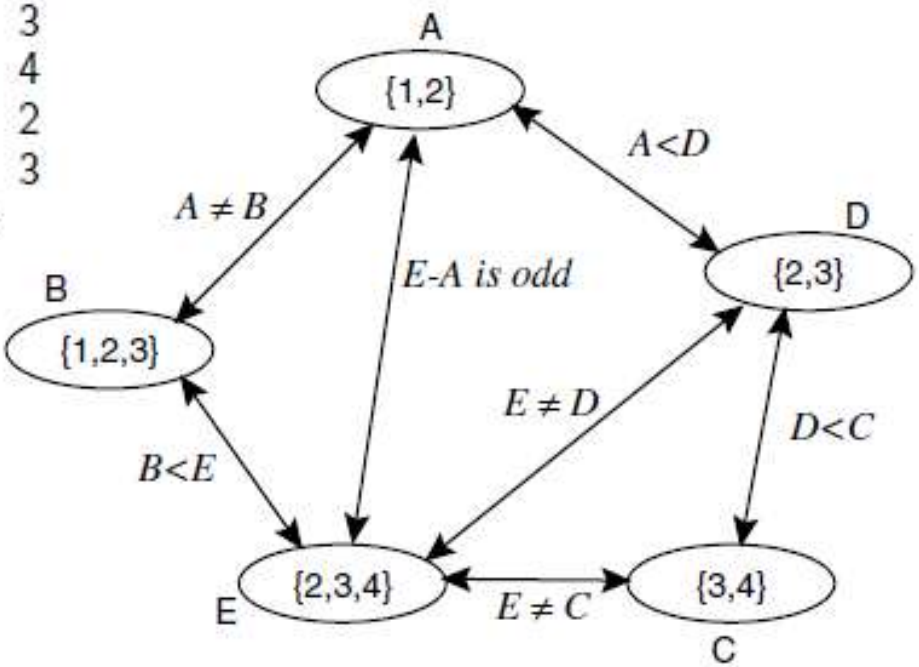
$r_1 : C \neq E$	C	E
	3	2
	3	4
	4	2
	4	3

$r_3 : r_1 \bowtie r2$	C	D	E
	3	2	2
	3	2	4
	4	2	2
	4	2	3
	4	3	2
	4	3	3

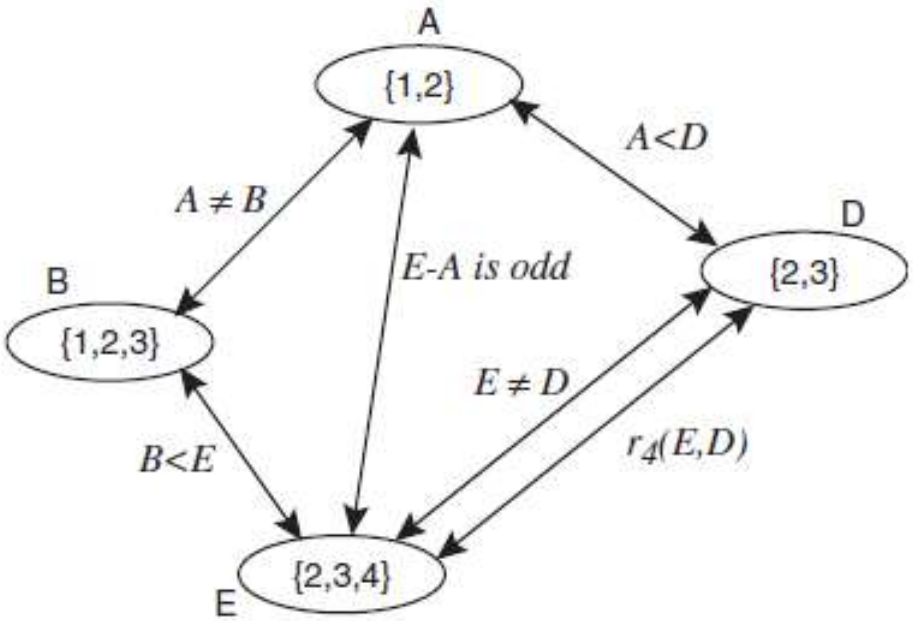
$r_4 : \pi_{\{D,E\}} r3$	D	E
	2	2
	2	3
	2	4
	3	2
	3	3

$\hookrightarrow$ new constraint

A constraint resultante não é uma regra, é apenas uma relação π{D,E} da restrição *r*3



Rede resultante após a eliminação de C



Tratando restrições especiais

- Certos tipos de restrições ocorrem com frequência em problemas reais.
- Podem ser tratados com algoritmos específicos, mais eficientes do que os algoritmos de uso geral.
- Exemplos:
  - Restrições *TodosDiferentes*
    - Problema de Criptoaritmética: todas as variáveis devem ter valores distintos.
  - Restrições *noMáximo*
    - Restrições de recurso: realizar alguma tarefa com no máximo ***X*** de recursos.

Restrições *TodosDiferentes*

- Detecção:
  - se existem m variáveis envolvidas na restrição.
  - se elas tem n valores totalmente distintos possíveis.
  - e se m>n, então a restrição não pode ser satisfeita.

- Para resolver:
  - Remova da restrição qualquer variável que tenha o domínio unitário.
  - Elimine o valor desta variável dos domínios das variáveis restantes.
  - Se, em qualquer instante for produzido um domínio vazio, ou existirem mais variáveis do que valores de domínios restantes.
  - Então existe uma inconsistência.

Outro problema com restrições todosDiferentes: Sudoku

- Qual é o número na posição com ?

				3				8
			8	5	4	1		
	2		7		9	5		
	9	7				4	1	
8	1						5	6
	6	2				8	3	
	5	4	6		7			
			4	9	5	2		
7				2				

- O problema do Sudoku é tipicamente um problema de satisfação de restrição
  - Apesar de muitos não saberem !!!
- Uma tábua de sudoku é composta de 81 quadrados
- Utiliza restrições todosDiferentes
- Se usar matemática pode ficar muito mais fácil resolver:
  - Quem sabe a resposta?

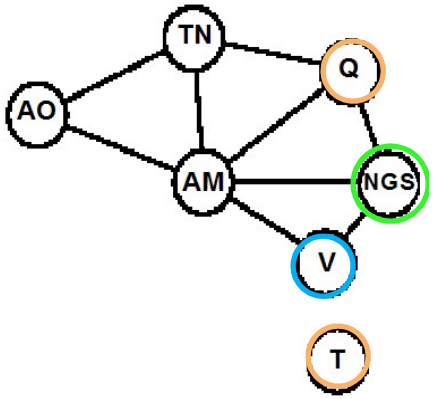
Restrições *noMáximo*

- Exemplo:
  - Sejam $PA_1,\ldots,PA_4$ os números de pessoas atribuídos a cada uma de 4 tarefas.
  - Temos a restrição de que não mais de 10 pessoas sejam atribuídas no total *noMáximo*(10, PA_1, PA_2, PA_3, PA_4).

- Detecção:
  - Podemos simplesmente verificar a soma dos valores mínimos de cada tarefa.
  - Se esta soma for mais do que o máximo permitido, temos uma inconsistência.
  - Também podemos excluir o valor máximo de qualquer domínio se ele não for consistente com os valores mínimos dos outros domínios.

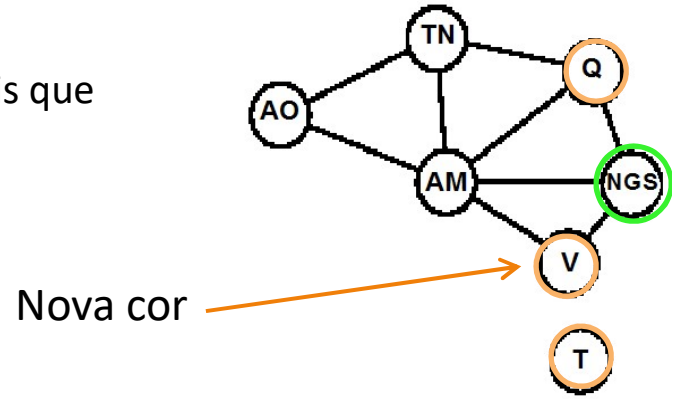
Retrocesso Inteligente

- BUSCA-COM-RETROCESSO: falha -> volta para a variável precedente.
  - Mas isto pode ser totalmente inútil.
- Exemplo: suponha as variáveis na ordem: Q, NGS, V, T, AM, AO, TN.
  - Supõe que geramos a atribuição {Q=R, NGS=G, V=B, T=R}.
  - Quando experimentamos AM, todo valor viola uma restrição.
  - Voltamos até T e experimentamos uma nova cor.
  - Obviamente, a mudança da cor atribuída a T não resolve o problema c
  - Isto acontece porque T não está no **conjunto de conflito** de AM.



Conjunto de Conflito

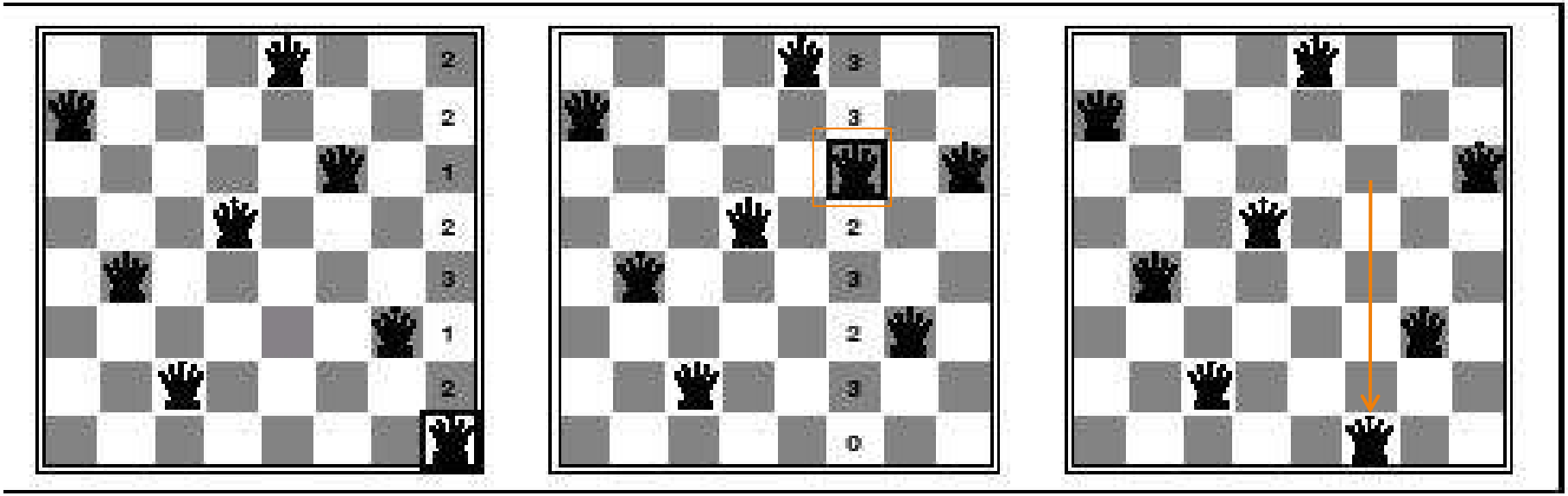
- É o conjunto de variáveis que causaram a falha.
  - Conjunto de conflito de AM={Q, NGS, V}.
- Conjunto de conflito de uma variável X são as variáveis previamente atribuídas que estão conectadas a X por uma restrição.
- **Método de retorno orientado por conflito.**
  - Volta até a variável mais **recente** do conjunto de conflito.
  - Experimenta um novo valor para ela.
  - Tenta resolver o problema mudando o valor das variáveis que causaram o problema.



Busca local para PSRs

- Utilizam uma formulação de estados completos.
- No estado inicial todas as variáveis tem um valor atribuído.
  - Mesmo que estes valores violem alguma restrição.
- Função sucessor funciona **alterando** o valor de uma variável de cada vez, para diminuir o número de restrições violadas.
  - Escolha da variável: qualquer uma com conflito escolhida aleatoriamente.
  - Escolha de um novo valor para uma variável: **Heurística de conflitos mínimos**.
- Exemplo: Problema das N-rainhas.
  - Estado inicial: configuração aleatória das N rainhas uma em cada coluna.
  - Função sucessor: escolhe uma rainha e considera a possibilidade de movê-la para outro lugar na mesma coluna.

Busca Local para Problema das n-rainhas



Heurística: Algoritmo de Conflitos Mínimos


```
def CONFLITOS-MÍNIMOS(psr, max_etapas):

    corrente ← uma atribuição inicial completa para o psr
    for i=1 to max_etapas:
        if corrente é uma solução para psr: return corrente
        var ← uma variável em conflito escolhida ao acaso a partir de
          VARIÁVEIS[psr]
        valor ← o valor v para var que minimiza CONFLITOS(var, v,
          corrente, psr)
        definir var = valor em corrente
    return falha
```

Heurística de conflitos mínimos para PSRs

- É surpreendente quando recebe um estado inicial razoável.
- O tempo de execução para problema das N-rainhas é praticamente independente do tamanho do problema.
  - Resolve até mesmo o problema de 1 milhão de rainhas em uma média de 50 etapas.
- A heurística é utilizada na programação de observações do telescópio Hubble.
  - Reduzindo o tempo necessário para programar uma semana de observações de três semanas para + ou – 10 minutos.

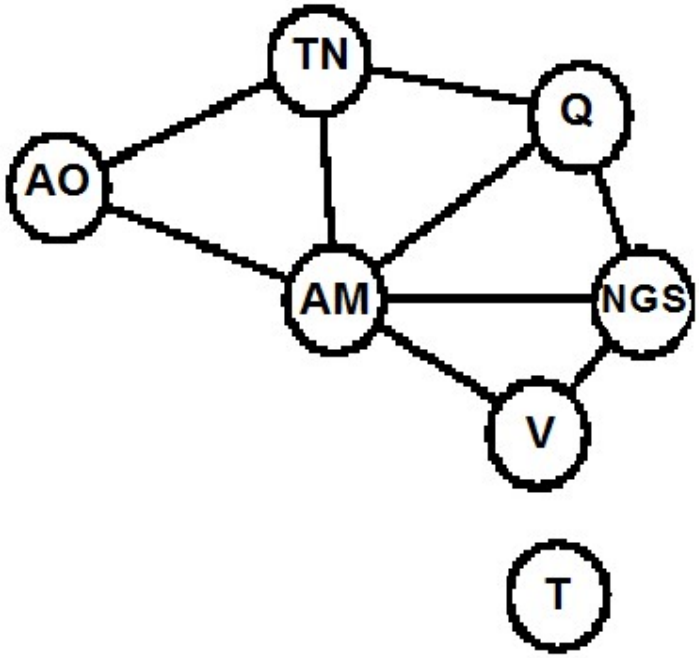
Heurística de conflitos mínimos

- Vantagem da busca local: utilizá-la em configuração *on-line* quando o problema se altera.
  - Importante para problemas de escalonamento.

- Exemplo: Escala de linhas aéreas para uma semana.
  - Pode envolver milhares de voos.
  - Dezenas de milhares de atribuições de pessoas.
  - Mas um mau tempo em um aeroporto pode torná-la inviável.
  - Gostaríamos de reparar a escala com um número mínimo de mudanças.
    - Solução: utilizar um algoritmo de busca local a partir da escala atual.

Podemos tirar vantagens da estrutura do problema?

- A Tasmânia e o território principal são **problemas independentes.**
- Pelo grafo de restrições podemos detectar os componentes conectados.
- Suponha cada subproblema tenha **c** variáveis das **n** totais com **d** valores possíveis.
  - O custo da solução no pior caso cai de d^n para $(n/c).d^c$
  - Para n=80, c=20 e d=2, o custo cai de 2^{80} para 4.2^{20}
- Subproblemas completamente independentes são interessantes, mas raros.



PSR estruturado em árvore

- Teorema: "Se o grafo de restrições não tem laços, o PSR pode ser resolvido em O(nd^2)".

- Algoritmo para resolver um PSR estruturado em árvore:

```
1) Escolha qualquer variável para a raiz da árvore e
   ordene-as da raiz até as folhas
2) Para j de n decrescendo até 2, aplique a
   Consistência do Arco em (Xi,Xj)*, removendo valores
   do Domínio[Xi] caso necessário
3) Para j de 1 até n, atribua qualquer valor a Xj
   consistente com o valor atribuído a Xi
* Onde Xi é o pai de Xj
```

PSR quase estruturado em árvore

- Se o PSR não for uma árvore, podemos construir uma árvore a partir do grafo de restrições que represente o PSR.

- Algoritmo para obtenção de um PSR estruturado em árvore:

1) Escolha e pode uma ou mais variáveis de modo que o grafo de restrições torne-se uma árvore*

2) Para cada atribuição possível ao conjunto de corte *S* que satisfaz todas as suas restrições faça:

2.1) Remova do domínio das variáveis restantes qualquer valor que seja inconsistente com a atribuição para *S*, e

2.2) Se o PSR resultante tem uma solução, então retorne-a junto com a atribuição para *S*

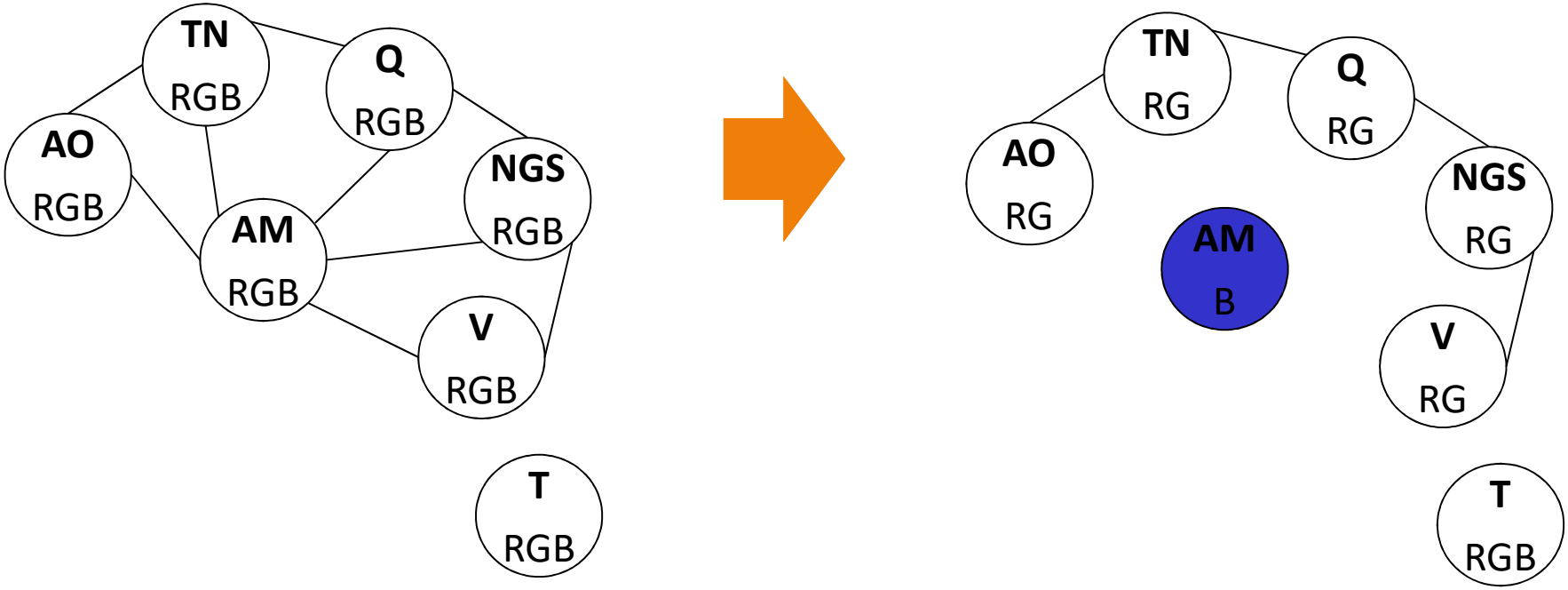
*** As variáveis podadas do grafo constituem o conjunto de corte *S***

Bibliografia

- RUSSEL, Stuart Jonathan; NORVIG, Peter. Inteligência Artificial. Tradução por Regina Célia Simille, Artificial intelligence, 3rd ed. Rio de Janeiro: Elsevier, 2013.

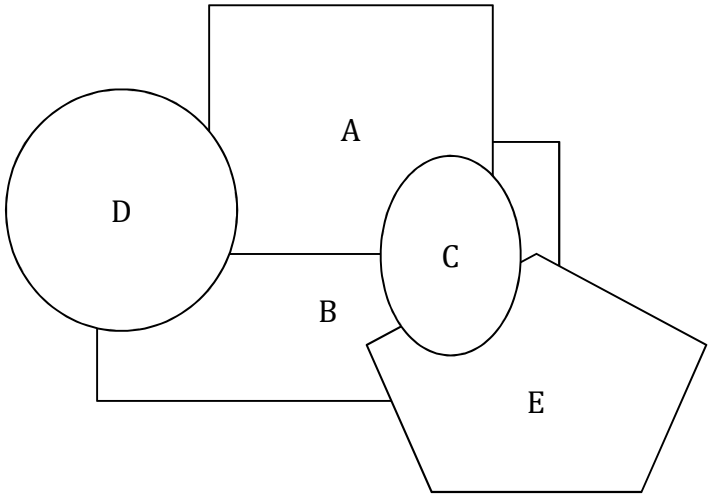
Exemplo: Problema do mapa da Austrália

- Conjunto de corte de tamanho *c*:
- Tempo de execução é de $O(d^c.(n\text{-}c)\ d^2)$
- Grafo muito parecido com uma árvore ⇒ muito rápido para *c* pequeno



EXERCÍCIOS

- Considere o problema de colorir o mapa mostrado ao lado com três cores (**V**ermelho, **P**reto e **L**aranja) e que regiões adjacentes devem ter cores diferentes.



EXERCÍCIOS (cont)

- Faça o grafo de restrições para o problema, colocando as restrições nos arcos e o domínio de cada variável.

- Faça a busca empregando as heurísticas de:
  - Variável mais restrita
  - Grau
  - Valor menos restritivo

EXERCÍCIOS (cont)

- **Considere a atribuição parcial {A=V, E=P}.**

  - Redesenhe o grafo de restrições para o problema, colocando as restrições nos arcos e o domínio de cada variável, já considerando a atribuição parcial acima.

  - Aplique o algoritmo AC-3, para tornar o grafo anterior arco-consistente. Revise os arcos na ordem abaixo, escrevendo para cada arco se ele está ok; ou qual valor do domínio analisado deve ser retirado e os arcos que devem ser revisados. Siga o exemplo:

EXERCÍCIOS (cont)

- <A, B>
- <A, C>
- <A, D>
- <B, A>
- <B, C>
- <B, D>
- <B, E>
- <C, A>
- <C, B>
- <C, E>
- <D, A>
- <D, B>
- <E, B>
- <E, C>

Outros links (Sudoku)

- http://www.ic.unicamp.br/~rocha/teaching/2011s1/mc906/trabalhos/tp/tp-gr-07.pdf
  - Backtracking com melhorias :
    - Fowarding Checking
    - Constraint Propagation
    - Minimum Remaining Values:
- Benchmarks Sudoku:
  - FTP: http://www.cse.unsw.edu.au/~fpt09/comp/benchmarks.html
  - Ordem 3: http://kjell.haxx.se/sudoku